

Std Code Library(Temp version1.0)

tingyx

Nanjing University of Science and Technology

October 19, 2024

Contents

二分图	2
串串	6
CDQ 解决高维偏序	20
dsu on tree	25
LCT	28
分块	45
Persistent Data Structure	53
Tree Dp	61
树状数组	70
数位 DP	71
math	72
图论	82
Segment Tree	90
线性 dp	120

二分图

Graph Coloring I(奇环判断染色)

```
1  #include <bits/stdc++.h>
2  #define endl '\n'
3  using namespace std;
4  typedef long double db;
5  typedef long long ll;
6
7  const ll N = 3e5 + 10;
8  const ll mod = 998244353;
9  const ll inf32 = 0x3f3f3f3f;
10 const ll inf64 = 5e18;
11
12 vector<int> G[N];
13
14 bool col[N], err;
15 int st[N], pos[N], top;
16
17 void dfs(int x, int fa, int c){
18     if (!err && pos[x] && pos[x] < top && col[x] != c){
19         err = 1;
20         cout << top - pos[x] + 1 << endl;
21         for (int i = pos[x]; i <= top; ++i)
22             cout << st[i] << " \n"[i == top];
23     }
24     if (err || pos[x]) return;
25     st[++top] = x;
26     pos[x] = top;
27     col[x] = c;
28     for (auto y : G[x]){
29         if (y == fa) continue;
30         dfs(y, x, c ^ 1);
31     }
32     --top;
33 }
34
35 void solve(){
36     int n, m;
37     cin >> n >> m;
38     for (int i = 1; i <= m; ++i){
39         int u, v;
40         cin >> u >> v;
41         G[u].push_back(v);
42         G[v].push_back(u);
43     }
44     dfs(1, 0, 0);
45     if (!err){
46         cout << 0 << endl;
47         for (int i = 1; i <= n; ++i) cout << col[i] << " \n"[i == n];
48     }
49 }
50
51 signed main(){
52     ios::sync_with_stdio(false), cin.tie(0), cout.tie(0);
53     int t = 1;
54     //cin >> t;
55     while(t--) solve();
56     return 0;
57 }
```

增广路算法(矩阵游戏)

矩阵进行行列交换，能否把对角线都变成 1

```
1  #include <bits/stdc++.h>
2  #define endl '\n'
3  using namespace std;
4  typedef long double db;
5  typedef long long ll;
```

```

6
7  const ll N = 2e5 + 10;
8  const ll mod = 998244353;
9  const ll inf32 = 0x3f3f3f3f;
10 const ll inf64 = 5e18;
11
12 struct augment_path
13 {
14     vector<vector<int>> g;
15     vector<int> pa; // 匹配
16     vector<int> pb;
17     vector<int> vis; // 访问
18     int n, m;      // 两个点集中的顶点数量
19     int dfn;       // 时间戳记
20     int res;       // 匹配数
21
22     augment_path(int _n, int _m) : n(_n), m(_m)
23     {
24         assert(0 <= n && 0 <= m);
25         pa = vector<int>(n, -1);
26         pb = vector<int>(m, -1);
27         vis = vector<int>(n);
28         g.resize(n);
29         res = 0;
30         dfn = 0;
31     }
32
33     void add(int from, int to)
34     {
35         assert(0 <= from && from < n && 0 <= to && to < m);
36         g[from].push_back(to);
37     }
38
39     bool dfs(int v)
40     {
41         vis[v] = dfn;
42         for (int u : g[v])
43         {
44             if (pb[u] == -1)
45             {
46                 pb[u] = v;
47                 pa[v] = u;
48                 return true;
49             }
50         }
51         for (int u : g[v])
52         {
53             if (vis[pb[u]] != dfn && dfs(pb[u]))
54             {
55                 pa[v] = u;
56                 pb[u] = v;
57                 return true;
58             }
59         }
60         return false;
61     }
62
63     int solve()
64     {
65         while (true)
66         {
67             dfn++;
68             int cnt = 0;
69             for (int i = 0; i < n; i++)
70             {
71                 if (pa[i] == -1 && dfs(i))
72                 {
73                     cnt++;
74                 }
75             }
76             if (cnt == 0)

```

```

77         {
78             break;
79         }
80         res += cnt;
81     }
82     return res;
83 }
84 };
85
86 void solve()
87 {
88     int n;
89     cin >> n;
90     augment_path ap(n, n);
91     for (int i = 0; i < n; ++i)
92         for (int j = 0; j < n; ++j){
93             int x;
94             cin >> x;
95             if (x) ap.g[i].push_back(j);
96         }
97     int res = ap.solve();
98     if (res == n){
99         cout << "Yes" << endl;
100        return;
101    }
102    cout << "No" << endl;
103 }
104
105 signed main()
106 {
107     ios::sync_with_stdio(false), cin.tie(0), cout.tie(0);
108     int t = 1;
109     cin >> t;
110     while (t--)
111         solve();
112     return 0;
113 }

```

3. 宿舍的假期 (简略版 AG)

```

1  #include <bits/stdc++.h>
2  #define endl '\n'
3  using namespace std;
4  typedef long double db;
5  typedef long long ll;
6
7  const ll N = 2e5 + 10;
8  const ll mod = 998244353;
9  const ll inf32 = 0x3f3f3f3f;
10 const ll inf64 = 5e18;
11
12 struct augment_path
13 {
14     vector<vector<int>> g;
15     vector<int> pa; // 匹配
16     vector<int> pb;
17     vector<int> vis; // 访问
18     int n, m; // 两个点集中的顶点数量
19     int dfn; // 时间戳记
20     int res; // 匹配数
21
22     augment_path(int _n, int _m) : n(_n), m(_m)
23     {
24         assert(0 <= n && 0 <= m);
25         pa = vector<int>(n, -1);
26         pb = vector<int>(m, -1);
27         vis = vector<int>(n);
28         g.resize(n);
29         res = 0;
30         dfn = 0;
31     }
32 }

```

```

33 void add(int from, int to)
34 {
35     assert(0 <= from && from < n && 0 <= to && to < m);
36     g[from].push_back(to);
37 }
38
39 bool dfs(int v)
40 {
41     vis[v] = dfn;
42     for (int u : g[v])
43     {
44         if (pb[u] == -1)
45         {
46             pb[u] = v;
47             pa[v] = u;
48             return true;
49         }
50     }
51     for (int u : g[v])
52     {
53         if (vis[pb[u]] != dfn && dfs(pb[u]))
54         {
55             pa[v] = u;
56             pb[u] = v;
57             return true;
58         }
59     }
60     return false;
61 }
62
63 int solve()
64 {
65     while (true)
66     {
67         dfn++;
68         int cnt = 0;
69         for (int i = 0; i < n; i++)
70         {
71             if (pa[i] == -1 && dfs(i))
72             {
73                 cnt++;
74             }
75         }
76         if (cnt == 0)
77         {
78             break;
79         }
80         res += cnt;
81     }
82     return res;
83 }
84 };
85
86 void solve()
87 {
88     int n;
89     cin >> n;
90     augment_path ap(n, n);
91     for (int i = 0; i < n; ++i)
92         for (int j = 0; j < n; ++j){
93             int x;
94             cin >> x;
95             if (x) ap.g[i].push_back(j);
96         }
97     int res = ap.solve();
98     if (res == n){
99         cout << "Yes" << endl;
100         return;
101     }
102     cout << "No" << endl;
103 }

```

```

104
105 signed main()
106 {
107     ios::sync_with_stdio(false), cin.tie(0), cout.tie(0);
108     int t = 1;
109     cin >> t;
110     while (t--)
111         solve();
112     return 0;
113 }

```

串串

本质不同子序列数量

```

1 vector<int> dp(n + 1);
2 map<char, int> vis;
3 for (int i = pl + 1; i < pr; ++i){
4     if(!vis.count(s[i])){
5         vis[s[i]] = 1;
6         dp[i] = (dp[i - 1] * 2 + 1) % P;
7     }else{
8         dp[i] = (((dp[i - 1] * 2) % P - dp[vis[s[i]]] - 1) + P) % P + P) % P;
9     }
10    vis[s[i]] = i;
11 }

```

子序列自动机

```

1  #include <bits/stdc++.h>
2  #define endl '\n'
3  #define pll pair<i64, i64>
4  #define tll tuple<i64, i64, i64>
5  #define all(a) a.begin() + 1, a.end()
6  using namespace std;
7  using i64 = long long;
8  using db = long double;
9  const i64 N = 1e5 + 10;
10 const i64 mod = 998244353;
11 const i64 inf32 = 1e9;
12 const i64 inf64 = 5e18;
13
14 int typ, n, q, m;
15 int root[N];
16 struct PersistentTree{
17     int ls[N << 5], rs[N << 5], nxt[N << 5], tot = 0;
18     //单点修改
19     inline void update(int & rt, int old, int l, int r, int p, int k){
20         rt = ++tot;
21         ls[rt] = ls[old], rs[rt] = rs[old], nxt[rt] = nxt[old];
22         if (l == r){
23             nxt[rt] = k;
24             return;
25         }
26         int mid = l + r >> 1;
27         if (p <= mid) update(ls[rt], ls[old], l, mid, p, k);
28         else update(rs[rt], rs[old], mid + 1, r, p, k);
29     }
30     //单点查询
31     inline int query(int rt, int l, int r, int p){
32         if (l == r) return nxt[rt];
33         int mid = l + r >> 1;
34         if (p <= mid) return query(ls[rt], l, mid, p);
35         else return query(rs[rt], mid + 1, r, p);
36     }
37     //建树
38     inline void build(vector<int> a){
39         int n = a.size() - 1;
40         for (int i = n; i >= 1; --i){
41             update(root[i], root[i + 1], 1, m, a[i], i + 1);

```

```

42     }
43 }
44 }T;
45
46 void solve(){
47     cin >> typ >> n >> q >> m;
48     vector<int> a(n + 1);
49     for (int i = 1; i <= n; ++i) cin >> a[i];
50     T.build(a);
51     while (q--){
52         int k;
53         cin >> k;
54         bool ok = true;
55         int rt = 1;
56         while (k--){
57             int p;
58             cin >> p;
59             int to = T.query(root[rt], 1, m, p);
60             if (!to) ok = false;
61             if (to) rt = to;
62         }
63         if (ok) cout << "Yes" << endl;
64         else cout << "No" << endl;
65     }
66 }
67
68 int main(){
69     ios::sync_with_stdio(false), cin.tie(0), cout.tie(0);
70     int t = 1;
71     //cin >> t;
72     while(t--){ solve();
73         return 0;
74     }
75 }

```

SAM

codeforces edu101 E

```

1  #include <bits/stdc++.h>
2  #define endl '\n'
3
4  using ll = long long;
5
6  using namespace std;
7
8  constexpr int N = 1e6 + 10;
9  constexpr int mod = 998244353;
10
11 struct SuffixAutomaton {
12     struct State {
13         int len, link;
14         map<char, int> next;
15     };
16
17     vector<State> st;
18     int last;
19
20     SuffixAutomaton(const string& s) {
21         st.reserve(2 * s.size());
22         st.push_back({0, -1});
23         last = 0;
24         for (char c : s) {
25             extend(c);
26         }
27     }
28
29     void extend(char c) {
30         int cur = st.size();
31         st.push_back({st[last].len + 1});
32         int p = last;
33         while (p != -1 && !st[p].next.count(c)) {

```



```

34         st[p].next[c] = cur;
35         p = st[p].link;
36     }
37     if (p == -1) {
38         st[cur].link = 0;
39     } else {
40         int q = st[p].next[c];
41         if (st[p].len + 1 == st[q].len) {
42             st[cur].link = q;
43         } else {
44             int clone = st.size();
45             st.push_back({st[p].len + 1, st[q].link, st[q].next});
46             while (p != -1 && st[p].next[c] == q) {
47                 st[p].next[c] = clone;
48                 p = st[p].link;
49             }
50             st[q].link = st[cur].link = clone;
51         }
52     }
53     last = cur;
54 }
55
56 bool contains(const string& t) {
57     int cur = 0;
58     for (char c : t) {
59         if (!st[cur].next.count(c)) {
60             return false;
61         }
62         cur = st[cur].next[c];
63     }
64     return true;
65 }
66 };
67
68
69 void solve(){
70     int n, k;
71     string s;
72     cin >> n >> k >> s;
73     if (n == k) {
74         cout << "YES" << endl;
75         bool ok = 0;
76         for (int i = 0; i < n - 1; ++i) {
77             if (s[i] == '0') {
78                 ok = 1;
79             }
80             s[i] = '0';
81         }
82         if (ok) s[n - 1] = '0';
83         cout << s << endl;
84         return;
85     }
86     int m = log2(n - k) + 1;
87     SuffixAutomaton sam(s);
88     string ans;
89     ans.resize(k);
90     auto dfs = [&](auto self, int cur) -> bool {
91         if (cur == k) {
92             return !sam.contains(ans);
93         }
94         ans[cur] = '1';
95         if (self(self, cur + 1)) return 1;
96         ans[cur] = '0';
97         return self(self, cur + 1);
98     };
99     if (!dfs(dfs, 0)) {
100         cout << "NO" << endl;
101         return;
102     }
103     cout << "YES" << endl;
104     for (auto &x : ans) {

```

```

105         if (x == '0') x = '1';
106         else x = '0';
107     }
108     cout << ans << endl;
109 }
110
111 signed main(){
112     ios::sync_with_stdio(false);
113     cin.tie(nullptr);
114     int t = 1;
115     cin >> t;
116     while(t--) solve();
117     return 0;
118 }

```

Trie 板子

```

1  #include <bits/stdc++.h>
2  #define endl '\n'
3
4  using ll = long long;
5
6  constexpr int N = 1e5 + 10;
7  constexpr int mod = 998244353;
8
9  using namespace std;
10
11 int tr[N << 5][26], idx = 0;
12 bool vis[N << 5];
13
14 void insert(string s) {
15     int p = 0;
16     for (auto ch : s) {
17         int c = ch - 'a';
18         if (!tr[p][c]) tr[p][c] = ++idx;
19         p = tr[p][c];
20     }
21     vis[p] = 1;
22 }
23
24 vector<int> G[26];
25 int in[26];
26
27 bool query(string s) {
28     auto topo = [&]() -> bool {
29         queue<int> q;
30         int cnt = 0;
31         for (int i = 0; i < 26; ++i) if (!in[i]) q.push(i);
32         while (!q.empty()) {
33             auto u = q.front();
34             q.pop();
35             cnt++;
36             for (auto v : G[u]) {
37                 if (--in[v] == 0) q.push(v);
38             }
39         }
40         return cnt == 26;
41     };
42     int p = 0;
43     for (int i = 0; i < s.size(); ++i) {
44         int c = s[i] - 'a';
45         for (int j = 0; j < 26; ++j) {
46             if (j == c || !tr[p][j]) continue;
47             G[c].push_back(j);
48             in[j]++;
49         }
50         p = tr[p][c];
51         if (vis[p] && i != s.size() - 1) return false;
52     }
53     return topo();
54 }

```

```

55 }
56
57 void solve(){
58     int n;
59     cin >> n;
60     vector<string> s(n + 1);
61     for (int i = 1; i <= n; ++i) cin >> s[i], insert(s[i]);
62     vector<string> ans;
63     for (int i = 1; i <= n; ++i) {
64         memset(in, 0, sizeof in);
65         for (int j = 0; j < 26; ++j) G[j].clear();
66         if (query(s[i])) ans.push_back(s[i]);
67     }
68     cout << ans.size() << endl;
69     for (auto x : ans) {
70         cout << x << endl;
71     }
72 }
73
74 signed main(){
75     ios::sync_with_stdio(false);
76     cin.tie(nullptr);
77     int t = 1;
78     //cin >> t;
79     while(t--) solve();
80     return 0;
81 }

```

异或最大值

```

1  int tr[maxm][2], idx, n;
2
3  void insert(int x){
4      int p = 0;
5      for (int i = 31; i >= 0; --i){
6          int c = x >> i & 1;
7          if (!tr[p][c]) tr[p][c] = ++idx;
8          p = tr[p][c];
9      }
10 }
11
12 int query(int x){
13     int res = 0, p = 0;
14     for (int i = 31; i >= 0; --i){
15         int c = x >> i & 1;
16         if (tr[p][c ^ 1]){
17             p = tr[p][c ^ 1];
18             res += 1 << i;
19         }else{
20             p = tr[p][c];
21         }
22     }
23     return res;
24 }

```

Border Tree1

Tricks:

1. 每个前缀 i 的所有 Border 为 i 到根结点的链
2. 哪些前缀有长度为 x 的 Border: x 的子树
3. 求两个前缀的公共 Border 等价于求 LCA

本题求出现次数大于等于 k 的 border

```

1  #include <bits/stdc++.h>
2  #define endl '\n'
3
4  using ll = long long;
5
6  constexpr int N = 1e6 + 10;

```

```

7  constexpr int mod = 998244353;
8
9  using namespace std;
10
11 vector<int> G[N];
12 int sz[N];
13
14 vector<int> prefix_function(string s){
15     G[0].push_back(1);
16     int n = (int) s.length();
17     vector<int> pi(n);
18     for(int i = 2; i < n; i++) {
19         pi[i] = pi[i - 1];
20         while(pi[i] && s[i] != s[pi[i] + 1])
21             pi[i] = pi[pi[i]];
22         pi[i] += (s[i] == s[pi[i] + 1]);
23         G[pi[i]].push_back(i);
24     }
25     return pi;
26 }
27
28 void dfs(int u) {
29     sz[u] = 1;
30     for(auto v : G[u]) {
31         dfs(v);
32         sz[u] += sz[v];
33     }
34 }
35
36 void solve(){
37     int n, k;
38     cin >> n >> k;
39     string s;
40     cin >> s;
41     s = " " + s;
42     auto nxt = prefix_function(s);
43     dfs(0);
44     int u = n;
45     while (u && sz[u] < k) u = nxt[u];
46     if (!u) cout << -1 << endl;
47     else cout << s.substr(1, u) << endl;
48 }
49
50 signed main(){
51     ios::sync_with_stdio(false);
52     cin.tie(nullptr);
53     int t = 1;
54     //cin >> t;
55     while(t--) solve();
56     return 0;
57 }

```

Border Tree2

本题可以动态在字符串末尾加一个字符

```

1  #include <bits/stdc++.h>
2  #define endl '\n'
3
4  using ll = long long;
5
6  constexpr int N = 1e6 + 10;
7  constexpr int mod = 998244353;
8
9  using namespace std;
10
11 struct BIT {
12     int tr[N];
13     int lowbit(int x) {return x & -x;}
14     void add(int p, int v) {
15         for (; p < N; p += lowbit(p)) tr[p] += v;

```

```

16     }
17     int query(int p) {
18         int res = 0;
19         for (; p; p -= lowbit(p)) {
20             res += tr[p];
21         }
22         return res;
23     }
24 }tr;
25
26 vector<int> G[N];
27
28 vector<int> prefix_function(string s){
29     G[0].push_back(1);
30     int n = (int) s.length();
31     vector<int> pi(n);
32     for(int i = 2; i < n; i++) {
33         pi[i] = pi[i - 1];
34         while(pi[i] && s[i] != s[pi[i] + 1])
35             pi[i] = pi[pi[i]];
36         pi[i] += (s[i] == s[pi[i] + 1]);
37         G[pi[i]].push_back(i);
38     }
39     return pi;
40 }
41
42 pair<int, int> qry[N];
43 int sz[N], fa[N][21], dfn[N], low[N], tot;
44
45 void dfs(int u) {
46     dfn[u] = ++tot;
47     for (auto v : G[u]) {
48         fa[v][0] = u;
49         for (int i = 1; i <= 20; ++i) fa[v][i] = fa[fa[v][i - 1]][i - 1];
50         dfs(v);
51     }
52     low[u] = tot;
53 }
54
55 void solve(){
56     int n, q;
57     cin >> n >> q;
58     string s;
59     cin >> s; s = " " + s;
60     for (int i = 1; i <= q; ++i) {
61         auto &[x, y] = qry[i];
62         cin >> x;
63         if (x == 2) cin >> y;
64         else {
65             char ch;
66             cin >> ch;
67             y = ch;
68             s += ch;
69         }
70     }
71     auto nxt = prefix_function(s);
72     dfs(0);
73     for (int i = 1; i <= n; ++i) {
74         tr.add(dfn[i], 1);
75     }
76     for (int i = 1; i <= q; ++i) {
77         auto [x, y] = qry[i];
78         if (x == 1) tr.add(dfn[+n], 1);
79         else {
80             int cur = n;
81             for (int j = 20; j >= 0; --j) {
82                 int k = y;
83                 int p = fa[cur][j];
84                 if (tr.query(low[p]) - tr.query(dfn[p] - 1) < k) {
85                     cur = p;
86                 }
87             }
88         }
89     }
90 }

```



```

53     mx[o] += v, mn[o] += v;
54     sum[o] += (R[o] - L[o] + 1) * v;
55 }
56 void pushdown(ll o){
57     if (L[o] == R[o])
58         return;
59     if (~set[o]){
60         maketag_set(o << 1, set[o]);
61         maketag_set(o << 1 | 1, set[o]);
62         set[o] = -1;
63     }
64     if (add[o]){
65         maketag_add(o << 1, add[o]);
66         maketag_add(o << 1 | 1, add[o]);
67         add[o] = 0;
68     }
69 }
70 void pushup(ll o){
71     mx[o] = max(mx[o << 1], mx[o << 1 | 1]);
72     mn[o] = min(mn[o << 1], mn[o << 1 | 1]);
73     sum[o] = sum[o << 1] + sum[o << 1 | 1];
74 }
75 void build(ll o, ll l, ll r, ll *array = NULL){
76     ll mid((l + r) >> 1);
77     L[o] = l, R[o] = r;
78     add[o] = 0;
79     set[o] = -1;
80     if (l == r)
81     {
82         if (array)
83             mn[o] = mx[o] = sum[o] = array[l];
84         else
85             mn[o] = mx[o] = sum[o] = 0;
86         return;
87     }
88     build(o << 1, l, mid, array);
89     build(o << 1 | 1, mid + 1, r, array);
90     pushup(o);
91 }
92 void Set(ll o, ll l, ll r, ll v)
93 {
94     ll mid((L[o] + R[o]) >> 1);
95     if (l <= L[o] and r >= R[o])
96     {
97         maketag_set(o, v);
98         return;
99     }
100    pushdown(o);
101    if (l <= mid)
102        Set(o << 1, l, r, v);
103    if (r > mid)
104        Set(o << 1 | 1, l, r, v);
105    pushup(o);
106 }
107 void Add(ll o, ll l, ll r, ll v)
108 {
109     ll mid((L[o] + R[o]) >> 1);
110     if (l <= L[o] and r >= R[o])
111     {
112         maketag_add(o, v);
113         return;
114     }
115    pushdown(o);
116    if (l <= mid)
117        Add(o << 1, l, r, v);
118    if (r > mid)
119        Add(o << 1 | 1, l, r, v);
120    pushup(o);
121 }
122 ll Sum(ll o, ll l, ll r)
123 {

```

```

124     pushdown(o);
125     ll mid((L[o] + R[o]) >> 1), ans(0);
126     if (l <= L[o] and r >= R[o])
127         return sum[o];
128     if (l <= mid)
129         ans += Sum(o << 1, l, r);
130     if (r > mid)
131         ans += Sum(o << 1 | 1, l, r);
132     return ans;
133 }
134 ll Min(ll o, ll l, ll r)
135 {
136     ll mid((L[o] + R[o]) >> 1), ans(linf);
137     if (l <= L[o] and r >= R[o])
138         return mn[o];
139     pushdown(o);
140     if (l <= mid)
141         ans = min(ans, Min(o << 1, l, r));
142     if (r > mid)
143         ans = min(ans, Min(o << 1 | 1, l, r));
144     return ans;
145 }
146 ll Max(ll o, ll l, ll r)
147 {
148     ll mid((L[o] + R[o]) >> 1), ans(-linf);
149     if (l <= L[o] and r >= R[o])
150         return mx[o];
151     pushdown(o);
152     if (l <= mid)
153         ans = max(ans, Max(o << 1, l, r));
154     if (r > mid)
155         ans = max(ans, Max(o << 1 | 1, l, r));
156     return ans;
157 }
158 } segtree;
159
160 void solve(){
161     int n, q;
162     cin >> n >> q;
163     string s;
164     cin >> s; s = " " + s;
165     mnc.calc(s, n);
166     vector<int> l(q + 1), r(q + 1), id(q + 1);
167     // for (int i = 1; i <= 2 * n + 1; ++i) {
168     //     cout << char(mnc.r[i]);
169     // }
170     // cout << endl;
171     // for (int i = 1; i <= 2 * n + 1; ++i) {
172     //     cout << mnc.p[i] << " \n"[i == 2 * n + 1];
173     // }
174     // cout << endl;
175     for (int i = 1; i <= q; ++i) {
176         cin >> l[i] >> r[i];
177         l[i] = 2 * l[i] - 1;
178         r[i] = 2 * r[i] + 1;
179         id[i] = i;
180     }
181     sort(id.begin() + 1, id.end(), [&](ll a, ll b) {
182         return l[a] + r[a] < l[b] + r[b];
183     });
184     segtree.build(1, 1, 2 * n + 1);
185     vector<ll> ans(q + 1);
186     for (int i = 1, j = 1; i <= q; ++i) {
187         while (j <= ((l[id[i]] + r[id[i]])) >> 1)) {
188             segtree.Add(1, j - mnc.p[j] + 1, j, 1);
189             ++j;
190         }
191         ans[id[i]] += segtree.Sum(1, l[id[i]], 2 * n);
192     }
193     segtree.build(1, 1, 2 * n + 1);
194     for (int i = q, j = 2 * n + 1; i >= 1; --i) {

```



```

195     while (j > ((l[id[i]] + r[id[i]]) >> 1)) {
196         segtree.Add(1, j, j + mnc.p[j] - 1, 1);
197         --j;
198     }
199     ans[id[i]] += segtree.Sum(1, 1, r[id[i]]);
200 }
201 for (int i = 1; i <= q; ++i) {
202     ans[i] -= (r[i] + 1) / 2 - (l[i] / 2);
203     cout << ans[i] / 2 << endl;
204 }
205 }
206
207 signed main(){
208     ios::sync_with_stdio(false);
209     cin.tie(nullptr);
210     int t = 1;
211     //cin >> t;
212     while(t--) solve();
213     return 0;
214 }

```

最长双回文串

```

1  #include <bits/stdc++.h>
2  #define endl '\n'
3
4  using ll = long long;
5
6  constexpr int N = 2e5 + 10;
7  constexpr int mod = 998244353;
8
9  using namespace std;
10
11 int L[N], R[N];
12
13 struct Manacher {
14     int r[N], p[N], n;
15     void clear() {
16         memset(r, 0, sizeof r);
17         memset(p, 0, sizeof p);
18     }
19
20     void calc(string s, int tn){
21         n = tn;
22         int mx(0), center;
23         r[0] = -2;
24         for (int i = 1; i <= n; i++)
25             r[2 * i] = s[i];
26         for (int i = 1; i <= n; i++)
27             r[2 * i - 1] = -1;
28         r[2 * n + 1] = -1;
29         for (int i = 1; i <= 2 * n + 1; i++){
30             if (mx >= i)
31                 p[i] = min(p[2 * center - i], mx - i + 1);
32             else
33                 p[i] = 1;
34             while (r[i - p[i]] == r[i + p[i]])
35                 p[i]++;
36             if (i + p[i] - 1 > mx){
37                 mx = i + p[i] - 1;
38                 center = i;
39             }
40             int ll = i - p[i] + 1;
41             int rr = i + p[i] - 1;
42             R[rr] = max(R[rr], p[i] - 1); // 以 rr 为回文右端点的最长回文串
43             L[ll] = max(L[ll], p[i] - 1); // 以 ll 为回文左端点的最长回文串
44         }
45     }
46 }mnc;
47
48 void solve(){

```

```

49     string s;
50     cin >> s;
51     int n = s.length();
52     s = " " + s;
53     mnc.calc(s, n);
54     for (int i = 2 * n - 1; i >= 1; i -= 2) {
55         R[i] = max(R[i], R[i + 2] - 2);
56     }
57     for (int i = 3; i <= 2 * n + 1; i += 2) {
58         L[i] = max(L[i], L[i - 2] - 2);
59     }
60     ll ans = 0;
61     for (int i = 1; i <= 2 * n + 1; i += 2) if (L[i] && R[i]) ans = max(ans, 1ll * (L[i] + R[i]));
62     cout << ans << endl;
63 }
64
65 signed main(){
66     ios::sync_with_stdio(false);
67     cin.tie(nullptr);
68     int t = 1;
69     //cin >> t;
70     while(t--) solve();
71     return 0;
72 }

```

每个回文串有不同字符累和输出

```

1  #include <bits/stdc++.h>
2  #define endl '\n'
3
4  using ll = long long;
5
6  constexpr int N = 3e5 + 10;
7  constexpr int ALP = 26;
8  constexpr int mod = 998244353;
9
10 using namespace std;
11
12 struct PAM {
13     int nxt[N][ALP]; // nxt 指针, 参照 Trie 树
14     int fail[N]; // fail 失配后缀链接
15     int cnt[N]; // 此回文串出现个数
16     int num[N];
17     int len[N]; // 回文串长度
18     int s[N]; // 存放添加的字符
19     int lst; // 指向上一个字符所在的节点, 方便下一次 add
20     int n; // 已添加字符个数
21     int p; // 节点个数
22
23     int newNode(int w) {
24         for (int i = 0; i < ALP; ++i) {
25             nxt[p][i] = 0;
26         }
27         cnt[p] = 0;
28         num[p] = 0;
29         len[p] = w;
30         return p++;
31     }
32
33     void init() {
34         p = 0;
35         newNode(0);
36         newNode(-1);
37         lst = 0;
38         n = 0;
39         s[n] = -1;
40         fail[0] = 1;
41     }
42
43     int getFail(int x) {
44         while (s[n - (len[x] + 1)] != s[n]) x = fail[x];

```

```

45     return x;
46 }
47
48 void add(int c) {
49     c -= 'a';
50     s[++n] = c;
51     int cur = getFail(lst);
52     if (!nxt[cur][c]) {
53         int now = newNode(len[cur] + 2);
54         fail[now] = nxt[getFail(fail[cur])][c];
55         nxt[cur][c] = now;
56         num[now] = num[fail[now]] + 1;
57     }
58     lst = nxt[cur][c];
59     cnt[lst]++;
60 }
61
62 void count() {
63     for (int i = p - 1; i >= 0; i--)
64         cnt[fail[i]] += cnt[i];
65 }
66
67
68 ll dfs(int u, int vis) {
69     int tot = 0;
70     ll res = 0;
71     for (int i = 0; i < ALP; ++i) if (vis >> i & 1) tot++;
72     if (u >= 2) res += tot * cnt[u];
73     for (int i = 0; i < ALP; ++i) {
74         if (nxt[u][i]) res += dfs(nxt[u][i], vis | (1 << i));
75     }
76     return res;
77 }
78
79 ll getAns() {
80     ll res = 0;
81     res = dfs(0, 0);
82     res += dfs(1, 0);
83     return res;
84 }
85 }pam;
86
87 void solve(){
88     string s;
89     cin >> s;
90     int n = s.length();
91     pam.init();
92     for (int i = 0; i < n; ++i) pam.add(s[i]);
93     pam.count();
94     cout << pam.getAns() << endl;
95 }
96
97 signed main(){
98     ios::sync_with_stdio(false);
99     cin.tie(nullptr);
100     int t = 1;
101     //cin >> t;
102     while(t--) solve();
103     return 0;
104 }

```

划分为若干个偶数回文串方案数

```

1  #include <bits/stdc++.h>
2  #define endl '\n'
3
4  using ll = long long;
5
6  constexpr int N = 1e6 + 10;
7  constexpr int mod = 1e9 + 7;
8  constexpr int ALP = 26;

```

```

9
10 using namespace std;
11
12 /*
13 1. 一个回文串 S 的后缀 T 如果是回文串等价于 T 是 S 的 border
14 2. 对于若干个回文后缀, 如果长度都大于 len/2, 那么它们构成了一个等差数列
15 3. 将一个串 S 的所有 border 按长度从小到大排序后, 能形成 log 个等差数列
16 */
17
18 ll f[N], g[N];
19
20 struct PAM {
21     int nxt[N][ALP]; // nxt 指针, 参照 Trie 树
22     int fail[N]; // fail 失配后缀链接
23     int cnt[N]; // 此回文串出现个数
24     int num[N];
25     int len[N]; // 回文串长度
26     int s[N]; // 存放添加的字符
27     int lst; // 指向上一个字符所在的节点, 方便下一次 add
28     int n; // 已添加字符个数
29     int p; // 节点个数
30     int lft[N]; // 除去回文后缀后剩余的长度
31     int top[N]; // 等差数列失配链接
32     int id[N]; // 位于回文树的下标
33
34     int newNode(int w) {
35         for (int i = 0; i < ALP; ++i) {
36             nxt[p][i] = 0;
37         }
38         cnt[p] = 0;
39         num[p] = 0;
40         len[p] = w;
41         return p++;
42     }
43
44     void init() {
45         p = 0;
46         newNode(0);
47         newNode(-1);
48         lst = 0;
49         n = 0;
50         s[n] = -1;
51         fail[0] = 1;
52     }
53
54     int getFail(int x) {
55         while (s[n - (len[x] + 1)] != s[n]) x = fail[x];
56         return x;
57     }
58
59     void add(int c) {
60         c -= 'a';
61         s[++n] = c;
62         int cur = getFail(lst);
63         if (!nxt[cur][c]) {
64             int now = newNode(len[cur] + 2);
65             fail[now] = nxt[getFail(fail[cur])][c];
66             nxt[cur][c] = now;
67             num[now] = num[fail[now]] + 1;
68             lft[now] = len[now] - len[fail[now]];
69             top[now] = lft[now] == lft[fail[now]] ? top[fail[now]] : fail[now];
70         }
71         id[n] = lst = nxt[cur][c];
72         cnt[lst]++;
73     }
74
75     void count() {
76         for (int i = p - 1; i >= 0; i--)
77             cnt[fail[i]] += cnt[i];
78     }
79 }

```

```

80
81 // f[i] 记录 [1...i] 时的方案数, g[i] 记录以回文串 i 为最大长度的等差数列的当前 f 值和
82 void solve() {
83     for (int i = 1; i <= n; ++i) {
84         int idx = id[i];
85         int tot = 0;
86         for (int j = idx; j; j = top[j]) {
87             tot++;
88             g[j] = f[i - len[top[j]] - lft[j]];
89             if (lft[j] == lft[fail[j]]) g[j] = (g[j] + g[fail[j]]) % mod;
90             if (i % 2 == 0) f[i] = (f[i] + g[j]) % mod;
91         }
92     }
93 }
94 }pam;
95
96 void solve(){
97     string s;
98     cin >> s;
99     int n = s.length();
100     s = " " + s;
101     string str = " ";
102     for (int i = 1; i <= n / 2; ++i) {
103         str.push_back(s[i]);
104         str.push_back(s[n - i + 1]);
105     }
106     pam.init();
107     for (int i = 1; i <= n; ++i) pam.add(str[i]);
108     f[0] = 1;
109     pam.solve();
110     cout << f[n] << endl;
111 }
112
113 signed main(){
114     ios::sync_with_stdio(false);
115     cin.tie(nullptr);
116     int t = 1;
117     //cin >> t;
118     while(t--) solve();
119     return 0;
120 }

```

CDQ 解决高维偏序

HDU2024 暑期多校 1007

$i < j$

$f[i] < f[j]$

$g[i] < g[j]$

满足上面三个条件则 i 向 j 建边, 计算每个点的出度

```

1  #include <algorithm>
2  #include <bits/stdc++.h>
3  #include <vector>
4  #define endl '\n'
5  using namespace std;
6  typedef long double db;
7  typedef long long ll;
8
9  const ll N = 2e5 + 10;
10 const ll mod = 998244353;
11 const ll inf32 = 0x3f3f3f3f;
12 const ll inf64 = 5e18;
13
14 int tr[N];
15 int lowbit(int x) {return x & -x;}
16 void add(int p, int val){
17     for (; p < N; p += lowbit(p)){
18         tr[p] += val;
19     }

```

```

20 }
21 int query(int p){
22     int res = 0;
23     for (; p; p -= lowbit(p)){
24         res += tr[p];
25     }
26     return res;
27 }
28
29 int query(int p1, int p2){
30     return query(p2) - query(p1 - 1);
31 }
32
33 struct Query{
34     int f, g, idx;
35 }node[N];
36
37 ll f_[N], g_[N];
38 ll n;
39 ll sum[N];
40
41 Query tmp[N];
42
43 void cdq1(int l, int r){
44     int mid = (l + r) >> 1;
45     if (l == r) {
46         node[l].f = n - f_[l] + 1;
47         node[l].g = n - g_[l] + 1;
48         node[l].idx = l;
49         return;
50     }
51     cdq1(l, mid);
52     cdq1(mid + 1, r);
53     int cnt = l, f1 = l, f2 = mid + 1;
54     while (f1 <= mid && f2 <= r){
55         if (node[f1].f <= node[f2].f) {
56             sum[node[f1].idx] += query(node[f1].g - 1);
57             tmp[cnt++] = node[f1++];
58         }else{
59             add(node[f2].g, 1);
60             tmp[cnt++] = node[f2++];
61         }
62     }
63     while (f1 <= mid){
64         sum[node[f1].idx] += query(node[f1].g - 1);
65         tmp[cnt++] = node[f1++];
66     }
67     for (int i = mid + 1; i < f2; ++i) add(node[i].g, -1);
68     while (f2 <= r){
69         tmp[cnt++] = node[f2++];
70     }
71     for (int i = l; i <= r; ++i){
72         node[i] = tmp[i];
73     }
74 }
75
76 void cdq2(int l, int r){
77     int mid = (l + r) >> 1;
78     if (l == r) {
79         node[l].f = f_[l];
80         node[l].g = g_[l];
81         node[l].idx = l;
82         return;
83     }
84     cdq2(l, mid);
85     cdq2(mid + 1, r);
86     int cnt = l, f1 = l, f2 = mid + 1;
87     while (f1 <= mid && f2 <= r){
88         if (node[f1].f < node[f2].f) {
89             add(node[f1].g, 1);
90             tmp[cnt++] = node[f1++];

```

```

91         }else{
92             sum[node[f2].idx] -= query(node[f2].g - 1);
93             tmp[cnt++] = node[f2++];
94         }
95     }
96     while(f2 <= r) {
97         sum[node[f2].idx] -= query(node[f2].g - 1);
98         tmp[cnt++] = node[f2++];
99     }
100     for(int i = l; i < f1; i++) add(node[i].g, -1);
101     while(f1 <= mid){
102         tmp[cnt++] = node[f1++];
103     }
104     for (int i = l; i <= r; ++i){
105         node[i] = tmp[i];
106     }
107 }
108
109 void solve(){
110     cin >> n;
111     for (int i = 1; i <= n; ++i) cin >> f_[i];
112     for (int i = 1; i <= n; ++i) cin >> g_[i];
113     ll ans = 1ll * n * (n - 1) * (n - 2) / 6;
114     for (int i = 1; i <= n; ++i) sum[i] = i - 1;
115     cdq1(1, n);
116     cdq2(1, n);
117     // for (int i = 1; i <= n; ++i){
118     //     cout << sum[i] << " \n"[i == n];
119     // }
120     for (int i = 1; i <= n; ++i) ans = ans - sum[i] * (sum[i] - 1) / 2;
121     cout << ans << endl;
122 }
123
124 signed main(){
125     ios::sync_with_stdio(false), cin.tie(0), cout.tie(0);
126     int t = 1;
127     //cin >> t;
128     while(t--) solve();
129     return 0;
130 }

```

CF1045G

火星上有 N 个机器人排成一行，第 i 个机器人的位置为 x_i ，视野为 r_i ，智商为 q_i 。我们认为第 i 个机器人可以看到的位置是 $[x_i - r_i, x_i + r_i]$ 。

如果一对机器人相互可以看到，且它们的智商 q_i 的差距不大于 K ，那么它们会开始聊天。为了防止它们吵起来，请计算有多少对机器人可能会聊天。

```

1  #include <algorithm>
2  #include <bits/stdc++.h>
3  #define endl '\n'
4  #define int ll
5  using namespace std;
6  typedef long double db;
7  typedef long long ll;
8
9  const ll N = 1e5 + 10;
10 const ll mod = 998244353;
11 const ll inf32 = 0x3f3f3f3f;
12 const ll inf64 = 5e18;
13
14 struct Query{
15     int pos, R, l, r;
16     int q;
17 }qry[N];
18
19 int tr[N << 2], mx, n, k, ans = 0;
20
21 int lowbit(int x) {return x & -x;}
22

```

```

23 void add(int p, int val){
24     for (int i = p; i <= mx; i += lowbit(i)) tr[i] += val;
25 }
26
27 int query(int p){
28     int res = 0;
29     for (int i = p; i; i -= lowbit(i)) res += tr[i];
30     return res;
31 }
32
33 int query(int p1, int p2){
34     return query(p2) - query(p1 - 1);
35 }
36
37 bool cmplen(const Query& x, const Query& y){
38     return x.R > y.R;
39 }
40
41 unordered_map<int, int> mp;
42 set<int> t;
43
44 Query tmp[N];
45
46 void merge(int l, int mid, int r){
47     int cnt = l;
48     int i = l, j = mid + 1;
49     while (i <= mid && j <= r) tmp[cnt++] = qry[i].q <= qry[j].q ? qry[i++] : qry[j++];
50     while (i <= mid) tmp[cnt++] = qry[i++];
51     while (j <= r) tmp[cnt++] = qry[j++];
52     for (int i = l; i <= r; ++i) qry[i] = tmp[i];
53 }
54
55 void cdq(int l, int r){
56     int mid = (l + r) >> 1;
57     if (l == r) return;
58     cdq(l, mid);
59     cdq(mid + 1, r);
60     int cl = l, cr = l - 1;
61     for (int i = mid + 1; i <= r; ++i){
62         auto [pos, R, ql, qr, q] = qry[i];
63         while (cl <= mid && q - k > qry[cl].q) add(qry[cl++].pos, -1);
64         while (cr < mid && qry[cr + 1].q <= q + k) add(qry[++cr].pos, 1);
65         ans += query(ql, qr);
66     }
67     for (int i = cl; i <= cr; ++i) add(qry[i].pos, -1);
68     merge(l, mid, r); // 对 q 排序满足第二个偏序条件
69 }
70
71 void solve(){
72     cin >> n >> k;
73     for (int i = 1; i <= n; ++i){
74         auto &[pos, R, _1, _2, q] = qry[i];
75         cin >> pos >> R >> q;
76         t.insert(pos);
77         t.insert(pos - R);
78         t.insert(pos + R);
79     }
80     for (auto v : t) mp[v] = ++mx;
81     for (int i = 1; i <= n; ++i){
82         qry[i].l = mp[qry[i].pos - qry[i].R];
83         qry[i].r = mp[qry[i].pos + qry[i].R];
84         qry[i].pos = mp[qry[i].pos];
85     }
86     // 对 R 排序满足第一个偏序条件
87     sort(qry + 1, qry + n + 1, cmplen);
88     cdq(1, n);
89     cout << ans << endl;
90 }
91
92 signed main(){
93     ios::sync_with_stdio(false), cin.tie(0), cout.tie(0);

```



```

94     int t = 1;
95     //cin >> t;
96     while(t--) solve();
97     return 0;
98 }

```

陌上开花 (‘纯板子’)

有 n 个元素，第 i 个元素有 a_i, b_i, c_i 三个属性，设 $f(i)$ 表示满足 $a_j \leq a_i$ 且 $b_j \leq b_i$ 且 $c_j \leq c_i$ 且 $j \neq i$ 的 j 的数量。

```

1  #include <bits/stdc++.h>
2  #include <vector>
3  #define endl '\n'
4  using namespace std;
5  typedef long double db;
6  typedef long long ll;
7
8  const ll N = 2e5 + 10;
9  const ll mod = 998244353;
10 const ll inf32 = 0x3f3f3f3f;
11 const ll inf64 = 5e18;
12
13 struct Query{
14     int a, b, c;
15     int w, ans;
16 }node[N], qry[N];
17
18 struct BIT{
19     vector<int> tr;
20     void init(int n){
21         vector<int>(n + 1).swap(tr);
22     }
23     int lowbit(int x) {return x & -x;}
24     void add(int x, int val){
25         for (; x < tr.size(); x += lowbit(x))
26             tr[x] += val;
27     }
28     int query(int x){
29         int res = 0;
30         for (; x; x -= lowbit(x)){
31             res += tr[x];
32         }
33         return res;
34     }
35 }T;
36
37 bool cmpa(const Query& u, const Query& v){
38     if (u.a == v.a){
39         if (u.b == v.b)
40             return u.c < v.c;
41         return u.b < v.b;
42     }
43     return u.a < v.a;
44 }
45
46 bool cmpb(const Query& u, const Query& v){
47     if (u.b == v.b){
48         return u.c < v.c;
49     }
50     return u.b < v.b;
51 }
52
53 Query tmp[N];
54 ll cnt[N];
55
56 void cdq(int l, int r){
57     if (l == r) return;
58     int mid = (l + r) >> 1;
59     cdq(l, mid);
60     cdq(mid + 1, r);

```

```

61     sort(qry + l, qry + mid + 1, cmpb);
62     sort(qry + mid + 1, qry + r + 1, cmpb);
63     int f2 = mid + 1, f1 = l;
64     for (; f2 <= r; ++f2){
65         while (qry[f1].b <= qry[f2].b && f1 <= mid){
66             T.add(qry[f1].c, qry[f1].w);
67             f1++;
68         }
69         qry[f2].ans += T.query(qry[f2].c);
70     }
71     for (int i = l; i < f1; ++i) T.add(qry[i].c, -qry[i].w);
72 }
73
74 void solve(){
75     int n_, k, n = 0;
76     cin >> n_ >> k;
77     T.init(k);
78     for (int i = 1; i <= n_; ++i){
79         auto &[a, b, c, w, ans] = node[i];
80         cin >> a >> b >> c;
81     }
82     sort(node + 1, node + n_ + 1, cmpa);
83     int c = 0;
84     for (int i = 1; i <= n_; ++i){
85         c++;
86         if (node[i].a != node[i + 1].a || node[i].b != node[i + 1].b || node[i].c != node[i + 1].c){
87             qry[++n] = node[i];
88             qry[n].w = c;
89             c = 0;
90         }
91     }
92     cdq(1, n);
93     for (int i = 1; i <= n; ++i){
94         cnt[qry[i].ans + qry[i].w - 1] += qry[i].w;
95     }
96     for (int i = 0; i < n_; ++i) cout << cnt[i] << endl;
97 }
98
99 signed main(){
100     ios::sync_with_stdio(false), cin.tie(0), cout.tie(0);
101     int t = 1;
102     //cin >> t;
103     while(t--) solve();
104     return 0;
105 }

```

dsu on tree

点分树模版 (距离为 k 是否存在)

```

1  #include <bits/stdc++.h>
2  #define endl '\n'
3
4  using ll = long long;
5
6  constexpr int N = 2e5 + 10;
7  constexpr int mod = 998244353;
8
9  using namespace std;
10
11 void solve() {
12     int n, m;
13     cin >> n >> m;
14     vector<vector<pair<int, int>>> G(n + 1);
15     for(int i = 1; i < n; i++) {
16         ll u, v, w;
17         cin >> u >> v >> w;
18         G[u].emplace_back(v, w);
19         G[v].emplace_back(u, w);
20     }
21 }

```

```

22     vector<int> q(m + 1), ans(m + 1);
23
24     for(int i = 1; i <= m; i++) {
25         cin >> q[i];
26     }
27
28     set<int> s;
29     for(int i = 1; i <= m; i++) {
30         s.insert(i);
31     }
32
33     vector<int> l(n + 1), r(n + 1), siz(n + 1), son(n + 1), id(n + 1);
34     vector<ll> dis(n + 1);
35     int idx = 0;
36
37     auto dfs1 = [&](auto self, int now, int p) -> void {
38         l[now] = ++idx, id[idx] = now;
39         for(auto [u, w] : G[now]) {
40             if(u == p) continue;
41             dis[u] = dis[now] + w;
42             self(self, u, now);
43             siz[now] += siz[u];
44             if(siz[u] > siz[son[now]]) {
45                 son[now] = u;
46             }
47         }
48         siz[now]++;
49         r[now] = idx;
50     };
51
52     dfs1(dfs1, 1, 0);
53
54     unordered_map<ll, int> cnt;
55
56     auto upd = [&](int x, int k) {
57         cnt[dis[x]] += k;
58     };
59
60     auto dfs2 = [&](auto self, int now, int p, int keep) -> void {
61         for(auto [u, w] : G[now]) {
62             if(u == p || u == son[now]) continue;
63             self(self, u, now, 0);
64         }
65
66         if(son[now]) self(self, son[now], now, 1);
67
68         vector<int> ww;
69         for(auto i : s) {
70             if(cnt[dis[now] + q[i]]) {
71                 ans[i] = 1;
72                 ww.push_back(i);
73             }
74         }
75
76         while(!ww.empty()) {
77             s.erase(ww.back());
78             ww.pop_back();
79         }
80
81         upd(now, 1);
82
83         for(auto [u, w] : G[now]) {
84             if(u == p || u == son[now]) continue;
85             for(int i = l[u]; i <= r[u]; i++) {
86                 for(auto j : s) {
87                     ll res = dis[now] + dis[u] + q[j] - dis[id[i]];
88                     if(cnt[res]) {
89                         ans[j] = 1;
90                         ww.push_back(j);
91                     }
92                 }

```

```

93         }
94
95         while(!ww.empty()) {
96             s.erase(ww.back());
97             ww.pop_back();
98         }
99
100        for(int i = l[u]; i <= r[u]; i++) {
101            upd(id[i], 1);
102        }
103    }
104
105    if(!keep) {
106        for(int i = l[now]; i <= r[now]; i++) {
107            upd(id[i], -1);
108        }
109    }
110 };
111
112 dfs2(dfs2, 1, 0, 1);
113
114 for(int i = 1; i <= m; i++) {
115     cout << (ans[i] ? "AYE" : "NAY") << endl;
116 }
117
118 }
119
120 signed main(){
121     ios::sync_with_stdio(false);
122     cin.tie(nullptr);
123     int t = 1;
124     //cin >> t;
125     while(t--) solve();
126     return 0;
127 }

```

CF600E(纯板子)

```

1  #include <bits/stdc++.h>
2  #define endl '\n'
3
4  using ll = long long;
5
6  constexpr int N = 2e5 + 10;
7  constexpr int mod = 998244353;
8
9  using namespace std;
10
11 int cnt[N];
12
13 void solve(){
14     int n;
15     cin >> n;
16     vector<int> col(n + 1);
17     for (int i = 1; i <= n; ++i){
18         cin >> col[i];
19     }
20     vector<vector<int>> G(n + 1);
21     for (int i = 1; i < n; ++i){
22         int u, v;
23         cin >> u >> v;
24         G[u].push_back(v);
25         G[v].push_back(u);
26     }
27
28     vector<int> l(n + 1), r(n + 1), rnk(n + 1), sz(n + 1), son(n + 1);
29     int tot = 0;
30
31     function<void(int, int)> dfs1 = [&](int u, int fa){
32         sz[u] = 1;
33         l[u] = ++tot;

```

```

34     rnk[tot] = u;
35     for (auto v : G[u]){
36         if (v == fa) continue;
37         dfs1(v, u);
38         sz[u] += sz[v];
39         if (sz[v] > sz[son[u]]) son[u] = v;
40     }
41     r[u] = tot;
42 };
43
44 dfs1(1, 0);
45
46 vector<ll> ans(n + 1);
47 ll res = 0, mx = -1;
48
49 auto modify = [&] (int x){
50     cnt[col[x]]++;
51     if (cnt[col[x]] == mx) res += col[x];
52     else if (cnt[col[x]] > mx) {
53         mx = cnt[col[x]];
54         res = col[x];
55     }
56 };
57
58 function<void(int, int, int)> dfs2 = [&](int u, int fa, int keep){
59     for (auto v : G[u]){
60         if (v == fa || v == son[u]) continue;
61         dfs2(v, u, 0);
62     }
63     if (son[u]) dfs2(son[u], u, 1);
64
65     modify(u);
66
67     for (auto v : G[u]){
68         if (v == fa || v == son[u]) continue;
69         for (int i = l[v]; i <= r[v]; ++i){
70             modify(rnk[i]);
71         }
72     }
73
74     ans[u] = res;
75
76     if (!keep){
77         for (int i = l[u]; i <= r[u]; ++i){
78             cnt[col[rnk[i]]]--;
79         }
80         mx = -1;
81         res = 0;
82     }
83 };
84
85 dfs2(1, 0, 1);
86
87 for (int i = 1; i <= n; ++i) cout << ans[i] << " \n"[i == n];
88 }
89
90 signed main(){
91     ios::sync_with_stdio(false);
92     cin.tie(nullptr);
93     int t = 1;
94     //cin >> t;
95     while(t--) solve();
96     return 0;
97 }

```

LCT

LCA(询问以 **root** 为根 **x, y** 的 LCA, 询问格式 **root, x, y**)

```

1  #include <algorithm>
2  #include <bits/stdc++.h>

```

```

3  #include <cassert>
4  #include <iostream>
5  #include <queue>
6  #include <vector>
7  #define endl '\n'
8  #define pll pair<i64, i64>
9  #define tll tuple<i64, i64, i64>
10 #define all(a) a.begin() + 1, a.end()
11 using namespace std;
12 using db = long double;
13 using i64 = long long;
14 const i64 N = 1e5 + 10;
15 const i64 mod = 998244353;
16 const i64 inf32 = 1e9;
17 const i64 inf64 = 5e18;
18
19 int n, m;
20 int st[N];
21 struct node{
22     int son[2];
23     int fa, tag;
24     int sz;
25 }tr[N];
26
27 inline int is_root(int rt){
28     return tr[tr[rt].fa].son[0] != rt && tr[tr[rt].fa].son[1] != rt;
29 }
30
31 inline void push_down(int rt){
32     if (!rt || !tr[rt].tag) return;
33     tr[tr[rt].son[0]].tag ^= 1;
34     tr[tr[rt].son[1]].tag ^= 1;
35     tr[rt].tag ^= 1;
36     swap(tr[rt].son[0], tr[rt].son[1]);
37 }
38
39
40 inline void rotate(int rt){
41     int f = tr[rt].fa, gf = tr[f].fa, k = tr[f].son[0] == rt ? 0 : 1;
42     if (!is_root(f)){
43         if (tr[gf].son[0] == f) tr[gf].son[0] = rt;
44         else tr[gf].son[1] = rt;
45     }
46     tr[rt].fa = gf; tr[f].fa = rt;
47     tr[tr[rt].son[!k]].fa = f;
48     tr[f].son[k] = tr[rt].son[!k];
49     tr[rt].son[!k] = f;
50 }
51
52 void splay(int rt){
53     int top = 0;
54     st[++top] = rt;
55     for (int i = rt; !is_root(i); i = tr[i].fa){
56         st[++top] = tr[i].fa;
57     }
58     while (top){
59         push_down(st[top--]);
60     }
61     while (!is_root(rt)){
62         int f = tr[rt].fa, gf = tr[f].fa;
63         if (!is_root(f)){
64             if ((tr[f].son[0] == rt) ^ (tr[gf].son[0] == f)) rotate(rt);
65             else rotate(f);
66         }
67         rotate(rt);
68     }
69 }
70
71
72 int access(int rt){
73     int tmp = 0;

```

```

74     while (rt){
75         splay(rt);
76         tr[rt].son[1] = tmp;
77         tmp = rt;
78         rt = tr[rt].fa;
79     }
80     return tmp;
81 }
82
83 void makeroot(int rt){
84     access(rt);
85     splay(rt);
86     tr[rt].tag ^= 1;
87 }
88
89 int findroot(int rt){
90     access(rt);
91     splay(rt);
92     int tmp = rt;
93     while (tr[tmp].son[0]) tmp = tr[tmp].son[0];
94     splay(tmp);
95     return tmp;
96 }
97
98 void cut(int x, int y){
99     makeroot(x);
100    access(y);
101    splay(y);
102    tr[y].son[0] = tr[x].fa = 0;
103 }
104
105 void link(int x, int y){
106     makeroot(x);
107     tr[x].fa = y;
108 }
109
110 int lca(int rt, int x, int y){
111     makeroot(rt);
112     access(x);
113     return access(y);
114 }
115
116 void solve(){
117     cin >> n;
118     for (int i = 1, u, v; i < n; ++i){
119         cin >> u >> v;
120         link(u, v);
121     }
122     cin >> m;
123     while (m--){
124         int rt, x, y;
125         cin >> rt >> x >> y;
126         cout << lca(rt, x, y) << endl;
127     }
128 }
129
130 int main(){
131     ios::sync_with_stdio(false), cin.tie(0), cout.tie(0);
132     int t = 1;
133     //cin >> t;
134     while(t-->0) solve();
135     return 0;
136 }

```

HNOI 弹飞绵羊 (树上两点间距离, 连/断)

```

1  #include <algorithm>
2  #include <bits/stdc++.h>
3  #include <cassert>
4  #include <iostream>
5  #include <queue>

```

```

6  #include <vector>
7  #define endl '\n'
8  #define pll pair<i64, i64>
9  #define tll tuple<i64, i64, i64>
10 #define all(a) a.begin() + 1, a.end()
11 using namespace std;
12 using db = long double;
13 using i64 = long long;
14 const i64 N = 2e5 + 10;
15 const i64 mod = 998244353;
16 const i64 inf32 = 1e9;
17 const i64 inf64 = 5e18;
18
19 int n, m;
20 int st[N];
21 struct node{
22     int son[2];
23     int fa, tag;
24     i64 sz;
25 }tr[N];
26
27 inline int is_root(int rt){
28     return tr[tr[rt].fa].son[0] != rt && tr[tr[rt].fa].son[1] != rt;
29 }
30
31 inline void push_down(int rt){
32     if (!rt || !tr[rt].tag) return;
33     tr[tr[rt].son[0]].tag ^= 1;
34     tr[tr[rt].son[1]].tag ^= 1;
35     tr[rt].tag ^= 1;
36     swap(tr[rt].son[0], tr[rt].son[1]);
37 }
38
39 inline void push_up(int rt){
40     tr[rt].sz = tr[tr[rt].son[0]].sz + tr[tr[rt].son[1]].sz + 1;
41 }
42
43 inline void rotate(int rt){
44     int f = tr[rt].fa, gf = tr[f].fa, k = tr[f].son[0] == rt ? 0 : 1;
45     if (!is_root(f)){
46         if (tr[gf].son[0] == f) tr[gf].son[0] = rt;
47         else tr[gf].son[1] = rt;
48     }
49     tr[rt].fa = gf; tr[f].fa = rt;
50     tr[tr[rt].son[!k]].fa = f;
51     tr[f].son[k] = tr[rt].son[!k];
52     tr[rt].son[!k] = f;
53     push_up(f);
54 }
55
56 void splay(int rt){
57     int top = 0;
58     st[++top] = rt;
59     for (int i = rt; !is_root(i); i = tr[i].fa){
60         st[++top] = tr[i].fa;
61     }
62     while (top){
63         push_down(st[top--]);
64     }
65     while (!is_root(rt)){
66         int f = tr[rt].fa, gf = tr[f].fa;
67         if (!is_root(f)){
68             if ((tr[f].son[0] == rt) ^ (tr[gf].son[0] == f)) rotate(rt);
69             else rotate(f);
70         }
71         rotate(rt);
72         push_up(rt);
73     }
74 }
75
76

```



```

77  int access(int rt){
78      int tmp = 0;
79      while (rt){
80          splay(rt);
81          tr[rt].son[1] = tmp;
82          push_up(rt);
83          tmp = rt;
84          rt = tr[rt].fa;
85      }
86      return tmp;
87  }
88
89  void makeroot(int rt){
90      access(rt);
91      splay(rt);
92      tr[rt].tag ^= 1;
93  }
94
95  int findroot(int rt){
96      access(rt);
97      splay(rt);
98      int tmp = rt;
99      while (tr[tmp].son[0]) tmp = tr[tmp].son[0];
100     splay(tmp);
101     return tmp;
102 }
103
104 void cut(int x, int y){
105     makeroot(x);
106     access(y);
107     splay(y);
108     tr[y].son[0] = tr[x].fa = 0;
109 }
110
111 void link(int x, int y){
112     makeroot(x);
113     tr[x].fa = y;
114 }
115
116 int lca(int rt, int x, int y){
117     makeroot(rt);
118     access(x);
119     return access(y);
120 }
121
122 void solve(){
123     cin >> n;
124     vector<int> p(n + 10);
125     for (int i = 1; i <= n; ++i){
126         tr[i].sz = 1;
127     }
128     for (int i = 1; i <= n; ++i){
129         cin >> p[i];
130         link(i, min(p[i] + i, n + 1));
131         p[i] = min(p[i] + i, n + 1);
132     }
133     cin >> m;
134     while(m--){
135         int op;
136         cin >> op;
137         if (op == 1){
138             int x;
139             cin >> x;
140             x++;
141             makeroot(n + 1);
142             access(x), splay(x);
143             cout << tr[tr[x].son[0]].sz << endl;
144         }else{
145             int x, y;
146             cin >> x >> y;
147             x++;

```

```

148         int t = min(n + 1, x + y);
149         cut(x, p[x]);
150         link(x, t);
151         p[x] = t;
152     }
153 }
154 }
155
156 int main(){
157     ios::sync_with_stdio(false), cin.tie(0), cout.tie(0);
158     int t = 1;
159     //cin >> t;
160     while(t--) solve();
161     return 0;
162 }

```

动态并查集

```

1  #include <algorithm>
2  #include <bits/stdc++.h>
3  #include <cassert>
4  #include <queue>
5  #include <vector>
6  #define endl '\n'
7  #define pll pair<i64, i64>
8  #define tll tuple<i64, i64, i64>
9  #define all(a) a.begin() + 1, a.end()
10 using namespace std;
11 using db = long double;
12 using i64 = long long;
13 const i64 N = 1e4 + 10;
14 const i64 mod = 998244353;
15 const i64 inf32 = 1e9;
16 const i64 inf64 = 5e18;
17
18 int n, m;
19 int st[N];
20 struct node{
21     int son[2];
22     int fa, tag;
23 }tr[N];
24
25 inline int is_root(int rt){
26     return tr[tr[rt].fa].son[0] != rt && tr[tr[rt].fa].son[1] != rt;
27 }
28
29 inline void push_down(int rt){
30     if (!rt || !tr[rt].tag) return;
31     tr[tr[rt].son[0]].tag ^= 1;
32     tr[tr[rt].son[1]].tag ^= 1;
33     tr[rt].tag ^= 1;
34     swap(tr[rt].son[0], tr[rt].son[1]);
35 }
36
37
38 inline void rotate(int rt){
39     int f = tr[rt].fa, gf = tr[f].fa, k = tr[f].son[0] == rt ? 0 : 1;
40     if (!is_root(f)){
41         if (tr[gf].son[0] == f) tr[gf].son[0] = rt;
42         else tr[gf].son[1] = rt;
43     }
44     tr[rt].fa = gf; tr[f].fa = rt;
45     tr[tr[rt].son[!k]].fa = f;
46     tr[f].son[k] = tr[rt].son[!k];
47     tr[rt].son[!k] = f;
48 }
49
50 void splay(int rt){
51     int top = 0;
52     st[++top] = rt;
53     for (int i = rt; !is_root(i); i = tr[i].fa){

```

```

54     st[++top] = tr[i].fa;
55 }
56 while (top){
57     push_down(st[top--]);
58 }
59 while (!is_root(rt)){
60     int f = tr[rt].fa, gf = tr[f].fa;
61     if (!is_root(f)){
62         if ((tr[f].son[0] == rt) ^ (tr[gf].son[0] == f)) rotate(rt);
63         else rotate(f);
64     }
65     rotate(rt);
66 }
67 }
68
69
70 int access(int rt){
71     int tmp = 0;
72     while (rt){
73         splay(rt);
74         tr[rt].son[1] = tmp;
75         tmp = rt;
76         rt = tr[rt].fa;
77     }
78     return tmp;
79 }
80
81 void makeroot(int rt){
82     access(rt);
83     splay(rt);
84     tr[rt].tag ^= 1;
85 }
86
87 int findroot(int rt){
88     access(rt);
89     splay(rt);
90     int tmp = rt;
91     while (tr[tmp].son[0]) tmp = tr[tmp].son[0];
92     splay(tmp);
93     return tmp;
94 }
95
96 void cut(int x, int y){
97     makeroot(x);
98     access(y);
99     splay(y);
100    tr[y].son[0] = tr[x].fa = 0;
101 }
102
103 void link(int x, int y){
104     makeroot(x);
105     tr[x].fa = y;
106 }
107
108 void solve(){
109     cin >> n >> m;
110     while (m--){
111         int x, y;
112         string s;
113         cin >> s;
114         cin >> x >> y;
115         if (s[0] == 'C') link(x, y);
116         if (s[0] == 'D') cut(x, y);
117         if (s[0] == 'Q'){
118             if (findroot(x) == findroot(y)){
119                 cout << "Yes" << endl;
120             }else{
121                 cout << "No" << endl;
122             }
123         }
124     }

```

```

125 }
126
127 int main(){
128     ios::sync_with_stdio(false), cin.tie(0), cout.tie(0);
129     int t = 1;
130     //cin >> t;
131     while(t--) solve();
132     return 0;
133 }

```

LCT(最小生成树)

```

1  #include <bits/stdc++.h>
2  #define endl '\n'
3  using namespace std;
4  typedef long double db;
5  typedef long long ll;
6
7  const ll N = 4e5 + 10;
8  const ll mod = 998244353;
9  const ll inf32 = 0x3f3f3f3f;
10 const ll inf64 = 5e18;
11
12 int n, m, idx, ans, cnt;
13 namespace LCT
14 {
15     struct node
16     {
17         int ch[2], w, id, fa, sz, mx, tag;
18     } t[N];
19     #define isroot(x) ((t[t[x].fa].ch[0] != x) && (t[t[x].fa].ch[1] != x))
20     void pushr(int x)
21     {
22         swap(t[x].ch[0], t[x].ch[1]);
23         t[x].tag ^= 1;
24     }
25     void push_up(int x)
26     {
27         t[x].id = x;
28         t[x].mx = t[x].w;
29         if (t[t[x].ch[0]].mx > t[x].mx)
30             t[x].mx = t[t[x].ch[0]].mx, t[x].id = t[t[x].ch[0]].id;
31         if (t[t[x].ch[1]].mx > t[x].mx)
32             t[x].mx = t[t[x].ch[1]].mx, t[x].id = t[t[x].ch[1]].id;
33     }
34     void push_down(int x)
35     {
36         if (t[x].tag)
37         {
38             if (t[x].ch[0])
39                 pushr(t[x].ch[0]);
40             if (t[x].ch[1])
41                 pushr(t[x].ch[1]);
42             t[x].tag = 0;
43         }
44     }
45     void push_all(int x)
46     {
47         if (!isroot(x))
48             push_all(t[x].fa);
49         push_down(x);
50     }
51     void rotate(int x)
52     {
53         int y = t[x].fa;
54         int z = t[y].fa;
55         int k = t[y].ch[1] == x;
56         if (!isroot(y))
57             t[z].ch[t[z].ch[1] == y] = x;
58         t[x].fa = z;
59         t[y].ch[k] = t[x].ch[k ^ 1];

```

```

60     t[t[x].ch[k ^ 1]].fa = y;
61     t[x].ch[k ^ 1] = y;
62     t[y].fa = x;
63     push_up(y);
64     push_up(x);
65 }
66 void splay(int x)
67 {
68     push_all(x);
69     while (!isroot(x))
70     {
71         int y = t[x].fa;
72         int z = t[y].fa;
73         if (!isroot(y))
74             (t[z].ch[1] == y ^ (t[y].ch[1] == x) ? rotate(x) : rotate(y);
75         rotate(x);
76     }
77 }
78 void access(int x)
79 {
80     for (int y = 0; x; y = x, x = t[x].fa)
81         splay(x), t[x].ch[1] = y, push_up(x);
82 }
83 void make_root(int x)
84 {
85     access(x);
86     splay(x);
87     pushr(x);
88 }
89 int find_root(int x)
90 {
91     access(x);
92     splay(x);
93     while (t[x].ch[0])
94         push_down(x), x = t[x].ch[0];
95     splay(x);
96     return x;
97 }
98 void split(int x, int y)
99 {
100     make_root(x);
101     access(y);
102     splay(y);
103 }
104 void link(int x, int y)
105 {
106     make_root(x);
107     if (find_root(y) != x)
108         t[x].fa = y;
109 }
110 int ck(int x, int y)
111 {
112     make_root(x);
113     return find_root(y) != x;
114 }
115 void cut(int x, int y)
116 {
117     make_root(x);
118     if (find_root(y) == x && t[y].fa == x && !t[y].ch[0])
119     {
120         t[x].ch[1] = t[y].fa = 0;
121         push_up(x);
122     }
123 }
124 }
125
126 void solve(){
127     using namespace LCT;
128     cin >> n >> m;
129     idx = n;
130     for (int i = 1, x, y, z; i <= m; i++)

```

```

131 {
132     cin >> x >> y >> z;
133     t[++idx].w = z;
134     if (x != y && ck(x, y))
135         link(x, idx), link(idx, y), ans += z, ++cnt;
136     else
137     {
138         split(x, y);
139         int now = t[y].id;
140         if (t[now].mx <= z)
141             continue;
142         ans -= t[now].mx - z;
143         splay(now);
144         t[t[now].ch[0]].fa = t[t[now].ch[1]].fa = 0;
145         link(x, idx);
146         link(idx, y);
147     }
148 }
149 if (cnt < n - 1) cout << "orz" << endl;
150 else cout << ans << endl;
151 }
152
153 signed main(){
154     ios::sync_with_stdio(false), cin.tie(0), cout.tie(0);
155     int t = 1;
156     //cin >> t;
157     while(t--) solve();
158     return 0;
159 }

```

最小生成树的运用 (树上构建最小生成树, 每个边有权值 a, b 满足 1 到 n 的 $\max(a) + \max(b)$ 最小)

```

1  #include <bits/stdc++.h>
2  #define endl '\n'
3  using namespace std;
4  typedef long double db;
5  typedef long long ll;
6
7  const ll N = 2e5 + 10;
8  const ll mod = 998244353;
9  const ll inf32 = 0x3f3f3f3f;
10 const ll inf64 = 5e18;
11
12 int n, m, ans = inf32;
13 namespace LCT
14 {
15     struct Edge{
16         int u, v, a, b;
17     }e[N];
18     int cmp(Edge a, Edge b){return a.a == b.a ? a.b < b.b : a.a < b.a;}
19     struct node
20     {
21         int ch[2], w, id, fa, sz, mx, tag;
22     } t[N];
23     #define isroot(x) ((t[t[x].fa].ch[0] != x) && (t[t[x].fa].ch[1] != x))
24     void pushr(int x)
25     {
26         swap(t[x].ch[0], t[x].ch[1]);
27         t[x].tag ^= 1;
28     }
29     void push_up(int x)
30     {
31         t[x].id = x;
32         t[x].mx = t[x].w;
33         if (t[t[x].ch[0]].mx > t[x].mx)
34             t[x].mx = t[t[x].ch[0]].mx, t[x].id = t[t[x].ch[0]].id;
35         if (t[t[x].ch[1]].mx > t[x].mx)
36             t[x].mx = t[t[x].ch[1]].mx, t[x].id = t[t[x].ch[1]].id;
37     }
38     void push_down(int x)
39     {

```

```

40     if (t[x].tag)
41     {
42         if (t[x].ch[0])
43             pushr(t[x].ch[0]);
44         if (t[x].ch[1])
45             pushr(t[x].ch[1]);
46         t[x].tag = 0;
47     }
48 }
49 void push_all(int x)
50 {
51     if (!isroot(x))
52         push_all(t[x].fa);
53     push_down(x);
54 }
55 void rotate(int x)
56 {
57     int y = t[x].fa;
58     int z = t[y].fa;
59     int k = t[y].ch[1] == x;
60     if (!isroot(y))
61         t[z].ch[t[z].ch[1] == y] = x;
62     t[x].fa = z;
63     t[y].ch[k] = t[x].ch[k ^ 1];
64     t[t[x].ch[k ^ 1]].fa = y;
65     t[x].ch[k ^ 1] = y;
66     t[y].fa = x;
67     push_up(y);
68     push_up(x);
69 }
70 void splay(int x)
71 {
72     push_all(x);
73     while (!isroot(x))
74     {
75         int y = t[x].fa;
76         int z = t[y].fa;
77         if (!isroot(y))
78             (t[z].ch[1] == y) ^ (t[y].ch[1] == x) ? rotate(x) : rotate(y);
79         rotate(x);
80     }
81 }
82 void access(int x)
83 {
84     for (int y = 0; x; y = x, x = t[x].fa)
85         splay(x), t[x].ch[1] = y, push_up(x);
86 }
87 void make_root(int x)
88 {
89     access(x);
90     splay(x);
91     pushr(x);
92 }
93 int find_root(int x)
94 {
95     access(x);
96     splay(x);
97     while (t[x].ch[0])
98         push_down(x), x = t[x].ch[0];
99     splay(x);
100    return x;
101 }
102 void split(int x, int y)
103 {
104     make_root(x);
105     access(y);
106     splay(y);
107 }
108 void link(int x, int y)
109 {
110     make_root(x);

```

```

111         if (find_root(y) != x)
112             t[x].fa = y;
113     }
114     int ck(int x, int y)
115     {
116         make_root(x);
117         return find_root(y) != x;
118     }
119     void cut(int x, int y)
120     {
121         make_root(x);
122         if (find_root(y) == x && t[y].fa == x && !t[y].ch[0])
123         {
124             t[x].ch[1] = t[y].fa = 0;
125             push_up(x);
126         }
127     }
128 }
129
130 void solve(){
131     using namespace LCT;
132     cin >> n >> m;
133     for (int i = 1; i <= m; ++i){
134         auto &[u, v, a, b] = e[i];
135         cin >> u >> v >> a >> b;
136     }
137     sort(e + 1, e + 1 + m, cmp);
138     for (int i = 1; i <= m; ++i){
139         int idx = n + i;
140         auto [x, y, a, b] = e[i];
141         t[idx].w = b;
142         if (x == y) continue;
143         if (ck(x, y)){
144             link(x, idx);
145             link(idx, y);
146         }else{
147             split(x, y);
148             int now = t[y].id;
149             if (t[now].mx <= b) continue;
150             splay(now);
151             t[t[now].ch[0]].fa = t[t[now].ch[1]].fa = 0;
152             link(x, idx);
153             link(idx, y);
154         }
155         if (!ck(1, n)){
156             split(1, n);
157             ans = min(ans, t[n].mx + a);
158         }
159     }
160     if (ans == inf32) cout << -1 << endl;
161     else cout << ans << endl;
162 }
163
164 signed main(){
165     ios::sync_with_stdio(false), cin.tie(0), cout.tie(0);
166     int t = 1;
167     //cin >> t;
168     while(t--) solve();
169     return 0;
170 }

```

LCT 树上路径信息维护

一棵 n 个点的树，每个点的初始权值为 1。对于这棵树有 q 个操作，每个操作为以下四种操作之一：

+ $u\ v\ c$ ：将 u 到 v 的路径上的点的权值都加上自然数 c ；

- $u_1\ v_1\ u_2\ v_2$ ：将树中原有的边 (u_1, v_1) 删除，加入一条新边 (u_2, v_2) ，保证操作完之后仍然是一棵树；

* $u\ v\ c$ ：将 u 到 v 的路径上的点的权值都乘上自然数 c ；

/u v: 询问 u 到 v 的路径上的点的权值和, 求出答案对于 51061 的余数。

数据范围: $1 \leq n, q \leq 10^5, 0 \leq c \leq 10^4$ 。

```
1  #include <bits/stdc++.h>
2  #define endl '\n'
3  using namespace std;
4  typedef long double db;
5  typedef long long ll;
6
7  const ll N = 2e5 + 10;
8  const ll mod = 51061;
9  const ll inf32 = 0x3f3f3f3f;
10 const ll inf64 = 5e18;
11
12 int n, q;
13 namespace LCT
14 {
15     struct node
16     {
17         int ch[2], id, fa, sz, tag;
18         ll taga, tagc; // 加法, 乘法 tag
19         ll sum, val;
20     } t[N];
21     #define isroot(x) ((t[t[x].fa].ch[0] != x) && (t[t[x].fa].ch[1] != x))
22     void push_up(int x)
23     {
24         t[x].sum = (t[x].val + t[t[x].ch[0]].sum % mod + t[t[x].ch[1]].sum) % mod;
25         t[x].sz = 1 + t[t[x].ch[0]].sz + t[t[x].ch[1]].sz;
26     }
27     void pushr(int x)
28     {
29         swap(t[x].ch[0], t[x].ch[1]);
30         t[x].tag ^= 1;
31     }
32     void pusha(int x, int c){
33         t[x].taga = t[x].taga + c % mod;
34         t[x].val = t[x].val + c % mod;
35         t[x].sum = t[x].sum + (c * t[x].sz % mod) % mod;
36     }
37     void pushc(int x, int c){
38         t[x].taga = t[x].taga * c % mod;
39         t[x].val = t[x].val * c % mod;
40         t[x].sum = t[x].sum * c % mod;
41         t[x].tagc = t[x].tagc * c % mod;
42     }
43     void push_down(int x)
44     {
45         if (t[x].tag)
46         {
47             if (t[x].ch[0])
48                 pushr(t[x].ch[0]);
49             if (t[x].ch[1])
50                 pushr(t[x].ch[1]);
51             t[x].tag = 0;
52         }
53         if (t[x].tagc != 1){
54             pushc(t[x].ch[0], t[x].tagc);
55             pushc(t[x].ch[1], t[x].tagc);
56             t[x].tagc = 1;
57         }
58         if (t[x].taga){
59             pusha(t[x].ch[0], t[x].taga);
60             pusha(t[x].ch[1], t[x].taga);
61             t[x].taga = 0;
62         }
63     }
64     void push_all(int x)
65     {
66         if (!isroot(x))
67             push_all(t[x].fa);
68         push_down(x);
```

```

69     }
70     void rotate(int x)
71     {
72         int y = t[x].fa;
73         int z = t[y].fa;
74         int k = t[y].ch[1] == x;
75         if (!isroot(y))
76             t[z].ch[t[z].ch[1] == y] = x;
77         t[x].fa = z;
78         t[y].ch[k] = t[x].ch[k ^ 1];
79         t[t[x].ch[k ^ 1]].fa = y;
80         t[x].ch[k ^ 1] = y;
81         t[y].fa = x;
82         push_up(y);
83         push_up(x);
84     }
85     void splay(int x)
86     {
87         push_all(x);
88         while (!isroot(x))
89         {
90             int y = t[x].fa;
91             int z = t[y].fa;
92             if (!isroot(y))
93                 (t[z].ch[1] == y) ^ (t[y].ch[1] == x) ? rotate(x) : rotate(y);
94             rotate(x);
95         }
96     }
97     void access(int x)
98     {
99         for (int y = 0; x; y = x, x = t[x].fa)
100             splay(x), t[x].ch[1] = y, push_up(x);
101     }
102     void make_root(int x)
103     {
104         access(x);
105         splay(x);
106         pushr(x);
107     }
108     int find_root(int x)
109     {
110         access(x);
111         splay(x);
112         while (t[x].ch[0])
113             push_down(x), x = t[x].ch[0];
114         splay(x);
115         return x;
116     }
117     void split(int x, int y)
118     {
119         make_root(x);
120         access(y);
121         splay(y);
122     }
123     void link(int x, int y)
124     {
125         make_root(x);
126         if (find_root(y) != x)
127             t[x].fa = y;
128     }
129     int ck(int x, int y)
130     {
131         make_root(x);
132         return find_root(y) != x;
133     }
134     void cut(int x, int y)
135     {
136         make_root(x);
137         if (find_root(y) == x && t[y].fa == x && !t[y].ch[0])
138         {
139             t[x].ch[1] = t[y].fa = 0;

```

```

140         push_up(x);
141     }
142 }
143 }
144
145 void solve(){
146     using namespace LCT;
147     cin >> n >> q;
148     for (int i = 1; i <= n; ++i) {
149         t[i].sz = t[i].tagc = 1;
150         t[i].val = t[i].sum = 1;
151         t[i].taga = 0;
152     }
153     for (int i = 1; i < n; ++i){
154         int u, v;
155         cin >> u >> v;
156         link(u, v);
157     }
158     while (q--){
159         char op;
160         int x, y, z;
161         cin >> op;
162         if (op == '+'){
163             cin >> x >> y >> z;
164             split(x, y);
165             pusha(y, z);
166         }else if (op == '-'){
167             cin >> x >> y;
168             cut(x, y);
169             cin >> x >> y;
170             link(x, y);
171         }else if (op == '*'){
172             cin >> x >> y >> z;
173             split(x, y);
174             pushc(y, z);
175         }else{
176             cin >> x >> y;
177             split(x, y);
178             cout << t[y].sum % mod << endl;
179         }
180     }
181 }
182
183 signed main(){
184     ios::sync_with_stdio(false), cin.tie(0), cout.tie(0);
185     int t = 1;
186     //cin >> t;
187     while(t--) solve();
188     return 0;
189 }

```

大融合 (子树大小查询)

给定 n 个点，进行 Q 次操作，操作有两种：

$A\ x\ y$ ：在 x, y 之间连一条边，保证 x, y 不连通

$Q\ x\ y$ ：求通过边 (x, y) 的简单路径数量，保证存在边 (x, y)

$N, Q \leq 10^5$

```

1  #include <bits/stdc++.h>
2  #define endl '\n'
3  using namespace std;
4  typedef long double db;
5  typedef long long ll;
6
7  const ll N = 2e5 + 10;
8  const ll mod = 998244353;
9  const ll inf32 = 0x3f3f3f3f;
10 const ll inf64 = 5e18;

```

```

11
12 namespace LCT
13 {
14     struct node
15     {
16         int ch[2], fa, tag;
17         int sz, szx; //sz 实子树, szx 虚子树
18     } t[N];
19     #define isroot(x) ((t[t[x].fa].ch[0] != x) && (t[t[x].fa].ch[1] != x))
20     void pushr(int x)
21     {
22         swap(t[x].ch[0], t[x].ch[1]);
23         t[x].tag ^= 1;
24     }
25     void push_up(int x)
26     {
27         t[x].sz = 1 + t[t[x].ch[0]].sz + t[t[x].ch[0]].szx + t[t[x].ch[1]].sz + t[t[x].ch[1]].szx;
28     }
29     void push_down(int x)
30     {
31         if (t[x].tag)
32         {
33             if (t[x].ch[0])
34                 pushr(t[x].ch[0]);
35             if (t[x].ch[1])
36                 pushr(t[x].ch[1]);
37             t[x].tag = 0;
38         }
39     }
40     void push_all(int x)
41     {
42         if (!isroot(x))
43             push_all(t[x].fa);
44         push_down(x);
45     }
46     void rotate(int x)
47     {
48         int y = t[x].fa;
49         int z = t[y].fa;
50         int k = t[y].ch[1] == x;
51         if (!isroot(y))
52             t[z].ch[t[z].ch[1] == y] = x;
53         t[x].fa = z;
54         t[y].ch[k] = t[x].ch[k ^ 1];
55         t[t[x].ch[k ^ 1]].fa = y;
56         t[x].ch[k ^ 1] = y;
57         t[y].fa = x;
58         push_up(y);
59         push_up(x);
60     }
61     void splay(int x)
62     {
63         push_all(x);
64         while (!isroot(x))
65         {
66             int y = t[x].fa;
67             int z = t[y].fa;
68             if (!isroot(y))
69                 (t[z].ch[1] == y) ^ (t[y].ch[1] == x) ? rotate(x) : rotate(y);
70             rotate(x);
71         }
72     }
73     void access(int x)
74     {
75         for (int y = 0; x; y = x, x = t[x].fa){
76             splay(x);
77             if (t[x].ch[1])
78                 t[x].szx += t[t[x].ch[1]].sz + t[t[x].ch[1]].szx;
79             t[x].ch[1] = y;
80             if (y)
81                 t[x].szx -= t[y].sz + t[y].szx;

```

```

82         push_up(x);
83     }
84
85 }
86 void make_root(int x)
87 {
88     access(x);
89     splay(x);
90     pushr(x);
91 }
92 int find_root(int x)
93 {
94     access(x);
95     splay(x);
96     while (t[x].ch[0])
97         push_down(x), x = t[x].ch[0];
98     splay(x);
99     return x;
100 }
101 void split(int x, int y)
102 {
103     make_root(x);
104     access(y);
105     splay(y);
106 }
107 void link(int x, int y)
108 {
109     split(x, y);
110     t[x].fa = y;
111     t[y].szx += t[x].sz + t[x].szx;
112 }
113 int ck(int x, int y)
114 {
115     make_root(x);
116     return find_root(y) != x;
117 }
118 void cut(int x, int y)
119 {
120     make_root(x);
121     if (find_root(y) == x && t[y].fa == x && !t[y].ch[0])
122     {
123         t[x].ch[1] = t[y].fa = 0;
124         push_up(x);
125     }
126 }
127 }
128
129 void solve(){
130     using namespace LCT;
131     int n, q, x, y;
132     cin >> n >> q;
133     for (int i = 1; i <= n; ++i) t[i].sz = 1;
134     while (q--){
135         char op;
136         cin >> op;
137         if (op == 'A'){
138             cin >> x >> y;
139             link(x, y);
140         }else{
141             cin >> x >> y;
142             split(x, y);
143             cout << 1ll * (t[x].sz + t[x].szx) * (t[y].sz + t[y].szx - t[x].sz - t[x].szx) << endl;
144         }
145     }
146 }
147
148 signed main(){
149     ios::sync_with_stdio(false), cin.tie(0), cout.tie(0);
150     int t = 1;
151     //cin >> t;
152     while(t--) solve();

```

```

153     return 0;
154 }

```

分块

分块 I(区间修改, 区间查询)

```

1  #include <bits/stdc++.h>
2  #define endl '\n'
3  using namespace std;
4  typedef long double db;
5  typedef long long ll;
6
7  const ll N = 2e5 + 10;
8  const ll mod = 998244353;
9  const ll inf32 = 0x3f3f3f3f;
10 const ll inf64 = 5e18;
11
12 int n, m, sq;
13 int v[N], bl[N], tag[N], ans[N];
14
15 void update(int a, int b){
16     for (int i = a; i <= min(bl[a] * sq, b); ++i) {
17         ans[bl[a]] -= (v[i] ^ tag[bl[a]]);
18         v[i] ^= 1;
19         ans[bl[a]] += (v[i] ^ tag[bl[a]]);
20     }
21     if (bl[a] != bl[b]){
22         for (int i = (bl[b] - 1) * sq + 1; i <= b; ++i) {
23             ans[bl[b]] -= (v[i] ^ tag[bl[b]]);
24             v[i] ^= 1;
25             ans[bl[b]] += (v[i] ^ tag[bl[b]]);
26         }
27     }
28     for (int i = bl[a] + 1; i <= bl[b] - 1; ++i){
29         tag[i] ^= 1;
30         ans[i] = sq - ans[i];
31     }
32 }
33
34 ll query(int a, int b){
35     ll res = 0;
36     for (int i = a; i <= min(bl[a] * sq, b); ++i) {
37         res += (v[i] ^ tag[bl[a]]);
38     }
39     if (bl[a] != bl[b]){
40         for (int i = (bl[b] - 1) * sq + 1; i <= b; ++i) {
41             res += (v[i] ^ tag[bl[b]]);
42         }
43     }
44     for (int i = bl[a] + 1; i <= bl[b] - 1; ++i) res += ans[i];
45     return res;
46 }
47
48 void solve(){
49     cin >> n >> m;
50     sq = sqrt(n);
51     for (int i = 1; i <= n; ++i) bl[i] = (i - 1) / sq + 1;
52     for (int i = 1; i <= m; ++i){
53         int op, l, r;
54         cin >> op >> l >> r;
55         if (op == 0) update(l, r);
56         else cout << query(l, r) << endl;
57     }
58 }
59
60 signed main(){
61     ios::sync_with_stdio(false), cin.tie(0), cout.tie(0);
62     int t = 1;
63     //cin >> t;
64     while(t--) solve();

```

```

65     return 0;
66 }

```

分块 II

给出一个长为 n 的数列，以及 n 个操作，操作涉及区间加法，询问区间内小于某个值 x 的元素个数。

```

1  #include <bits/stdc++.h>
2  #define endl '\n'
3  using namespace std;
4  typedef long double db;
5  typedef long long ll;
6
7  const ll N = 2e5 + 10;
8  const ll mod = 998244353;
9  const ll inf32 = 0x3f3f3f3f;
10 const ll inf64 = 5e18;
11
12 int n, blo;
13 int v[N], bl[N], atag[N];
14 vector<int> ve[2005];
15
16 void reset(int x)
17 {
18     ve[x].clear();
19     for (int i = (x - 1) * blo + 1; i <= min(x * blo, n); i++)
20         ve[x].push_back(v[i]);
21     sort(ve[x].begin(), ve[x].end());
22 }
23
24 void add(int a, int b, int c)
25 {
26     for (int i = a; i <= min(bl[a] * blo, b); i++)
27         v[i] += c;
28     reset(bl[a]);
29     if (bl[a] != bl[b])
30     {
31         for (int i = (bl[b] - 1) * blo + 1; i <= b; i++)
32             v[i] += c;
33         reset(bl[b]);
34     }
35     for (int i = bl[a] + 1; i <= bl[b] - 1; i++)
36         atag[i] += c;
37 }
38
39 int query(int a, int b, int c)
40 {
41     int ans = 0;
42     for (int i = a; i <= min(bl[a] * blo, b); i++)
43         if (v[i] + atag[bl[a]] < c)
44             ans++;
45     if (bl[a] != bl[b])
46         for (int i = (bl[b] - 1) * blo + 1; i <= b; i++)
47             if (v[i] + atag[bl[b]] < c)
48                 ans++;
49     for (int i = bl[a] + 1; i <= bl[b] - 1; i++)
50     {
51         int x = c - atag[i];
52         ans += lower_bound(ve[i].begin(), ve[i].end(), x) - ve[i].begin();
53     }
54     return ans;
55 }
56
57 int main()
58 {
59     cin >> n;
60     blo = sqrt(n);
61     for (int i = 1; i <= n; i++)
62         cin >> v[i];
63     for (int i = 1; i <= n; i++)
64     {

```

```

65         bl[i] = (i - 1) / blo + 1;
66         ve[bl[i]].push_back(v[i]);
67     }
68     for (int i = 1; i <= bl[n]; i++)
69         sort(ve[i].begin(), ve[i].end());
70     for (int i = 1; i <= n; i++)
71     {
72         int f, a, b, c;
73         cin >> f >> a >> b >> c;
74         if (f == 0)
75             add(a, b, c);
76         if (f == 1)
77             printf("%d\n", query(a, b, c * c));
78     }
79     return 0;
80 }

```

线性 RMQ

```

1  /**
2   * 线性 RMQ
3   */
4
5  #include <bits/stdc++.h>
6  #include <limits>
7  #define endl '\n'
8  using namespace std;
9  typedef long long ll;
10
11 const int N = 5e4 + 10;
12 const int M = 2e4 + 10;
13 const int L = 80;
14 const int mod = 998244353;
15 const int inf32 = 0x3f3f3f3f;
16 const ll inf64 = 4e18;
17
18 int a[N + M];
19 int highbit[M];
20 int stmax[M][L], stmin[M][L];
21 int premax[M][L], premin[M][L];
22 int sufmax[M][L], sufmin[M][L];
23 int quemax[M][L], quemin[M][L];
24 int stackmax[L], stackmin[L];
25
26 void solve(){
27     int n, q;
28     cin >> n >> q;
29     int B = int(log2(n)); // 块的大小
30     int S = (n - 1) / B + 1; // 块的个数
31     for (int b = 0; b < S; ++b) stmin[b][0] = inf32;
32     for (int i = 0; i < n; ++i){
33         cin >> a[i];
34         stmin[i / B][0] = min(stmin[i / B][0], a[i]);
35         stmax[i / B][0] = max(stmax[i / B][0], a[i]);
36     }
37     for (int b = S - 1; b >= 0; b--){
38         for (int k = 1; b + (1 << k) - 1 < S; ++k){
39             stmin[b][k] = min(stmin[b][k - 1], stmin[b + (1 << (k - 1))][k - 1]);
40             stmax[b][k] = max(stmax[b][k - 1], stmax[b + (1 << (k - 1))][k - 1]);
41         }
42     }
43     for (int b = 0; b < S; ++b){
44         int be = b * B;
45         premin[b][0] = premax[b][0] = a[be];
46         for (int k = 1; k < B; ++k){
47             premin[b][k] = min(premin[b][k - 1], a[be + k]);
48             premax[b][k] = max(premax[b][k - 1], a[be + k]);
49         }
50         sufmin[b][B - 1] = sufmax[b][B - 1] = a[be + B - 1];
51         for (int k = B - 2; k >= 0; --k){
52             sufmin[b][k] = min(sufmin[b][k + 1], a[be + k]);

```



```

53     sufmax[b][k] = max(sufmax[b][k + 1], a[be + k]);
54 }
55 }
56 for (int b = 0; b < S; ++b){
57     int be = b * B;
58     int spmin = 0, nowmin = 0;
59     int spmax = 0, nowmax = 0;
60     for (int i = 0; i < B; ++i){
61         while (spmin && a[be + stackmin[spmin]] > a[be + i]) nowmin ^= 1 << stackmin[spmin--];
62         while (spmax && a[be + stackmax[spmax]] < a[be + i]) nowmax ^= 1 << stackmax[spmax--];
63         quemin[b][i] = (nowmin ^= 1 << (stackmin[++spmin] = i));
64         quemax[b][i] = (nowmax ^= 1 << (stackmax[++spmax] = i));
65     }
66 }
67 for (int i = 2; i <= S; ++i) highbit[i] = highbit[i >> 1] + 1;
68
69 while(q --) {
70     int l, r;
71     cin >> l >> r;
72     l--, r--;
73     int L = l / B, R = r / B;
74     int li = l % B, ri = r % B;
75
76     int mn = inf32, mx = 0;
77     if(L == R) {
78         mn = min(mn, a[l + __builtin_ctz(quemin[R][ri] >> li)]);
79         mx = max(mx, a[l + __builtin_ctz(quemax[R][ri] >> li)]);
80     }
81     else {
82         mn = min(mn, sufmin[L][li]);
83         mn = min(mn, premin[R][ri]);
84         mx = max(mx, sufmax[L][li]);
85         mx = max(mx, premax[R][ri]);
86         int len = R - L - 1;
87         int k = highbit[len];
88         if(len) {
89             mn = min(mn, stmin[L + 1][k]);
90             mn = min(mn, stmin[R - (1 << k)][k]);
91             mx = max(mx, stmax[L + 1][k]);
92             mx = max(mx, stmax[R - (1 << k)][k]);
93         }
94     }
95     cout << mx - mn << endl;
96 }
97 }
98
99 signed main(){
100     ios::sync_with_stdio(false);
101     cin.tie(nullptr);
102     int t = 1;
103     //cin >> t;
104     while(t--) solve();
105     return 0;
106 }

```

区间 [l, r] 有多少数字出现正偶数次

```

1 // https://www.luogu.com.cn/problem/P4135
2 #include <bits/stdc++.h>
3 #define endl '\n'
4
5 using ll = long long;
6
7 constexpr int N = 1e5 + 10;
8 constexpr int M = 320;
9 constexpr int mod = 998244353;
10
11 using namespace std;
12
13 int a[N], block[N], cnt[M][N];
14 int L[M], f[M][M], num[N];

```

```

15
16 int n, c, q, B;
17
18 void solve(){
19     cin >> n >> c >> q;
20     B = sqrt(n);
21     for (int i = 1; i <= n; ++i) {
22         cin >> a[i];
23         block[i] = (i - 1) / B + 1;
24         if (block[i] != block[i - 1]) L[block[i]] = i;
25     }
26     block[n + 1] = block[n] + 1;
27     L[block[n + 1]] = n + 1;
28     for (int i = 1; i <= block[n]; ++i){
29         int t = 0;
30         for (int j = L[i]; j <= n; ++j) {
31             cnt[i][a[j]]++;
32             if ((cnt[i][a[j]] & 1) && cnt[i][a[j]] > 1) t--;
33             else if (cnt[i][a[j]] % 2 == 0) t++;
34             if (block[j] != block[j + 1]) {
35                 f[i][block[j]] = t;
36             }
37         }
38     }
39     int ans = 0;
40     stack<int> st;
41     while(q--){
42         int l, r;
43         cin >> l >> r;
44         l = (l + ans) % n + 1, r = (r + ans) % n + 1;
45         if (l > r) swap(l, r);
46         ans = 0;
47         if (block[l] == block[r]) {
48             for (int i = l; i <= r; ++i) {
49                 num[a[i]]++;
50                 st.push(a[i]);
51             }
52             while (!st.empty()) {
53                 int t = st.top();
54                 st.pop();
55                 if (num[t]){
56                     ans += (num[t] & 1) ^ 1;
57                     num[t] = 0;
58                 }
59             }
60             cout << ans << endl;
61             continue;
62         }
63         if (block[l] + 1 <= block[r] - 1) {
64             ans = f[block[l] + 1][block[r] - 1];
65         }
66         for (int i = l; i < L[block[l] + 1]; ++i) {
67             num[a[i]]++;
68             st.push(a[i]);
69         }
70         for (int i = L[block[r]]; i <= r; ++i) {
71             num[a[i]]++;
72             st.push(a[i]);
73         }
74         while (!st.empty()) {
75             int t = st.top();
76             st.pop();
77             if (!num[t]) continue;
78             if (cnt[block[l] + 1][t] - cnt[block[r]][t] > 0 && (cnt[block[l] + 1][t] - cnt[block[r]][t]) % 2 == 0 &&
↪ num[t] % 2) ans--;
79             else if (cnt[block[l] + 1][t] - cnt[block[r]][t] > 0 && (cnt[block[l] + 1][t] - cnt[block[r]][t]) % 2 &&
↪ num[t] % 2) ans++;
80             else if (cnt[block[l] + 1][t] - cnt[block[r]][t] == 0 && num[t] % 2 == 0) ans++;
81             num[t] = 0;
82         }
83         cout << ans << endl;

```

```

84     }
85 }
86
87 signed main(){
88     ios::sync_with_stdio(false);
89     cin.tie(nullptr);
90     int t = 1;
91     //cin >> t;
92     while(t--> 0) solve();
93     return 0;
94 }

```

区间众数查询 1

```

1  #include <bits/stdc++.h>
2  #define endl '\n'
3
4  using ll = long long;
5
6  constexpr int N = 1e5 + 10;
7  constexpr int M = 320;
8  constexpr int mod = 998244353;
9
10 using namespace std;
11
12 int n, m, q, B;
13 int a[N], b[N], block[N], cnt[M][N], L[M];
14 int c[N];
15 pair<int, int> f[M][M];
16
17 void solve(){
18     int n, q;
19     cin >> n >> q;
20     B = sqrt(n);
21     for (int i = 1; i <= n; ++i) {
22         cin >> a[i];
23         b[i] = a[i];
24         block[i] = (i - 1) / B + 1;
25         if (block[i] != block[i - 1]) L[block[i]] = i;
26     }
27     block[n + 1] = block[n] + 1;
28     L[block[n + 1]] = n + 1;
29     sort(b + 1, b + n + 1);
30     for (int i = 1; i <= n; ++i) a[i] = lower_bound(b + 1, b + n + 1, a[i]) - b;
31     for (int i = 1; i <= block[n]; ++i){
32         int mxcnt = 0, num = n;
33         for (int j = L[i]; j <= n; ++j) {
34             cnt[i][a[j]]++;
35             if (cnt[i][a[j]] > mxcnt) {
36                 mxcnt = cnt[i][a[j]];
37                 num = a[j];
38             }else if (cnt[i][a[j]] == mxcnt) {
39                 num = min(num, a[j]);
40             }
41             if (block[j] != block[j + 1]) {
42                 f[i][block[j]] = {mxcnt, num};
43             }
44         }
45     }
46
47     int ans = 0;
48     stack<int> st;
49     while (q--> 0) {
50         int l, r;
51         cin >> l >> r;
52         l = (l + ans - 1) % n + 1, r = (r + ans - 1) % n + 1;
53         if (l > r) swap(l, r);
54         ans = 0;
55         int mxcnt = 0, num = n;
56         if (block[l] == block[r]) {
57             for (int i = l; i <= r; ++i) {

```

```

58         c[a[i]]++;
59         st.push(a[i]);
60     }
61     while (!st.empty()) {
62         int t = st.top();
63         st.pop();
64         if (c[t]) {
65             if (c[t] > mxcnt) {
66                 mxcnt = c[t];
67                 num = t;
68             } else if (c[t] == mxcnt) {
69                 num = min(num, t);
70             }
71             c[t] = 0;
72         }
73     }
74     ans = b[num];
75     cout << ans << endl;
76     continue;
77 }
78 if (block[l] + 1 <= block[r] - 1) {
79     mxcnt = f[block[l] + 1][block[r] - 1].first;
80     num = f[block[l] + 1][block[r] - 1].second;
81 }
82 for (int i = l; i < L[block[l] + 1]; ++i) {
83     c[a[i]]++;
84     st.push(a[i]);
85 }
86 for (int i = L[block[r]]; i <= r; ++i) {
87     c[a[i]]++;
88     st.push(a[i]);
89 }
90 while (!st.empty()) {
91     int t = st.top();
92     st.pop();
93     if (!c[t]) continue;
94     if (c[t] + cnt[block[l] + 1][t] - cnt[block[r]][t] > mxcnt) {
95         mxcnt = c[t] + cnt[block[l] + 1][t] - cnt[block[r]][t];
96         num = t;
97     } else if (c[t] + cnt[block[l] + 1][t] - cnt[block[r]][t] == mxcnt) {
98         num = min(num, t);
99     }
100     c[t] = 0;
101 }
102 ans = b[num];
103 cout << ans << endl;
104 }
105 }
106
107 signed main(){
108     ios::sync_with_stdio(false);
109     cin.tie(nullptr);
110     int t = 1;
111     //cin >> t;
112     while(t--) solve();
113     return 0;
114 }

```

区间众数查询 2

```

1  #include <bits/stdc++.h>
2  #define endl '\n'
3
4  using ll = long long;
5
6  constexpr int N = 5e5 + 10;
7  constexpr int M = 710;
8  constexpr int mod = 998244353;
9
10 using namespace std;
11

```

```

12 int a[N], b[N], block[N], L[M], R[M], f[M][M], tot[N], pos[N];
13 vector<int> v[N];
14 int B;
15
16 void solve(){
17     int n, m;
18     cin >> n >> m;
19     for (int i = 1; i <= n; ++i) {
20         cin >> a[i];
21         b[i] = a[i];
22     }
23     sort(b + 1, b + n + 1);
24     int len = unique(b + 1, b + n + 1) - b - 1;
25     for (int i = 1; i <= n; ++i) {
26         a[i] = lower_bound(b + 1, b + len + 1, a[i]) - b;
27     }
28     for (int i = 1; i <= n; ++i) {
29         v[a[i]].push_back(i);
30         pos[i] = v[a[i]].size();
31         pos[i]--;
32     }
33     int B = sqrt(n);
34     for (int i = 1; i <= n; ++i) {
35         block[i] = (i - 1) / B + 1;
36     }
37     for (int i = 1; i <= block[n]; ++i) {
38         L[i] = (i - 1) * B + 1;
39         R[i] = i * B;
40     }
41     R[block[n]] = n;
42     for (int i = 1; i <= block[n]; ++i) {
43         memset(tot, 0, sizeof tot);
44         for (int j = i; j <= block[n]; ++j) {
45             f[i][j] = f[i][j - 1];
46             for (int k = L[j]; k <= R[j]; ++k) {
47                 f[i][j] = max(f[i][j], ++tot[a[k]]);
48             }
49         }
50     }
51
52     auto query = [&](int l, int r) -> int {
53         int ans = 0;
54         if (block[l] == block[r]) {
55             for (int i = l; i <= r; ++i) tot[a[i]] = 0;
56             for (int i = l; i <= r; ++i) ans = max(ans, ++tot[a[i]]);
57             return ans;
58         }
59         ans = f[block[l] + 1][block[r] - 1];
60         for (int i = l; i <= R[block[l]]; ++i) {
61             auto it = pos[i];
62             while (it + ans < v[a[i]].size() && v[a[i]][it + ans] <= r) {
63                 ans++;
64             }
65         }
66         for (int i = L[block[r]]; i <= r; ++i) {
67             auto it = pos[i];
68             while (it - ans >= 0 && v[a[i]][it - ans] >= l) ++ans;
69         }
70         return ans;
71     };
72     int lstans = 0;
73     while (m--) {
74         int l, r;
75         cin >> l >> r;
76         l ^= lstans, r ^= lstans;
77         if (l > r) swap(l, r);
78         cout << (lstans = query(l, r)) << endl;
79     }
80 }
81
82 signed main(){

```

```

83     ios::sync_with_stdio(false);
84     cin.tie(nullptr);
85     int t = 1;
86     //cin >> t;
87     while(t--) solve();
88     return 0;
89 }

```

Persistent Data Structure

区间第 K 小

```

1  //HH 的项链
2  #include <bits/stdc++.h>
3  #define endl '\n'
4  #define pll pair<ll, ll>
5  #define tll tuple<ll, ll, ll>
6  #define all(a) a.begin() + 1, a.end()
7  using namespace std;
8  using i64 = long long;
9  const i64 N = 1e6 + 10;
10 const i64 mod = 998244353;
11 const int inf32 = 1e9;
12 const i64 inf64 = 5e18;
13
14 int lst[N];
15 int root[N];
16 struct PersistentTree{
17     int ls[N << 5], rs[N << 5], sum[N << 5], tot = 0;
18     //单点修改
19     inline void update(int & rt, int old, int l, int r, int p, int k){
20         rt = ++tot;
21         ls[rt] = ls[old], rs[rt] = rs[old], sum[rt] = sum[old] + k;
22         if (l == r) return;
23         int mid = l + r >> 1;
24         if (p <= mid) update(ls[rt], ls[old], l, mid, p, k);
25         else update(rs[rt], rs[old], mid + 1, r, p, k);
26     }
27     //区间查询
28     inline int query(int rt, int l, int r, int p){
29         if (l == r) return sum[rt];
30         int mid = l + r >> 1;
31         if (p <= mid) return query(ls[rt], l, mid, p) + sum[rs[rt]];
32         else return query(rs[rt], mid + 1, r, p);
33     }
34 }T;
35
36 void solve(){
37     int n;
38     cin >> n;
39     vector<int> a(n + 1);
40     for (int i = 1; i <= n; ++i) cin >> a[i];
41     for (int i = 1, tmp; i <= n; ++i){
42         if (!lst[a[i]]) T.update(root[i], root[i - 1], 1, n, i, 1);
43         else{
44             T.update(tmp, root[i - 1], 1, n, lst[a[i]], -1);
45             T.update(root[i], tmp, 1, n, i, 1);
46         }
47         lst[a[i]] = i;
48     }
49     int q;
50     cin >> q;
51     while (q--){
52         int l, r;
53         cin >> l >> r;
54         cout << T.query(root[r], 1, n, l) << endl;
55     }
56 }
57
58
59 int main(){

```

```

60     ios::sync_with_stdio(false), cin.tie(0), cout.tie(0);
61     int t = 1;
62     //cin >> t;
63     while(t--){
64         solve();
65     }
66     return 0;
67 }

```

最大异或和

```

1  #include <bits/stdc++.h>
2  #define endl '\n'
3  using namespace std;
4  typedef long double db;
5  typedef long long ll;
6
7  const ll N = 6e5 + 10;
8  const ll mod = 998244353;
9  const ll inf32 = 0x3f3f3f3f;
10 const ll inf64 = 5e18;
11
12 int a[N], s[N];
13 struct Trie
14 {
15     int cnt, rt[N], ch[N * 33][2], val[N * 33];
16     void insert(int o, int lst, int v)
17     {
18         for (int i = 28; i >= 0; i--)
19             {
20                 val[o] = val[lst] + 1; // 在原本的基础上更新
21                 if ((v & (1 << i)) == 0)
22                     {
23                         if (!ch[o][0])
24                             ch[o][0] = ++cnt;
25                         ch[o][1] = ch[lst][1];
26                         o = ch[o][0];
27                         lst = ch[lst][0];
28                     }
29                 else
30                     {
31                         if (!ch[o][1])
32                             ch[o][1] = ++cnt;
33                         ch[o][0] = ch[lst][0];
34                         o = ch[o][1];
35                         lst = ch[lst][1];
36                     }
37             }
38         val[o] = val[lst] + 1;
39     }
40
41     int query(int o1, int o2, int v)
42     {
43         int ret = 0;
44         for (int i = 28; i >= 0; i--)
45             {
46                 int t = ((v & (1 << i)) ? 1 : 0);
47                 if (val[ch[o1][!t]] - val[ch[o2][!t]])
48                     ret += (1 << i), o1 = ch[o1][!t],
49                     o2 = ch[o2][!t]; // 尽量向不同的地方跳
50                 else
51                     o1 = ch[o1][t], o2 = ch[o2][t];
52             }
53         return ret;
54     }
55 } st;
56
57 void solve()
58 {
59     int n, q;
60     cin >> n >> q;

```

```

61     for (int i = 1; i <= n; ++i) cin >> a[i], s[i] = s[i - 1] ^ a[i];
62     for (int i = 1; i <= n; ++i){
63         st.rt[i] = ++st.cnt;
64         st.insert(st.rt[i], st.rt[i - 1], s[i]);
65     }
66     while (q--){
67         char op;
68         cin >> op;
69         if (op == 'A'){
70             ++n;
71             cin >> a[n];
72             s[n] = s[n - 1] ^ a[n];
73             st.rt[n] = ++st.cnt;
74             st.insert(st.rt[n], st.rt[n - 1], s[n]);
75         }else{
76             int l, r, x;
77             cin >> l >> r >> x;
78             l--, r--;
79             if (l == 0) cout << max(s[n] ^ x, st.query(st.rt[r], st.rt[0], s[n] ^ x)) << endl;
80             else cout << st.query(st.rt[r], st.rt[l - 1], s[n] ^ x) << endl;
81         }
82     }
83 }
84
85 signed main()
86 {
87     ios::sync_with_stdio(false), cin.tie(0), cout.tie(0);
88     int t = 1;
89     // cin >> t;
90     while (t--)
91         solve();
92     return 0;
93 }

```

区间 MEX

```

1  #include <bits/stdc++.h>
2  #define endl '\n'
3  #define pll pair<i64, i64>
4  #define tll tuple<i64, i64, i64>
5  #define all(a) a.begin() + 1, a.end()
6  using namespace std;
7  using db = long double;
8  using i64 = long long;
9  const i64 N = 3e5 + 10;
10 const i64 mod = 998244353;
11 const i64 inf32 = 1e9;
12 const i64 inf64 = 5e18;
13
14 int rt[N];
15 struct PersistentTree{
16     int ls[N << 5], rs[N << 5], mn[N << 5], sum[N << 5], tot = 0;
17     inline void build(int &rt, int l, int r){
18         rt = ++tot;
19         if (l == r) return;
20         int mid = l + r >> 1;
21         build(ls[rt], l, mid);
22         build(rs[rt], mid + 1, r);
23     }
24     inline void update(int &rt, int old, int l, int r, int p, int k){
25         rt = ++tot;
26         ls[rt] = ls[old], rs[rt] = rs[old], mn[rt] = mn[old], sum[rt] = sum[old];
27         if (l == r){
28             mn[rt] = k;
29             sum[rt]++;
30             return;
31         }
32         int mid = l + r >> 1;
33         if (p <= mid) update(ls[rt], ls[old], l, mid, p, k);
34         else update(rs[rt], rs[old], mid + 1, r, p, k);
35         mn[rt] = min(mn[ls[rt]], mn[rs[rt]]);

```



```

36     sum[rt] = sum[ls[rt]] + sum[rs[rt]];
37 }
38 inline int query(int rt, int old, int p){
39     if (mn[rt] >= p) return sum[rt] - sum[old];
40     if (ls[rt] == 0) return 0;
41     int res = 0;
42     if (mn[ls[rt]] >= p) res = sum[ls[rt]] - sum[ls[old]] + query(rs[rt], rs[old], p);
43     else res = query(ls[rt], ls[old], p);
44     return res;
45 }
46 }T;
47
48 void solve(){
49     int n, q;
50     cin >> n;
51     T.build(rt[0], 1, n);
52     vector<int> a(n + 1);
53     for (int i = 1; i <= n; ++i) {
54         cin >> a[i];
55         if (a[i] <= n) T.update(rt[i], rt[i - 1], 1, n, a[i], i);
56         else rt[i] = rt[i - 1];
57     }
58     cin >> q;
59     while (q--){
60         int l, r;
61         cin >> l >> r;
62         cout << (r - l + 1) - T.query(rt[r], rt[l - 1], l) << endl;
63     }
64 }
65
66 int main(){
67     ios::sync_with_stdio(false), cin.tie(0), cout.tie(0);
68     int t = 1;
69     //cin >> t;
70     while(t--){ solve();
71         return 0;
72     }

```

区间多少数出现多次

```

1 //采花 60Pts
2 #include <bits/stdc++.h>
3 #define endl '\n'
4 #define pll pair<ll, ll>
5 #define tll tuple<ll, ll, ll>
6 #define all(a) a.begin() + 1, a.end()
7 using namespace std;
8 using i64 = long long;
9 using db = long double;
10 const i64 N = 2e6 + 10;
11 const i64 mod = 998244353;
12 const i64 inf32 = 1e9;
13 const i64 inf64 = 5e18;
14
15 int lst[N], pre[N];
16 int root[N];
17 struct PersistentTree{
18     int ls[N << 5], rs[N << 5], sum[N << 5], tot = 0;
19     //单点修改
20     inline void update(int & rt, int old, int l, int r, int p, int k){
21         rt = ++tot;
22         ls[rt] = ls[old], rs[rt] = rs[old], sum[rt] = sum[old] + k;
23         if (l == r) return;
24         int mid = l + r >> 1;
25         if (p <= mid) update(ls[rt], ls[old], l, mid, p, k);
26         else update(rs[rt], rs[old], mid + 1, r, p, k);
27     }
28     //区间查询
29     inline int query(int rt, int l, int r, int p){
30         if (l == r) return sum[rt];
31         int mid = l + r >> 1;

```

```

32         if (p <= mid) return query(ls[rt], l, mid, p) + sum[rs[rt]];
33         else return query(rs[rt], mid + 1, r, p);
34     }
35 }T;
36
37 void solve(){
38     int n, c, q;
39     cin >> n >> c >> q;
40     vector<int> a(n + 1);
41     for (int i = 1; i <= n; ++i) cin >> a[i];
42     for (int i = 1; i <= n; ++i){
43         root[i] = root[i - 1]; //防止第二个 if 无继承
44         if (lst[a[i]]) T.update(root[i], root[i - 1], 1, n, lst[a[i]], 1);
45         if (pre[lst[a[i]]]) T.update(root[i], root[i], 1, n, pre[lst[a[i]]], -1);
46         pre[i] = lst[a[i]];
47         lst[a[i]] = i;
48     }
49     for (int i = 1; i <= q; ++i){
50         int l, r;
51         cin >> l >> r;
52         cout << T.query(root[r], 1, n, l) << endl;
53     }
54 }
55
56 int main(){
57     ios::sync_with_stdio(false), cin.tie(0), cout.tie(0);
58     int t = 1;
59     //cin >> t;
60     while(t--) solve();
61     return 0;
62 }

```

区间种类数

```

1 //HH 的项链
2 #include <bits/stdc++.h>
3 #define endl '\n'
4 #define pll pair<ll, ll>
5 #define tll tuple<ll, ll, ll>
6 #define all(a) a.begin() + 1, a.end()
7 using namespace std;
8 using i64 = long long;
9 const i64 N = 1e6 + 10;
10 const i64 mod = 998244353;
11 const int inf32 = 1e9;
12 const i64 inf64 = 5e18;
13
14 int lst[N];
15 int root[N];
16 struct PersistentTree{
17     int ls[N << 5], rs[N << 5], sum[N << 5], tot = 0;
18     //单点修改
19     inline void update(int & rt, int old, int l, int r, int p, int k){
20         rt = ++tot;
21         ls[rt] = ls[old], rs[rt] = rs[old], sum[rt] = sum[old] + k;
22         if (l == r) return;
23         int mid = l + r >> 1;
24         if (p <= mid) update(ls[rt], ls[old], l, mid, p, k);
25         else update(rs[rt], rs[old], mid + 1, r, p, k);
26     }
27     //区间查询
28     inline int query(int rt, int l, int r, int p){
29         if (l == r) return sum[rt];
30         int mid = l + r >> 1;
31         if (p <= mid) return query(ls[rt], l, mid, p) + sum[rs[rt]];
32         else return query(rs[rt], mid + 1, r, p);
33     }
34 }T;
35
36 void solve(){
37     int n;

```

```

38     cin >> n;
39     vector<int> a(n + 1);
40     for (int i = 1; i <= n; ++i) cin >> a[i];
41     for (int i = 1, tmp; i <= n; ++i){
42         if (!lst[a[i]]) T.update(root[i], root[i - 1], 1, n, i, 1);
43         else{
44             T.update(tmp, root[i - 1], 1, n, lst[a[i]], -1);
45             T.update(root[i], tmp, 1, n, i, 1);
46         }
47         lst[a[i]] = i;
48     }
49     int q;
50     cin >> q;
51     while (q--){
52         int l, r;
53         cin >> l >> r;
54         cout << T.query(root[r], 1, n, l) << endl;
55     }
56 }
57
58
59 int main(){
60     ios::sync_with_stdio(false), cin.tie(0), cout.tie(0);
61     int t = 1;
62     //cin >> t;
63     while(t--){
64         solve();
65     }
66     return 0;
67 }

```

可持久化并查集

```

1  #include <bits/stdc++.h>
2  #define endl '\n'
3  #define pll pair<i64, i64>
4  #define tll tuple<i64, i64, i64>
5  #define all(a) a.begin() + 1, a.end()
6  using namespace std;
7  using db = long double;
8  using i64 = long long;
9  const i64 N = 2e5 + 10;
10 const i64 mod = 998244353;
11 const i64 inf32 = 1e9;
12 const i64 inf64 = 5e18;
13
14 int n, m;
15 int rootfa[N], rootdep[N];
16 struct PersistentDSU{
17     int ls[N << 5], rs[N << 5], val[N << 5], tot = 0, idx = 0;
18     inline void build(int &rt, int l, int r){
19         rt = ++tot;
20         if (l == r){
21             val[rt] = ++idx;
22             return;
23         }
24         int mid = (l + r) >> 1;
25         build(ls[rt], l, mid);
26         build(rs[rt], mid + 1, r);
27     }
28     inline void update(int &rt, int old, int l, int r, int p, int k){
29         rt = ++tot;
30         ls[rt] = ls[old], rs[rt] = rs[old], val[rt] = val[old];
31         if (l == r){
32             val[rt] = k;
33             return;
34         }
35         int mid = (l + r) >> 1;
36         if (p <= mid) update(ls[rt], ls[old], l, mid, p, k);
37         else update(rs[rt], rs[old], mid + 1, r, p, k);
38     }

```

```

39 inline int query(int rt, int l, int r, int p){
40     if (l == r) return val[rt];
41     int mid = (l + r) >> 1;
42     if (p <= mid) return query(ls[rt], l, mid, p);
43     else return query(rs[rt], mid + 1, r, p);
44 }
45 //找到第 ver 个版本中 x 的父亲
46 inline int find(int ver, int x){
47     //去 ver 版本线段树 rootfa[ver] 中查询 x 上的值
48     int fx = query(rootfa[ver], 1, n, x);
49     if (x != fx) x = find(ver, fx); //不能进行路径压缩
50     return x;
51 }
52 //在 ver 个版本中合并集合 x 和集合 y
53 inline void merge(int ver, int x, int y){
54     //千万要注意 rootdep 和 rootfa!!!
55     x = find(ver - 1, x);
56     y = find(ver - 1, y);
57     if (x == y){
58         rootfa[ver] = rootfa[ver - 1];
59         rootdep[ver] = rootdep[ver - 1];
60     }else{
61         int depx = query(rootdep[ver - 1], 1, n, x);
62         int depy = query(rootdep[ver - 1], 1, n, y);
63         if (depx < depy){
64             //fa[x] = y
65             update(rootfa[ver], rootfa[ver - 1], 1, n, x, y);
66             rootdep[ver] = rootdep[ver - 1];
67         }else if (depy < depx){
68             //fa[y] = x
69             update(rootfa[ver], rootfa[ver - 1], 1, n, y, x);
70             rootdep[ver] = rootdep[ver - 1];
71         }else{
72             //fa[x] = y
73             update(rootfa[ver], rootfa[ver - 1], 1, n, x, y);
74             update(rootdep[ver], rootdep[ver - 1], 1, n, y, depy + 1);
75         }
76     }
77 }
78 }D;
79
80 void solve(){
81     cin >> n >> m;
82     D.build(rootfa[0], 1, n);
83     for (int i = 1; i <= m; ++i){
84         int op, a, b;
85         cin >> op;
86         if (op == 1){ //合并
87             cin >> a >> b;
88             D.merge(i, a, b);
89         }else if (op == 2){ //回滚
90             int k;
91             cin >> k;
92             rootfa[i] = rootfa[k];
93             rootdep[i] = rootdep[k];
94         }else if (op == 3){ //查询
95             cin >> a >> b;
96             rootfa[i] = rootfa[i - 1];
97             rootdep[i] = rootdep[i - 1];
98             a = D.find(i, a);
99             b = D.find(i, b);
100             if (a == b) cout << 1 << endl;
101             else cout << 0 << endl;
102         }
103     }
104 }
105
106 int main(){
107     ios::sync_with_stdio(false), cin.tie(0), cout.tie(0);
108     int t = 1;
109     //cin >> t;

```

```

110     while(t--> solve());
111     return 0;
112 }

```

子序列自动机

```

1  #include <bits/stdc++.h>
2  #define endl '\n'
3  #define pll pair<i64, i64>
4  #define tll tuple<i64, i64, i64>
5  #define all(a) a.begin() + 1, a.end()
6  using namespace std;
7  using i64 = long long;
8  using db = long double;
9  const i64 N = 1e5 + 10;
10 const i64 mod = 998244353;
11 const i64 inf32 = 1e9;
12 const i64 inf64 = 5e18;
13
14 int typ, n, q, m;
15 int root[N];
16 struct PersistentTree{
17     int ls[N << 5], rs[N << 5], nxt[N << 5], tot = 0;
18     //单点修改
19     inline void update(int & rt, int old, int l, int r, int p, int k){
20         rt = ++tot;
21         ls[rt] = ls[old], rs[rt] = rs[old], nxt[rt] = nxt[old];
22         if (l == r){
23             nxt[rt] = k;
24             return;
25         }
26         int mid = l + r >> 1;
27         if (p <= mid) update(ls[rt], ls[old], l, mid, p, k);
28         else update(rs[rt], rs[old], mid + 1, r, p, k);
29     }
30     //单点查询
31     inline int query(int rt, int l, int r, int p){
32         if (l == r) return nxt[rt];
33         int mid = l + r >> 1;
34         if (p <= mid) return query(ls[rt], l, mid, p);
35         else return query(rs[rt], mid + 1, r, p);
36     }
37     //建树
38     inline void build(vector<int> a){
39         int n = a.size() - 1;
40         for (int i = n; i >= 1; --i){
41             update(root[i], root[i + 1], 1, m, a[i], i + 1);
42         }
43     }
44 }T;
45
46 void solve(){
47     cin >> typ >> n >> q >> m;
48     vector<int> a(n + 1);
49     for (int i = 1; i <= n; ++i) cin >> a[i];
50     T.build(a);
51     while (q--){
52         int k;
53         cin >> k;
54         bool ok = true;
55         int rt = 1;
56         while (k--){
57             int p;
58             cin >> p;
59             int to = T.query(root[rt], 1, m, p);
60             if (!to) ok = false;
61             if (to) rt = to;
62         }
63         if (ok) cout << "Yes" << endl;
64         else cout << "No" << endl;
65     }

```

```

66 }
67
68 int main(){
69     ios::sync_with_stdio(false), cin.tie(0), cout.tie(0);
70     int t = 1;
71     //cin >> t;
72     while(t--) solve();
73     return 0;
74 }

```

Tree Dp

树的平衡点

```

1 function<void(int, int)> dfs = [&](int u, int fa){
2     sz[u] = 1;
3     for (auto v : G[u]){
4         if (v == fa) continue;
5         dfs(v, u);
6         sz[u] += sz[v];
7         int mx = max(sz[u], n - sz[u]);
8         if (mx <= mn){
9             mn = mx;
10            id = min(u, id);
11        }
12    }
13 };

```

树的最小点覆盖 (最少的点覆盖所有边)

```

1 void dp(int u) {
2     bool fg = 0;
3     for(int i = h[u]; ~i; i = nex[i]) {
4         int j = v[i];
5         fg = 1;
6         dp(j);
7         f[u][0] += f[j][1];
8         f[u][1] += min(f[j][0], f[j][1]);
9     }
10    f[u][1] += 1;
11    if(!fg) {
12        f[u][0] = 0; f[u][1] = 1;
13    }
14 }

```

树的最小支配集 (最少的点覆盖所有点)

$f[i][0]$ 选 i 且 i 及 i 的子树都被覆盖了 $f[i][1]$ 不选 i 且 i 被其儿子覆盖 $f[i][2]$ 不选 i 且 i 被其父亲覆盖 (儿子可选可不选)

```

1 void dfs(int u, int fa){
2     f[u][0] = 1; f[u][1] = f[u][2] = 0;
3     bool ok = false;
4     int tmp = inf32;
5     for (auto v : G[u]){
6         if (v == fa) continue;
7         dfs(v, u);
8         f[u][2] += min(f[v][1], f[v][0]);
9         f[u][0] += min({f[v][0], f[v][1], f[v][2]});
10        if (f[v][0] <= f[v][1]){
11            ok = true;
12            f[u][1] += f[v][0];
13        }else{
14            f[u][1] += f[v][1];
15            tmp = min(tmp, f[v][0] - f[v][1]);
16        }
17    }
18    if (!ok) f[u][1] += tmp;
19 }

```

树的最大独立集 (选定的任意两点之间无边)

```
1 function<void(int, int)> dfs = [&](int u, int fa)
2 {
3     dp[u][1] = h[u];
4     for (auto v : G[u])
5     {
6         if (v == fa)
7             continue;
8         dfs(v, u);
9         dp[u][0] += max(dp[v][1], dp[v][0]);
10        dp[u][1] += dp[v][0];
11    }
12 };
```

基环树最大独立集

```
1 #include <bits/stdc++.h>
2 #define endl '\n'
3
4 using ll = long long;
5 using db = long double;
6
7 constexpr int N = 2e5 + 10;
8 constexpr int mod = 998244353;
9
10 using namespace std;
11
12 void solve(){
13     int n;
14     cin >> n;
15     vector<int> f(n + 1);
16     auto find = [&](auto self, int x) -> int {
17         if (x == f[x]) return x;
18         return f[x] = self(self, f[x]);
19     };
20     vector<int> p(n + 1);
21     int S, T;
22     for (int i = 1; i <= n; ++i) cin >> p[i], f[i] = i;
23     vector<vector<int>> G(n + 1);
24     for (int i = 1; i <= n; ++i) {
25         int u, v;
26         cin >> u >> v;
27         u++, v++;
28         int fu = find(find, u);
29         int fv = find(find, v);
30         if (fu == fv) {
31             S = u, T = v;
32             continue;
33         }
34         G[u].push_back(v);
35         G[v].push_back(u);
36         f[fu] = fv;
37     }
38     double k;
39     cin >> k;
40     ll ans = 0;
41
42     vector dp(n + 1, vector<ll>(2));
43     auto dfs = [&](auto self, int u, int fa) -> void {
44         dp[u][1] = p[u];
45         dp[u][0] = 0;
46         //cerr << u << endl;
47         for (auto v : G[u]) {
48             if (v == fa) continue;
49             self(self, v, u);
50             dp[u][1] += dp[v][0];
51             dp[u][0] += max(dp[v][1], dp[v][0]);
52         }
53     };
54 }
```

```

55     dfs(dfs, S, 0);
56     ans = max(ans, dp[S][0]);
57     dfs(dfs, T, 0);
58     ans = max(ans, dp[T][0]);
59     cout << fixed << setprecision(1) << 1.0 * ans * k << endl;
60 }
61
62 signed main(){
63     ios::sync_with_stdio(false);
64     cin.tie(nullptr);
65     int t = 1;
66     //cin >> t;
67     while(t--> solve());
68     return 0;
69 }

```

树上背包 (最多不超过 m 条边)

```

1  #include <bits/stdc++.h>
2  #include <cstring>
3  #include <vector>
4  #define endl '\n'
5  using namespace std;
6  typedef long double db;
7  typedef long long ll;
8
9  const ll N = 1e2 + 10;
10 const ll mod = 998244353;
11 const ll inf32 = 0x3f3f3f3f;
12 const ll inf64 = 5e18;
13
14 vector<pair<int, int>> G[N];
15 int n, m, dp[N][N], sz[N], tmp[N];
16
17 void dfs(int u, int fa){
18     sz[u] = 1;
19     for (auto [v, w] : G[u]){
20         if (v == fa) continue;
21         dfs(v, u);
22         memset(tmp, 0, sizeof tmp);
23         for (int i = sz[u] - 1; i >= 0; --i){
24             for (int j = sz[v] - 1; j >= 0; --j){
25                 int nxt = i + j + 1;
26                 if (nxt <= m){
27                     tmp[nxt] = max(tmp[nxt], dp[u][i] + dp[v][j] + w);
28                 }
29             }
30         }
31         sz[u] += sz[v];
32         for (int i = 0; i <= sz[u] - 1; ++i)
33             dp[u][i] = max(dp[u][i], tmp[i]);
34     }
35 }
36
37 void solve(){
38     cin >> n >> m;
39     for (int i = 1; i < n; ++i){
40         int u, v, w;
41         cin >> u >> v >> w;
42         G[u].emplace_back(v, w);
43         G[v].emplace_back(u, w);
44     }
45     dfs(1, 0);
46     int ans = 0;
47     for (int i = 1; i <= m; ++i){
48         ans = max(ans, dp[1][i]);
49     }
50     cout << ans << endl;
51 }
52
53 signed main(){

```



```

54     ios::sync_with_stdio(false), cin.tie(0), cout.tie(0);
55     int t = 1;
56     //cin >> t;
57     while(t--) solve();
58     return 0;
59 }

```

2022CCPC-A(树上背包)

爱丽丝想在公园里找到她丢失的猫。

爱丽丝想在公园里找到她丢失的猫。

公园是一棵有根的树，由 n 个顶点组成。顶点的编号从 1 到 n ，根顶点为 1。

爱丽丝现在位于顶点 1。她知道猫已经从顶点 1 跑到了树的某片叶子上，而且没有顶点被访问超过一次。叶子是没有子顶点的顶点。

每个顶点上都有一个监视器。顶点 i 上的监视器可以观察到猫是否访问了顶点 i ，以及猫去了哪个顶点 (如果顶点 i 不是叶子)。爱丽丝需要花费 a_i 秒来检查 i_{th} 监视器的数据。

爱丽丝也可以自己搜索一些叶子。搜索 i 个叶子需要 t_i 秒。请注意， i 是顶点的计数，而不是顶点的标签。

帮助爱丽丝确定要检查哪些监视器和搜索哪些树叶，以便唯一确定猫的位置，并尽可能减少所需的总时间。请注意，要检查的监视器和要搜索的树叶应该在一开始就决定好，之后不得更改。

找出最短的时间。

```

1  #include <bits/stdc++.h>
2  #include <vector>
3  #define endl '\n'
4  using namespace std;
5  typedef long double db;
6  typedef long long ll;
7
8  const ll N = 2e3 + 10;
9  const ll mod = 998244353;
10 const ll inf32 = 0x3f3f3f3f;
11 const ll inf64 = 5e18;
12
13 vector<int> G[N];
14 ll sz[N], a[N], t[N];
15 ll dp[N][N][2], g[N][N];
16 ll tmp1[N][2], tmp2[N];
17
18
19 void dfs(int u, int fa){
20     if (G[u].size() == 1 && fa){
21         sz[u] = 1;
22         dp[u][0][0] = dp[u][1][1] = 0;
23         dp[u][1][0] = a[u];
24         return;
25     }
26     dp[u][0][0] = 0;
27     g[u][0] = 0;
28     for (auto v : G[u]){
29         if (v == fa) continue;
30         dfs(v, u);
31         memset(tmp1, 0x3f, sizeof tmp1);
32         memset(tmp2, 0x3f, sizeof tmp2);
33         for (int i = sz[u]; i >= 0; --i){
34             for (int j = sz[v]; j >= 0; --j){
35                 int nxt = i + j;
36                 tmp1[nxt][0] = min(tmp1[nxt][0], dp[u][i][0] + dp[v][j][0]);
37                 tmp1[nxt][1] = min({tmp1[nxt][1], dp[u][i][0] + dp[v][j][1], dp[u][i][1] + dp[v][j][0]});
38                 tmp2[nxt] = min(tmp2[nxt], g[u][i] + min(dp[v][j][0], dp[v][j][1]));
39             }
40         }
41         sz[u] += sz[v];
42         memcpy(dp[u], tmp1, sizeof tmp1);
43         memcpy(g[u], tmp2, sizeof tmp2);
44     }

```

```

45     for (int i = 1; i <= sz[u]; ++i)
46         dp[u][i][0] = min(dp[u][i][0], g[u][i] + a[u]);
47 }
48
49 void solve(){
50     int n;
51     cin >> n;
52     for (int i = 1; i <= n; ++i) cin >> a[i];
53     for (int i = 1; i <= n; ++i) cin >> t[i];
54     for (int i = 1; i <= n; ++i) sz[i] = 0;
55     for (int i = 1; i <= n; ++i) G[i].clear();
56     for (int i = 1, u, v; i < n; ++i){
57         cin >> u >> v;
58         G[u].push_back(v);
59         G[v].push_back(u);
60     }
61     memset(dp, 0x3f, sizeof dp);
62     memset(g, 0x3f, sizeof g);
63     if (n == 1) {
64         cout << 0 << endl;
65         return;
66     }
67     ll ans = inf64;
68     dfs(1, 0);
69     for (int i = 1; i <= sz[1]; ++i){
70         ans = min(ans, t[sz[1] - i] + min(dp[1][i][0], dp[1][i][1]));
71     }
72     cout << ans << endl;
73 }
74
75 signed main(){
76     ios::sync_with_stdio(false), cin.tie(0), cout.tie(0);
77     int t = 1;
78     cin >> t;
79     while(t--) solve();
80     return 0;
81 }

```

```

1  #include <bits/stdc++.h>
2  #include <vector>
3  #define endl '\n'
4  using namespace std;
5  typedef long double db;
6  typedef long long ll;
7
8  const ll N = 2e3 + 10;
9  const ll mod = 998244353;
10 const ll inf32 = 0x3f3f3f3f;
11 const ll inf64 = 5e18;
12
13 vector<int> G[N];
14 ll sz[N], a[N], t[N];
15 ll dp[N][N][2], g[N][N];
16
17 void dfs(int u, int fa){
18     if (G[u].size() == 1 && fa){
19         sz[u] = 1;
20         dp[u][0][0] = dp[u][1][1] = 0;
21         dp[u][1][0] = a[u];
22         return;
23     }
24     dp[u][0][0] = 0;
25     g[u][0] = 0;
26     for (auto v : G[u]){
27         if (v == fa) continue;
28         dfs(v, u);
29         for (int i = sz[u]; i >= 0; --i){
30             for (int j = sz[v]; j >= 0; --j){
31                 int nxt = i + j;
32                 dp[u][nxt][0] = min(dp[u][nxt][0], dp[u][i][0] + dp[v][j][0]);
33                 dp[u][nxt][1] = min({dp[u][nxt][1], dp[u][i][0] + dp[v][j][1], dp[u][i][1] + dp[v][j][0]});
34                 g[u][nxt] = min(g[u][nxt], g[u][i] + min(dp[v][j][0], dp[v][j][1]));

```

```

35     }
36     }
37     sz[u] += sz[v];
38 }
39 for (int i = 1; i <= sz[u]; ++i)
40     dp[u][i][0] = min(dp[u][i][0], g[u][i] + a[u]);
41 }
42
43 void solve(){
44     int n;
45     cin >> n;
46     for (int i = 1; i <= n; ++i) cin >> a[i];
47     for (int i = 1; i <= n; ++i) cin >> t[i];
48     for (int i = 1; i <= n; ++i) sz[i] = 0;
49     for (int i = 1; i <= n; ++i) G[i].clear();
50     for (int i = 1, u, v; i < n; ++i){
51         cin >> u >> v;
52         G[u].push_back(v);
53         G[v].push_back(u);
54     }
55     memset(dp, 0x3f, sizeof dp);
56     memset(g, 0x3f, sizeof g);
57     if (n == 1) {
58         cout << 0 << endl;
59         return;
60     }
61     ll ans = inf64;
62     dfs(1, 0);
63     for (int i = 1; i <= sz[1]; ++i){
64         ans = min(ans, t[sz[1] - i] + min(dp[1][i][0], dp[1][i][1]));
65     }
66     cout << ans << endl;
67 }
68
69 signed main(){
70     ios::sync_with_stdio(false), cin.tie(0), cout.tie(0);
71     int t = 1;
72     cin >> t;
73     while(t--) solve();
74     return 0;
75 }

```

树联通点集 (换根)

```

1  #include <bits/stdc++.h>
2  #include <vector>
3  #define endl '\n'
4  using namespace std;
5  typedef long double db;
6  typedef long long ll;
7
8  const ll N = 1e6 + 10;
9  const ll mod = 1e9 + 7;
10 const ll inf32 = 0x3f3f3f3f;
11 const ll inf64 = 5e18;
12
13 int n;
14 ll f[N], ans[N];
15 vector<int> G[N];
16
17 ll qmi(ll a, ll b = mod - 2){
18     ll res = 1;
19     while (b){
20         if (b & 1) res = res * a % mod;
21         a = a * a % mod;
22         b >>= 1;
23     }
24     return res;
25 }
26
27 void dfs1(int u, int fa){

```

```

28     f[u] = 1;
29     for (auto v : G[u]){
30         if (v == fa) continue;
31         dfs1(v, u);
32         f[u] = f[u] * (f[v] + 1) % mod;
33     }
34 }
35
36 void dfs2(int u, int fa){
37     if (!fa) ans[u] = f[u];
38     else if ((f[u] + 1) % mod == 0){
39         dfs1(u, u);
40         ans[u] = f[u];
41     }else ans[u] = (ans[fa] % mod * qmi(f[u] + 1) + 1) % mod * f[u] % mod;
42     for (auto v : G[u]){
43         if (v == fa) continue;
44         dfs2(v, u);
45     }
46 }
47
48 void solve(){
49     cin >> n;
50     for (int i = 1; i < n; ++i){
51         int u, v;
52         cin >> u >> v;
53         G[u].push_back(v);
54         G[v].push_back(u);
55     }
56     dfs1(1, 1);
57     dfs2(1, 0);
58     for (int i = 1; i <= n; ++i) cout << ans[i] << endl;
59 }
60
61 signed main(){
62     ios::sync_with_stdio(false), cin.tie(0), cout.tie(0);
63     int t = 1;
64     //cin >> t;
65     while(t--) solve();
66     return 0;
67 }

```

树划分联通块大小小于等于 k(树上背包)

另一种背包写法

```

1  #include <bits/stdc++.h>
2  #include <cstring>
3  #include <vector>
4  #define endl '\n'
5  using namespace std;
6  typedef long double db;
7  typedef long long ll;
8
9  const ll N = 2e3 + 10;
10 const ll mod = 998244353;
11 const ll inf32 = 0x3f3f3f3f;
12 const ll inf64 = 5e18;
13
14 int n, k, sz[N];
15 vector<int> G[N];
16 ll dp[N][N], ans = 0; //dp[i][j] i 所在的联通块大小为 j
17
18 void dfs(int u, int fa){
19     sz[u] = dp[u][1] = 1;
20     for (auto v : G[u]){
21         if (v == fa) continue;
22         dfs(v, u);
23         ll sum = 0;
24         for (int i = 1; i <= sz[v] && i <= k; ++i) sum = (sum + dp[v][i]) % mod;
25         for (int i = min(sz[u], k); i >= 1; --i){
26             for (int j = min(sz[v], k); j >= 1; --j){

```

```

27         int nxt = i + j;
28         if (nxt <= k){
29             dp[u][nxt] = (dp[u][nxt] + dp[u][i] * dp[v][j]) % mod;
30         }
31     }
32     dp[u][i] = (dp[u][i] * sum) % mod;
33 }
34 sz[u] += sz[v];
35 }
36 }
37
38 void solve(){
39     cin >> n >> k;
40     for (int i = 1; i < n; ++i){
41         int u, v;
42         cin >> u >> v;
43         G[u].push_back(v);
44         G[v].push_back(u);
45     }
46     dfs(1, 0);
47     for (int i = 1; i <= k && i <= sz[1]; ++i) ans = (ans + dp[1][i]) % mod;
48     cout << ans << endl;
49 }
50
51 signed main(){
52     ios::sync_with_stdio(false), cin.tie(0), cout.tie(0);
53     int t = 1;
54     //cin >> t;
55     while(t--) solve();
56     return 0;
57 }

```

树上子链(点权和最大)

给定一棵树 T，树 T 上每个点都有一个权值。定义一颗树的子链的大小为：这个子链上所有结点的权值和。请在树 T 中找出一条最大的子链并输出。

```

1  #include <bits/stdc++.h>
2  #define endl '\n'
3  #define pll pair<ll, ll>
4  #define tll tuple<ll, ll, ll>
5  #define ios ios::sync_with_stdio(false), cin.tie(0), cout.tie(0)
6  using namespace std;
7  typedef long long ll;
8  const ll maxn = 2e5 + 10;
9  const ll mod = 998244353;
10 vector<ll> G[maxn];
11 ll w[maxn], dis[maxn], ans = -1e18;
12
13 void solve(){
14     int n;
15     cin >> n;
16     for (int i = 1; i <= n; ++i){
17         cin >> w[i];
18     }
19     for (int i = 1; i <= n - 1; ++i){
20         int u, v;
21         cin >> u >> v;
22         G[u].push_back(v);
23         G[v].push_back(u);
24     }
25     function<void(int, int)> dfs = [&](int u, int fa){
26         ll tmp = 0, mx1 = 0, mx2 = 0;
27         for (auto v: G[u]){
28             if (v == fa) continue;
29             dfs(v, u);
30             tmp = dis[v];
31             if (tmp >= mx1){
32                 mx2 = mx1;
33                 mx1 = tmp;
34             }else if (tmp >= mx2){

```

```

35         mx2 = tmp;
36     }
37 }
38 ans = max(ans, mx1 + mx2 + w[u]);
39 dis[u] = mx1 + w[u];
40 };
41 dfs(1, 0);
42 cout << ans << endl;
43 }
44
45 int main(){
46     ios;
47     int t = 1;
48     //cin >> t;
49     while(t--){
50         solve();
51     }
52     return 0;
53 }

```

Nim Cheater(树剖优化 dp)

```

1  #include <bits/stdc++.h>
2  #define endl '\n'
3
4  using ll = long long;
5
6  constexpr int N = 4e4 + 10;
7  constexpr int mod = 998244353;
8
9  using namespace std;
10
11 int dp[16390];
12 int f[32][16390];
13
14 void solve(){
15     int n;
16     cin >> n;
17     vector<int> a(n + 1), b(n + 1), fa(n + 1), ask(n + 1), ans(n + 1);
18     vector<vector<int>> G(n + 1);
19     int lst = 0, rt = 0;
20     for (int i = 1; i <= n; ++i) {
21         string s;
22         cin >> s;
23         if (s[0] == 'A'){
24             cin >> a[i] >> b[i];
25             if (rt == 0) rt = i, lst = i;
26             else{
27                 G[lst].push_back(i);
28                 fa[i] = lst;
29                 lst = i;
30             }
31             ask[i] = i;
32         }else{
33             lst = fa[lst];
34             ask[i] = lst;
35         }
36     }
37     vector<int> sz(n + 1), son(n + 1);
38
39     function<void(int)> dfs1 = [&](int u) {
40         sz[u] = 1;
41         for (auto v : G[u]){
42             dfs1(v);
43             sz[u] += sz[v];
44             if (sz[v] > sz[son[u]]) son[u] = v;
45         }
46     };
47
48     dfs1(rt);
49

```

```

50     function<void(int, int, int)> dfs2 = [&](int u, int num, int sum){
51         memcpy(dp, f[num], sizeof(dp));
52         for (int i = 0; i < 16384; ++i){
53             dp[i] = max(dp[i], f[num][i ^ a[u]] + b[u]);
54         }
55         ans[u] = sum - dp[0];
56         memcpy(f[num], dp, sizeof(f[num]));
57         for (auto v : G[u]){
58             if (v == son[u]) continue;
59             memcpy(f[num + 1], f[num], sizeof(f[num]));
60             dfs2(v, num + 1, sum + b[v]);
61         }
62         if (son[u])
63             dfs2(son[u], num, sum + b[son[u]]);
64     };
65
66     memset(f, -0x3f, sizeof(f));
67     f[0][0] = 0;
68     dfs2(rt, 0, b[rt]);
69     for (int i = 1; i <= n; ++i) cout << ans[ask[i]] << endl;
70 }
71
72 signed main(){
73     ios::sync_with_stdio(false);
74     cin.tie(nullptr);
75     int t = 1;
76     //cin >> t;
77     while(t--) solve();
78     return 0;
79 }

```

树状数组

二维偏序

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  constexpr int MAXN = 1e7 + 5;
5  int sum[MAXN], ans[MAXN];
6  vector<pair<int, int>> vec;
7  vector<tuple<int, int, int, int>> q;
8
9  inline int lowbit(int x) { return x & (-x); }
10
11 void add(int pos, int x)
12 {
13     for (; pos < MAXN; pos += lowbit(pos))
14         sum[pos] += x;
15 }
16
17 int query_presum(int pos)
18 {
19     int ans = 0;
20     for (; pos > 0; pos -= lowbit(pos))
21         ans += sum[pos];
22     return ans;
23 }
24
25 int main()
26 {
27     ios::sync_with_stdio(0), cin.tie(0), cout.tie(0);
28     int n, m, x1, x2, y1, y2;
29     cin >> n >> m;
30     for (int i = 0; i < n; i++)
31     {
32         cin >> x1 >> y1, ++x1, ++y1;
33         vec.emplace_back(x1, y1);
34     }
35     sort(vec.begin(), vec.end());
36     for (int i = 0; i < m; i++)

```

```

37 {
38     cin >> x1 >> y1 >> x2 >> y2;
39     ++x1, ++y1, ++x2, ++y2;
40     q.emplace_back(x1 - 1, y1 - 1, 1, i);
41     q.emplace_back(x1 - 1, y2, -1, i);
42     q.emplace_back(x2, y1 - 1, -1, i);
43     q.emplace_back(x2, y2, 1, i);
44 }
45 sort(q.begin(), q.end());
46 int cur = 0;
47 for (auto [x, y, c, id] : q)
48 {
49     while (cur < n && vec[cur].first <= x)
50         add(vec[cur].second, 1), ++cur;
51     ans[id] += c * query_presum(y);
52 }
53 for (int i = 0; i < m; i++)
54     cout << ans[i] << "\n";
55
56 return 0;
57 }

```

数位 DP

XOR SUM

路易斯喜欢整数。他将非负整数序列 $A = [a_1, a_2, \dots, a_k]$ 的值定义为以下公式 ($\oplus$ 表示位排他性-或):

$$\sum_{i=1}^k \sum_{j=1}^{i-1} a_i \oplus a_j$$

他想知道有多少个不同的序列 A 满足以下条件:

- A 的长度是 k 。
- A 的值是 n 。
- $0 \leq a_i \leq m$ ($1 \leq i \leq k$).

当且仅当存在一个索引 i 时, $[a_1, \dots, a_k]$ 、 $[b_1, \dots, b_k]$ 这两个序列被认为是不同的, 即 $a_i \neq b_i$ 。告诉路易斯答案模块 $10^9 + 7$ 。

输入一行包含三个整数 n, m, k , ($0 \leq n \leq 10^{15}, 0 \leq m \leq 10^{12}, 1 \leq k \leq 18$)。

```

1  #include <bits/stdc++.h>
2  #define endl '\n'
3  using namespace std;
4  typedef long double db;
5  typedef long long ll;
6
7  const ll N = 21;
8  const ll mod = 1e9 + 7;
9  const ll inf32 = 0x3f3f3f3f;
10 const ll inf64 = 5e18;
11 int C[N][N];
12
13 void init(int n){
14     int i, j;
15     for (C[0][0] = i = 1; i <= n; ++i)
16         for (C[i][0] = j = 1; j <= i; ++j)
17             C[i][j] = (C[i - 1][j] + C[i - 1][j - 1]) % mod;
18 }
19
20 ll n, m, k;
21 unordered_map<ll, ll> f[N << 1][N];
22 ll dp(int now, int lim, ll sum){
23     if (sum > n) return 0;
24     if (sum + (k >> 1ll) * (k + 1 >> 1ll) * ((1ll << now + 1) - 1) < n) return 0ll;
25     if (now < 0) return sum == n;
26     if (f[now][lim].count(sum)) return f[now][lim][sum];
27     ll res = 0;

```



```

28     if (m >> now & 1){
29         for (int i = 0; i <= lim; ++i) //之前卡上界的个数
30             for (int j = 0; j <= k - lim; ++j)
31                 (res += C[lim][i] * C[k - lim][j] % mod * dp(now - 1, i, sum + (i + j) * (k - i - j) * (1ll << now)) %
↪ mod) %= mod;
32     }else{
33         for (int j = 0; j <= k - lim; ++j){
34             (res += C[lim][0] * C[k - lim][j] % mod * dp(now - 1, lim, sum + j * (k - j) * (1ll << now)) % mod) %= mod;
35         }
36     }
37     return f[now][lim][sum] = res;
38 }
39
40 void solve(){
41     cin >> n >> m >> k;
42     init(k);
43     ll ans = dp(39, k, 0);
44     cout << ans << endl;
45 }
46
47 signed main(){
48     ios::sync_with_stdio(false), cin.tie(0), cout.tie(0);
49     int t = 1;
50     //cin >> t;
51     while(t--) solve();
52     return 0;
53 }

```

math

区间筛法

```

1  #include <bits/stdc++.h>
2  #define endl '\n'
3  #define int ll
4
5  using ll = long long;
6
7  constexpr int N = 1.2e6 + 10;
8  constexpr int P = 1e5 + 10;
9  constexpr int M = 108;
10 constexpr int mod = 998244353;
11
12 using namespace std;
13
14 bool vis[N];
15 int prime[P], pcnt, ans;
16
17 void init(){
18     for (int i = 2; i < N; ++i) if (!vis[i]){
19         prime[++pcnt] = i;
20         for (int j = 2; j * i < N; ++j){
21             vis[i * j] = 1;
22         }
23     }
24 }
25
26 inline int S(ll x){
27     int res = 0;
28     while(x) res += x % 10, x /= 10;
29     return res;
30 }
31
32 int val[110];
33
34 void solve(){
35     ll n;
36     ans = 0;
37     cin >> n;
38     if (n <= M){
39         for (int i = 1; i <= n; ++i)

```

```

40         if (n % i == S(i)) ans++;
41         cout << ans << endl;
42         return;
43     }
44     vector<pair<ll, int>> a[M + 1];
45     for (int i = 0; i <= M; ++i) val[i] = n - i;
46     for (int i = 1; 1ll * prime[i] * prime[i] <= n; ++i){
47         if (prime[i] <= M){
48             for (int j = 0; j <= M; ++j){
49                 int cnt = 0;
50                 while (val[j] % prime[i] == 0){
51                     val[j] /= prime[i];
52                     cnt++;
53                 }
54                 if (cnt) a[j].push_back({prime[i], cnt});
55             }
56         }else if (n % prime[i] <= M){
57             int j = n % prime[i], cnt = 0;
58             while (val[j] % prime[i] == 0){
59                 val[j] /= prime[i];
60                 cnt++;
61             }
62             if (cnt) a[j].push_back({prime[i], cnt});
63         }
64     }
65     for (int i = 0; i <= M; ++i){
66         if (val[i] > 1) a[i].push_back({val[i], 1});
67     }
68
69     using pt = vector<pair<ll, int>>::iterator;
70     auto dfs = [&](auto self, ll x, pt s, pt t, int res) -> void {
71         if (s == t) {
72             if (S(x) == res) ans += (x > res);
73             return;
74         }
75         auto [y, z] = *s;
76         ++s, self(self, x, s, t, res);
77         for (int i = 1; i <= z; ++i){
78             x *= y;
79             self(self, x, s, t, res);
80         }
81         return;
82     };
83
84     for (int i = 0; i <= M; ++i){
85         dfs(dfs, 1, a[i].begin(), a[i].end(), i);
86     }
87     cout << ans << endl;
88 }
89
90 signed main(){
91     ios::sync_with_stdio(false);
92     cin.tie(nullptr);
93     init();
94     int t = 1;
95     cin >> t;
96     while(t--) solve();
97     return 0;
98 }

```

矩阵四则运算

```

1  #include <bits/stdc++.h>
2  #define endl '\n'
3
4  using ll = long long;
5
6  constexpr int N = 2e5 + 10;
7  constexpr int mod = 1e9 + 7;
8
9  using namespace std;

```

```

10
11 constexpr int SIZE = 100;
12
13 struct Matrix {
14     ll M[SIZE + 5][SIZE + 5];
15
16     void clear() {memset(M, 0, sizeof(M));}
17
18     void reset() {
19         clear();
20         for (int i = 1; i <= SIZE; i++) M[i][i] = 1;
21     }
22
23     Matrix friend operator * (const Matrix & A, const Matrix & B) {
24         Matrix C;
25         C.clear();
26         for (int i = 1; i <= SIZE; i++) {
27             for (int j = 1; j <= SIZE; j++) {
28                 for (int k = 1; k <= SIZE; k++) {
29                     C.M[i][j] = (C.M[i][j] + A.M[i][k] * B.M[k][j]) % mod;
30                 }
31             }
32         }
33         return C;
34     }
35
36     Matrix friend operator + (const Matrix & A, const Matrix & B) {
37         Matrix C;
38         C.clear();
39         for (int i = 1; i <= SIZE; i++) {
40             for (int j = 1; j <= SIZE; j++) {
41                 C.M[i][j] = (A.M[i][j] + B.M[i][j]) % mod;
42             }
43         }
44         return C;
45     }
46
47     Matrix friend operator - (const Matrix & A, const Matrix & B) {
48         Matrix C;
49         C.clear();
50         for (int i = 1; i <= SIZE; i++){
51             for (int j = 1; j <= SIZE; j++){
52                 C.M[i][j] = (A.M[i][j] - B.M[i][j] + mod) % mod;
53             }
54         }
55         return C;
56     }
57
58     Matrix friend operator ^ (const Matrix & A, ll p) {
59         Matrix C;
60         C.reset();
61         auto tmp = A;
62         while (p) {
63             if (p & 1) C = C * tmp;
64             tmp = tmp * tmp;
65             p >>= 1;
66         }
67         return C;
68     }
69 };
70
71 Matrix t3, t4, t5;
72
73 Matrix MatrixSum(const Matrix & A, ll c) {
74     if (c == 1){
75         t3 = A;
76         return A;
77     }
78     Matrix t2 = MatrixSum(A, c / 2ll);
79     if (c % 2) {
80         t4 = t3 * t3 * A;

```

```

81         t5 = t3;
82         t3 = t4;
83         return (t2 + (t2 * t5)) + t3;
84     }else{
85         t4 = t3 * t3;
86         t5 = t3;
87         t3 = t4;
88         return t2 + (t2 * t5);
89     }
90 }
91
92 Matrix f1, f2;
93
94 void solve(){
95     ll n, m, L, R;
96     cin >> n >> m >> L >> R;
97     f1.clear(), f2.clear();
98     vector<vector<ll>> e(2 * n + 1, vector<ll>(2 * n + 1, 0ll));
99     for (int i = 1; i <= n * 2; ++i){
100         for (int j = 1; j <= n * 2; ++j) {
101             e[i][j] = 0;
102         }
103     }
104     for (int i = 1; i <= m; ++i){
105         ll u, v, w;
106         cin >> u >> v >> w;
107         e[u][v] = (e[u][v] + w) % mod;
108     }
109     f1.M[1][1] = 1;
110     for (int i = 2; i <= n; ++i){
111         for (int j = 1; j < i; ++j){
112             f1.M[1][i] = (f1.M[1][i] + f1.M[1][j] * e[j][i]) % mod;
113         }
114     }
115
116     for (int i = 1; i <= n; ++i){
117         for (int j = 1; j < n + i; ++j){
118             if (j <= n) {
119                 f2.M[j][i] = (f2.M[j][i] + e[j][n + i]) % mod;
120             }else {
121                 for (int k = 1; k <= n; ++k){
122                     f2.M[k][i] = (f2.M[k][i] + f2.M[k][j - n] * e[j - n][i]) % mod;
123                 }
124             }
125         }
126     }
127     ll st = L / n; if (L % n == 0) st--;
128     ll ed = R / n; if (R % n == 0) ed--;
129     Matrix f3, f4;
130     f3.clear(), f4.clear();
131     if (!st) {
132         f3 = f1;
133     }else {
134         auto p = f2 ^ st;
135         for (int i = 1; i <= n; ++i){
136             for (int j = 1; j <= n; ++j){
137                 f3.M[1][i] = (f3.M[1][i] + f1.M[1][j] * p.M[j][i]) % mod;
138             }
139         }
140     }
141     ll ans = 0;
142     ll t1 = L % n;
143     if (!t1) t1 = n;
144     if (st == ed){
145         ll t2 = R % n;
146         if (!t2) t2 = n;
147         for (int i = t1; i <= t2; ++i){
148             ans = (ans + f3.M[1][i]) % mod;
149         }
150     }else{
151         for (int i = t1; i <= n; ++i) ans = (ans + f3.M[1][i]) % mod;

```

```

152     f4 = f3;
153     f3.clear();
154     auto p = f2 ^ ed;
155     for (int i = 1; i <= n; ++i){
156         for (int j = 1; j <= n; ++j){
157             f3.M[1][i] = (f3.M[1][i] + f1.M[1][j] * p.M[j][i]) % mod;
158         }
159     }
160     ll t2 = R % n;
161     if (!t2) t2 = n;
162     for (int i = 1; i <= t2; ++i){
163         ans = (ans + f3.M[1][i]) % mod;
164     }
165     if (st != ed - 1){
166         auto p = MatrixSum(f2, ed - st - 1);
167         f3.clear();
168         for (int i = 1; i <= n; ++i) {
169             for (int j = 1; j <= n; ++j){
170                 f3.M[1][i] = (f3.M[1][i] + f4.M[1][j] * p.M[j][i]) % mod;
171             }
172         }
173         for (int i = 1; i <= n; ++i){
174             ans = (ans + f3.M[1][i]) % mod;
175         }
176     }
177 }
178 cout << ans << endl;
179 }
180
181 signed main(){
182     ios::sync_with_stdio(false);
183     cin.tie(nullptr);
184     int t = 1;
185     cin >> t;
186     while(t--) solve();
187     return 0;
188 }

```

构造异或方程组非 0 解

```

1  #include <bits/stdc++.h>
2  #define endl '\n'
3
4  using ll = long long;
5
6  constexpr int N = 2e5 + 10;
7  constexpr int mod = 998244353;
8
9  using namespace std;
10
11 array<bitset<60>, 60> matrix;
12 array<ll, 60> ans = {0};
13
14 void solve(){
15     int n, m;
16     cin >> n >> m;
17     for (int i = 1; i <= m; ++i){
18         int u, v;
19         cin >> u >> v; u--, v--;
20         matrix[u].set(v);
21         matrix[v].set(u);
22     }
23
24     for (int i = 0, j = 0; i < n; ++i){
25         for (int k = j; k < n; ++k){
26             if (matrix[k][i]){
27                 swap(matrix[j], matrix[k]);
28                 for (int l = 0; l < n; ++l){
29                     if (l != j && matrix[l][i]){
30                         matrix[l] ^= matrix[j];
31                     }

```

```

32         }
33         ++j;
34         break;
35     }
36 }
37 }
38
39 for (int i = 0; i < n; ++i){
40     if (matrix[i].count() == 1){
41         cout << "No" << endl;
42         return;
43     }
44     if (!matrix[i].count()) break;
45     int a = 0;
46     while (!matrix[i][a]) ++a;
47     for (int j = a + 1; j < n; ++j){
48         if (matrix[i][j]){
49             ans[a] |= 1ll << n - j - 1;
50             ans[j] |= 1ll << n - j - 1;
51         }
52     }
53 }
54
55 cout << "Yes" << endl;
56 for (int i = 0; i < n; ++i){
57     cout << ans[i] + !ans[i] << " \n"[i == n - 1];
58 }
59 }
60
61 signed main(){
62     ios::sync_with_stdio(false);
63     cin.tie(nullptr);
64     int t = 1;
65     //cin >> t;
66     while(t--> solve());
67     return 0;
68 }

```

4. exgcd

```

1 ll exgcd(ll a, ll b, ll &x, ll &y){
2     if (b == 0)
3         return x = 1, y = 0, a;
4     int r = exgcd(b, a % b, x, y);
5     tie(x, y) = make_tuple(y, x - (a / b) * y);
6     return r;
7 }

```

$ax + by = (a, b)$ 有无数多组解, 假设 x_0, y_0 是其中一解, 则 $(x_0 + k\frac{b}{(a,b)}, y_0 - k\frac{a}{(a,b)})$ 为其所有解, 其中 $(x_0 \% \frac{b}{(a,b)} + \frac{b}{(a,b)}) \% \frac{b}{(a,b)}$ 为其最小正整数解。

线性筛

```

1 void getPrime(){
2     for (int i = 2; i < maxn; ++i){
3         if (!vis[i]) prime[cnt++] = i;
4         for (int j = 0; prime[j] < maxn / i; ++j){
5             vis[prime[j] * i] = 1;
6             if (i % prime[j] == 0) break;
7         }
8     }
9 }

```

埃氏筛求区间质数

```

1 int prime[maxn], cnt;
2 bool vis[maxn];
3 int p[N];
4
5 void getPrime()

```

```

6  {
7      for (int i = 2; i < maxn; ++i)
8      {
9          if (!vis[i])
10             prime[++cnt] = i;
11             for (int j = 1; prime[j] <= (maxn - 10) / i; ++j)
12             {
13                 vis[prime[j] * i] = 1;
14                 if (i % prime[j] == 0)
15                     break;
16             }
17         }
18     }
19
20 void solve()
21 {
22     int l, r;
23     cin >> l >> r;
24     for (int i = 1; i <= cnt && prime[i] <= r; ++i)
25     {
26         int w = (l + prime[i] - 1) / prime[i];
27         for (int j = max(2ll, w); prime[i] * j <= r; ++j)
28         {
29             p[prime[i] * j - l] = 1;
30         }
31     }
32     vi vec;
33     for (int i = max(2ll, l); i <= r; ++i)
34     {
35         if (!p[i - l])
36             vec.push_back(i);
37         p[i - l] = 0;
38     }
39 }
40
41 signed main()
42 {
43     ios;
44     getPrime();
45     int t = 1;
46     cin >> t;
47     while (t--)
48     {
49         solve();
50     }
51     return 0;
52 }

```

质因数分解

```

1  int p[maxn], a[maxn], cnt;
2  void div(int x)
3  {
4      memset(a, 0, sizeof a);
5      cnt = 0;
6      for (int i = 2; i * i <= x; ++i)
7      {
8          if (x % i == 0)
9          {
10             p[++cnt] = i;
11             while (x % i == 0)
12             {
13                 x /= i;
14                 a[cnt]++;
15             }
16         }
17     }
18     if (x != 1)
19     {
20         p[++cnt] = x;
21         a[cnt] = 1;

```

```

22     }
23 }

```

同余方程 (模版)

$ax \equiv 1 \pmod{b}$ 等价于 $ax + by = 1$

```

1  int exgcd(int a, int b, int & x, int & y){
2      if (b == 0){
3          x = 1, y = 0;
4          return a;
5      }
6      int d = exgcd(b, a % b, y, x);
7      y -= a / b * x;
8      return d;
9  }
10
11 void solve(){
12     int a, b;
13     cin >> a >> b;
14     if (gcd(a, b) != 1){
15         cout << -1 << endl;
16         return;
17     }
18     int x, y;
19     int d = exgcd(a, b, x, y);
20     b /= d;
21     cout << (x % b + b) % b << endl;
22 }

```

中国剩余定理 (任意两个模数需要互质)

- 1) 计算所有模数的积 M
- 2) 对于第 i 个方程计算 $M_i = M/m_i$ 在模上 m_i 上逆元, 并计算出 $M_i * inv(M_i, m_i)$
- 3) $ans \equiv \sum_{i=1}^k r_i * M_i * inv(M_i, m_i) \pmod{M}$

```

1  #include <bits/stdc++.h>
2  #define endl '\n'
3  #define pll pair<ll, ll>
4  #define tll tuple<ll, ll, ll>
5  #define vi vector<int>
6  #define vl vector<ll>
7  #define x first
8  #define y second
9  #define all(a) a.begin() + 1, a.end()
10 #define int ll
11 #define rep(i, j, k) for (int i = (j); i <= (k); i++)
12 #define per(i, j, k) for (int i = (j); i >= (k); i--)
13 #define ios ios::sync_with_stdio(false), cin.tie(0), cout.tie(0)
14 using namespace std;
15 using i64 = long long;
16 typedef long long ll;
17 const ll maxn = 2e5 + 10;
18 const ll mod = 998244353;
19 const ll inf32 = 1e9;
20 const ll inf64 = 1e18;
21
22 typedef __int128 i128;
23
24 i128 read()
25 {
26     i128 x = 0;
27     bool f = 0;
28     char c = getchar();
29     while (c < '0' || c > '9')
30     {
31         if (c == '-')
32             f = 1;

```



```

33     c = getchar();
34 }
35 while (c >= '0' && c <= '9')
36 {
37     x = (x << 1) + (x << 3) + (c ^ 48);
38     c = getchar();
39 }
40 return f ? -x : x;
41 }
42
43 inline void write(i128 x)
44 {
45     if (x < 0)
46         putchar('-'), x = -x;
47     if (x > 9)
48         write(x / 10);
49     putchar(x % 10 + '0');
50 }
51
52 i64 exgcd(i64 a, i64 b, i64 &x, i64 &y)
53 {
54     if (b == 0)
55     {
56         x = 1, y = 0;
57         return a;
58     }
59     i64 d = exgcd(b, a % b, y, x);
60     y -= a / b * x;
61     return d;
62 }
63
64 void solve()
65 {
66     i64 p1, r1; // 定义两个 64 位整数 p1 和 r1, 分别表示第一个同余方程的模数和余数
67     int n; // 定义整数 n, 表示同余方程的数量
68     cin >> n; // 从输入中读取 n
69     cin >> p1 >> r1; // 从输入中读取第一个同余方程的模数和余数
70     bool flag = false; // 定义一个标志变量 flag, 用于标记是否存在无解的情况
71
72     for (int i = 0; i < n - 1; i++) // 对于剩下的 n-1 个同余方程
73     {
74         i64 p2, r2; // 定义两个 64 位整数 p2 和 r2, 分别表示当前同余方程的模数和余数
75         cin >> p2 >> r2; // 从输入中读取当前同余方程的模数和余数
76         if (flag == true) // 如果已经发现存在无解的情况, 则跳过剩下的处理
77             continue;
78         i64 y1, y2; // 定义两个 64 位整数 y1 和 y2, 用于存储扩展欧几里得算法的结果
79         i64 d = exgcd(p1, p2, y1, y2); // 调用扩展欧几里得算法, 计算 p1 和 p2 的最大公约数 d, 并得到 y1 和 y2 的值
80         if ((r2 - r1) % d != 0) // 如果 (r2 - r1) 不能被 d 整除, 则说明这两个同余方程无解
81             flag = true; // 设置 flag 为 true, 表示存在无解的情况
82         y1 = (i128)y1 * (r2 - r1) / d % (p2 / d); // 计算新的 y1 的值
83         i64 lc = (i128)p1 * p2 / d; // 计算 p1 和 p2 的最小公倍数 lc
84         r1 = (((i128)p1 * y1 % lc + r1) % lc + lc) % lc; // 计算新的 r1 的值
85         p1 = lc; // 更新 p1 的值为 lc
86     }
87     if (flag) // 如果存在无解的情况
88         cout << -1 << endl; // 输出-1
89     else // 否则
90         cout << r1 << endl; // 输出最终的结果 r1
91 }
92
93 signed main()
94 {
95     ios;
96     int t = 1;
97     //cin >> t;
98     while (t--)
99     {
100         solve();
101     }
102     return 0;
103 }

```

第一个大于 u 的特解

```
1  i64 exgcd(i64 a, i64 b, i64 &x, i64 &y)
2  {
3      if (b == 0)
4      {
5          x = 1, y = 0;
6          return a;
7      }
8      i64 d = exgcd(b, a % b, y, x);
9      y -= a / b * x;
10     return d;
11 }
12
13 int p, e, k, u;
14 int a[3], m[] = {23, 28, 33};
15 void solve()
16 {
17     cin >> p >> e >> k >> u;
18     a[0] = p, a[1] = e, a[2] = k;
19     int a1 = a[0], m1 = m[0];
20     i64 lc;
21     for (int i = 1; i <= 2; i++)
22     {
23         int a2 = a[i], m2 = m[i];
24         i64 y1, y2;
25         i64 d = exgcd(m1, m2, y1, y2);
26         y1 = (i128)y1 * (a2 - a1) / d % (m2 / d);
27         lc = (i128)m1 * m2 / d;
28         a1 = (((i128)m1 * y1 % lc + a1) % lc + lc) % lc;
29         m1 = lc;
30     }
31     if (a1 <= u) a1 += lc;
32     cout << a1 - u << endl;
33 }
```

欧拉降幂

$$a^b \equiv \begin{cases} a^{b \bmod \varphi(p)}, & \gcd(a, p) = 1 \\ a^b, & \gcd(a, p) \neq 1, b < \varphi(p) \pmod{p} \\ a^{b \bmod \varphi(p) + \varphi(p)}, & \gcd(a, p) \neq 1, b \geq \varphi(p) \end{cases}$$

$m \rightarrow \varphi(m) \rightarrow \varphi(\varphi(m)) \rightarrow \dots \rightarrow 1$ 这个过程最多进行 $\log(m)$ 次

```
1  #include <bits/stdc++.h>
2  #define int long long
3  using namespace std;
4
5  const int N = 1e5 + 10;
6
7  int n, m, w[N], q;
8  int phi_m[N], total;
9  bool k;
10
11 int power(int a, int b, int mod) {
12     int res = 1;
13     while (b) {
14         if (b & 1) {
15             res = res * a;
16             if (res >= mod) {
17                 k = true;
18                 res %= mod;
19             }
20         }
21         a = a * a;
22         if (a >= mod) {
23             k = true;
24             a %= mod;
25         }
26         b >>= 1;
27     }
```

```

28     return res;
29 }
30
31 int f(int n) {
32     int res = n;
33     for (int i = 2; i * i <= n; i++) {
34         if (n % i == 0) {
35             res = res / i * (i - 1);
36             while (n % i == 0) n /= i;
37         }
38     }
39     if (n > 1) res = res / n * (n - 1);
40     return res;
41 }
42
43 void init() {
44     int tmp = m;
45     total = 0;
46     phi_m[++total] = tmp;
47     while (tmp != 1) {
48         tmp = f(tmp);
49         phi_m[++total] = tmp;
50     }
51 }
52
53 void solve(int l, int r) {
54     r = min(r, l - 1 + total);
55     int res = 1;
56     for (int i = r; i >= l; i--) {
57         k = false;
58         res = power(w[i], res, phi_m[i - l + 1]);
59         if (k) res += phi_m[i - l + 1];
60     }
61     cout << res % phi_m[l] << "\n";
62 }
63
64 signed main() {
65     cin >> n >> m;
66     init(); // 预处理欧拉函数的值
67     for (int i = 1; i <= n; i++) cin >> w[i];
68     cin >> q;
69     while (q--) {
70         int l, r;
71         cin >> l >> r;
72         solve(l, r);
73     }
74     return 0;
75 }

```

容斥的一点点小结论

错排问题

$$d(n) = \sum_{i=0}^n (-1)^i \binom{n}{i} (n-i)! = n! \sum_{i=0}^n (-1)^i \frac{1}{i!}$$

$$\lim_{n \rightarrow \infty} \frac{d(n)}{n!} = \frac{1}{e}$$

$$N((1-a_1)(1-a_2)\dots(1-a_n)) = \sum_{i=0}^n (-1)^i \sum_{1 \leq j_1 < j_2 < \dots < j_i \leq n} N(a_{j_1} a_{j_2} \dots a_{j_i})$$

$$N(a_1 \dots a_x (1-a_{x+1}) \dots (1-a_{x+n})) = \sum_{i=0}^n (-1)^i \sum_{x \leq j_1 < j_2 < \dots < j_i \leq n} N(a_1 \dots a_x a_{j_1} \dots a_{j_i})$$

图论

(有向图) 强联通分量缩点

强联通分量缩点后的图称为 SCC。以 $O(N+M)$ 的复杂度完成上述全部操作。

缩点后的图拥有拓扑序

```

1  #include <bits/stdc++.h>
2  #define endl '\n'
3
4  using ll = long long;
5
6  constexpr int N = 2e5 + 10;
7  constexpr int mod = 998244353;
8
9  using namespace std;
10
11 struct SCC {
12     int n, now, cnt;
13     vector<vector<int>> ver;
14     vector<int> dfn, low, col, id, S;
15
16     SCC(int n) : n(n), ver(n + 1), low(n + 1), id(n + 1) {
17         dfn.resize(n + 1, -1);
18         col.resize(n + 1, -1);
19         now = cnt = 0;
20     }
21
22     void add(int x, int y) {
23         ver[x].push_back(y);
24     }
25
26     void tarjan(int x) {
27         dfn[x] = low[x] = now++;
28         S.push_back(x);
29         for (auto y : ver[x]) {
30             if (dfn[y] == -1) {
31                 tarjan(y);
32                 low[x] = min(low[x], low[y]);
33             } else if (col[y] == -1) {
34                 low[x] = min(low[x], dfn[y]);
35             }
36         }
37         if (dfn[x] == low[x]) {
38             int pre;
39             cnt++;
40             id[cnt] = x;
41             do {
42                 pre = S.back();
43                 col[pre] = cnt;
44                 S.pop_back();
45             } while (pre != x);
46         }
47     }
48
49     auto work () {
50         for (int i = 1; i <= n; ++i) {
51             if (dfn[i] == -1) tarjan(i);
52         }
53
54         vector<int> siz(cnt + 1);
55         vector<vector<int>> adj(cnt + 1);
56
57         for (int i = 1; i <= n; ++i) {
58             siz[col[i]]++;
59             for (auto j : ver[i]) {
60                 int x = col[i], y = col[j];
61                 if (x != y) {
62                     adj[x].push_back(y);
63                 }
64             }
65         }
66
67         //return tuple{cnt, adj, col, sz};
68         return tuple{cnt, col, siz, id};
69     }
70 };
71

```

```

72 void solve() {
73     int n, m;
74     cin >> n >> m;
75     SCC scc(n);
76     vector<pair<int, int>> e(n + 1);
77     for (int i = 1; i <= m; ++i) {
78         int u, v;
79         cin >> u >> v;
80         e[i] = {u, v};
81         scc.add(u, v);
82     }
83
84     auto [cnt, col, sz, id] = scc.work();
85
86     vector<int> in(n + 1);
87     for (int i = 1; i <= m; ++i) {
88         auto [u, v] = e[i];
89         if (col[u] != col[v]) {
90             in[col[v]]++;
91         }
92     }
93     vector<int> ans;
94     // for (int i = 1; i <= cnt; ++i){
95     //     cout << id[i] << " \n"[i == cnt];
96     // }
97     for (int i = 1; i <= cnt; ++i) {
98         if (in[i] == 0) ans.push_back(id[i]);
99     }
100     sort(ans.begin(), ans.end());
101     cout << ans.size() << endl;
102     for (auto x : ans) {
103         cout << x << " ";
104     }
105     cout << endl;
106 }
107
108 signed main() {
109     ios::sync_with_stdio(false);
110     cin.tie(nullptr);
111     int t = 1;
112     //cin >> t;
113     while (t--) solve();
114     return 0;
115 }

```

(‘无向图’) 割点缩点

割点缩点后的图称为点双连通图 (V - DCC)，该模板可以在 $O(N + M)$ 复杂度内求解图中全部割点、划分点双（颜色相同的点位于同一个点双连通分量中）。

```

1  VDCC(int n) : n(n) {
2      ver.resize(n + 1);
3      dfn.resize(n + 1);
4      low.resize(n + 1);
5      col.resize(2 * n + 1);
6      point.resize(n + 1);
7      S.clear();
8      cnt = now = 0;
9  }
10
11 void add(int u, int v) {
12     if (u == v) return;
13     ver[u].push_back(v);
14     ver[v].push_back(u);
15 }
16
17
18 void tarjan(int u, int rt) {
19     low[u] = dfn[u] = ++now;
20     S.push_back(u);
21     if (u == rt && !ver[u].size()) {

```

```

22         ++cnt;
23         col[cnt].push_back(u);
24         return;
25     }
26     int flag = 0;
27     for (auto v : ver[u]) {
28         if (!dfn[v]) {
29             tarjan(v, rt);
30             low[u] = min(low[u], low[v]);
31             if (dfn[u] <= low[v]) {
32                 flag++;
33                 if (u != rt || flag > 1) {
34                     point[u] = 1;
35                 }
36                 int pre = 0;
37                 cnt++;
38                 do {
39                     pre = S.back();
40                     col[cnt].push_back(pre);
41                     S.pop_back();
42                 } while (pre != v);
43                 col[cnt].push_back(u);
44             }
45         } else {
46             low[u] = min(low[u], dfn[v]);
47         }
48     }
49 }
50 pair<int, vector<vector<int>>> rebuild() { // 新图的顶点数量, 新图
51     work();
52     vector<vector<int>> adj(cnt + 1);
53     for (int i = 1; i <= cnt; ++i) {
54         if (!col[i].size()) {
55             continue;
56         }
57         for (auto j : col[i]) {
58             if (point[j]) {
59                 adj[i].push_back(point[j]);
60                 adj[point[j]].push_back(i);
61             }
62         }
63     }
64     return {cnt, adj};
65 }
66
67 void work() {
68     for (int i = 1; i <= n; ++i) {
69         if (!dfn[i]) tarjan(i, i);
70     }
71 }
72 };

```

(‘无向图’) 边双, 不求桥

割边缩点后的图称为边双连通图 (E - DCC), 该模板可以在 $O(N + M)$ 复杂度内求解图中全部割边、划分边双 (颜色相同的点位于同一个边双连通分量中)。

性质补充: 对于一个边双, 删去任意边后依旧联通; 对于边双中的任意两点, 一定存在两条不相交的路径连接这两个点 (路径上可以有公共点, 但是没有公共边)。

```

1  struct EDCC {
2      int n, now, cnt;
3      vector<vector<pair<int, int>>> ver;
4      vector<int> col, dfn, low, vis, ind, S;
5
6      EDCC(int n) : n(n), ver(n + 1), col(n + 1), dfn(n + 1), low(n + 1), vis(n + 1), ind(n + 1) {
7          now = cnt = 0;
8      }
9
10     void add(int u, int v, int id) {
11         if (u == v) return;

```

```

12     ver[u].emplace_back(v, id << 1);
13     ver[v].emplace_back(u, id << 1 | 1);
14 }
15
16 void tarjan(int u, int fa) {
17     low[u] = dfn[u] = ++now;
18     S.push_back(u);
19     vis[u] = 1;
20     for (auto [v, t] : ver[u]) {
21         if (t == (fa ^ 1)) continue;
22         if (!dfn[v]) {
23             tarjan(v, t);
24             low[u] = min(low[u], low[v]);
25         } else if (vis[v]) {
26             low[u] = min(low[u], dfn[v]);
27         }
28     }
29     if (low[u] == dfn[u]) {
30         int pre;
31         cnt++;
32         do {
33             pre = S.back();
34             vis[pre] = 0;
35             col[pre] = cnt;
36             S.pop_back();
37         } while (pre != u);
38     }
39 }
40
41 auto work() {
42     for (int i = 1; i <= n; ++i) if (!dfn[i]) tarjan(i, 0);
43     vector<int> sz(cnt + 1);
44     vector<vector<int>> adj(cnt + 1);
45     for (int i = 1; i <= n; ++i) {
46         int x = col[i];
47         sz[x]++;
48         for (auto [j, t] : ver[i]) {
49             int y = col[j];
50             if (x != y) {
51                 adj[x].push_back(y);
52                 ind[y]++;
53             }
54         }
55     }
56
57     return tuple{cnt, adj, col, sz, ind};
58 }
59 };
60

```

求桥

下面代码实现了求割边，其中，当 `isbridge[x]` 为真时，`(father[x],x)` 为一条割边。（下面代码无法处理重边）

```

1  int low[MAXN], dfn[MAXN], dfs_clock;
2  bool isbridge[MAXN];
3  vector<int> G[MAXN];
4  int cnt_bridge;
5  int father[MAXN];
6
7  void tarjan(int u, int fa) {
8      father[u] = fa;
9      low[u] = dfn[u] = ++dfs_clock;
10     for (int i = 0; i < G[u].size(); i++) {
11         int v = G[u][i];
12         if (!dfn[v]) {
13             tarjan(v, u);
14             low[u] = min(low[u], low[v]);
15             if (low[v] > dfn[u]) {
16                 isbridge[v] = true;
17                 ++cnt_bridge;

```

```

18     }
19     } else if (dfn[v] < dfn[u] && v != fa) {
20         low[u] = min(low[u], dfn[v]);
21     }
22 }
23 }

```

求无向图中不在奇圈上的点

1. 如果一个双连通分量内的某些顶点在一个奇圈中（即双连通分量含有奇圈），那么这个双连通分量的其他顶点也在某个奇圈中；2. 如果一个双连通分量含有奇圈，则他必定不是一个二分图。反过来也成立，这是一个充要条件。基于以上俩点，Tarjan 求出每个点双连通图分量后，用交叉染色法判断是否存在奇圈！

```

1  #include<bits/stdc++.h>
2  using namespace std;
3  const int maxn = 2e6 + 50;
4  const int N = 1050;
5  struct node{
6      int to, next;
7  } edge[maxn];
8  int top, stack1[maxn];
9  int tot, cnt, dccn;
10 int dfn[N], low[N];
11 int head[N];
12 vector<int> dcc[N];
13 void add(int from, int to){
14     edge[++cnt].to = to; edge[cnt].next = head[from]; head[from] = cnt;
15 }
16 void ins(int u, int v){add(u, v); add(v, u);}
17 void Tarjan(int u, int rt){
18     dfn[u] = low[u] = ++tot;
19     if(rt == u && head[u] == -1){
20         dccn++;
21         dcc[dccn].clear();
22         dcc[dccn].push_back(u);
23         return;
24     }
25     stack1[++top] = u;
26     for(int i = head[u]; i != -1; i = edge[i].next){
27         int v = edge[i].to;
28         if(!dfn[v]){
29             Tarjan(v, rt);
30             low[u] = min(low[u], low[v]);
31             if(low[v] >= dfn[u]){
32                 dccn++;
33                 dcc[dccn].clear();
34                 dcc[dccn].push_back(u);
35                 while(1){
36                     int w = stack1[top];
37                     dcc[dccn].push_back(w);
38                     top--;
39                     if (w == v){ // 做到子树全部弹出为止，不是 u，不然 v 的兄弟也会被弹出
40                         break;
41                     }
42                 }
43             }
44         }
45         else low[u] = min(low[u], dfn[v]);
46     }
47 }
48 int now[N], col[N], vis[N];
49 bool dfs(int u, int c){
50     col[u] = c;
51     for(int i = head[u]; i != -1; i = edge[i].next){
52         int v = edge[i].to;
53         if(!now[v]) continue;
54         if(col[v] == c) return false;
55         if(col[v] == 0 && !dfs(v, 3 - c)) return false;
56     }
57     return true;

```



```

58 }
59 int mp[N][N];
60 void init(){
61     cnt = 0; tot = 0, dccn = 0, top = 0;
62     memset(head, -1, sizeof(head));
63     memset(mp, 0, sizeof(mp));
64     memset(low, 0, sizeof(low));
65     memset(dfn, 0, sizeof(dfn));
66     memset(vis, 0, sizeof(vis));
67 }
68 int main()
69 {
70     std::ios::sync_with_stdio(false);
71     int n, m;
72     while(cin >> n >> m){
73         if(n == 0 && m == 0) break;
74         init();
75         for(int i = 0; i < m; i++){
76             int u, v;
77             cin >> u >> v;
78             mp[u][v] = mp[v][u] = 1;
79         }
80         for(int i = 1; i <= n; i++){
81             for(int j = i + 1; j <= n; j++){
82                 if(mp[i][j] == 0){
83                     ins(i, j);
84                 }
85             }
86         }
87         for(int i = 1; i <= n; i++) if(!dfn[i]) Tarjan(i, i);
88         for(int i = 1; i <= dccn; i++){
89             memset(now, 0, sizeof(now));
90             memset(col, 0, sizeof(col));
91             for(int j = 0; j < dcc[i].size(); j++){
92                 now[dcc[i][j]] = 1;
93             }
94             if(!dfs(dcc[i][0], 1)){
95                 for(int j = 0; j < dcc[i].size(); j++){
96                     vis[dcc[i][j]] = 1;
97                 }
98             }
99         }
100         int ans = 0;
101         for(int i = 1; i <= n; i++) if(!vis[i]) ans++;
102         cout << ans << endl;
103     }
104     return 0;
105 }

```

DAG 最小路径覆盖

```

1  #include <bits/stdc++.h>
2  #define endl '\n'
3
4  using ll = long long;
5
6  constexpr int N = 2e5 + 10;
7  constexpr int mod = 998244353;
8
9  using namespace std;
10
11 struct SCC {
12     int n, now, cnt;
13     vector<vector<int>> ver;
14     vector<int> dfn, low, col, id, S;
15
16     SCC(int n) : n(n), ver(n + 1), low(n + 1), id(n + 1) {
17         dfn.resize(n + 1, -1);
18         col.resize(n + 1, -1);
19         now = cnt = 0;
20     }

```

```

21
22 void add(int x, int y) {
23     ver[x].push_back(y);
24 }
25
26 void tarjan(int x) {
27     dfn[x] = low[x] = now++;
28     S.push_back(x);
29     for (auto y : ver[x]) {
30         if (dfn[y] == -1) {
31             tarjan(y);
32             low[x] = min(low[x], low[y]);
33         } else if (col[y] == -1) {
34             low[x] = min(low[x], dfn[y]);
35         }
36     }
37     if (dfn[x] == low[x]) {
38         int pre;
39         cnt++;
40         id[cnt] = x;
41         do {
42             pre = S.back();
43             col[pre] = cnt;
44             S.pop_back();
45         } while (pre != x);
46     }
47 }
48 };
49
50 void solve(){
51     int n;
52     cin >> n;
53     vector<string> s(n + 1);
54     for (int i = 1; i <= n; ++i) {
55         cin >> s[i];
56     }
57     SCC scc(n);
58     for (int i = 1; i <= n; ++i) {
59         for (int j = 1; j <= n; ++j) {
60             if (s[i].back() == s[j].front()) {
61                 scc.add(i, j);
62             }
63         }
64     }
65
66     for (int i = 1; i <= n; ++i) {
67         if (scc.dfn[i] == -1) scc.tarjan(i);
68     }
69
70     int cnt = scc.cnt;
71
72     vector<int> siz(cnt + 1);
73     vector<vector<int>> adj(cnt + 1, vector<int>(cnt + 1));
74
75     for (int i = 1; i <= n; ++i) {
76         siz[scc.col[i]]++;
77         for (auto j : scc.ver[i]) {
78             int x = scc.col[i], y = scc.col[j];
79             if (x != y) {
80                 adj[x][y] = 1;
81             }
82         }
83     }
84
85     for (int k = 1; k <= cnt; ++k) {
86         for (int i = 1; i <= cnt; ++i) {
87             for (int j = 1; j <= cnt; ++j) {
88                 if (i == j) continue;
89                 if (adj[i][k] && adj[k][j]) adj[i][j] = 1;
90             }
91         }

```

```

92     }
93
94     vector<int> vis(cnt + 1), pr(cnt + 1, -1);
95     int tot = 0;
96     auto dfs = [&](auto self, int u) -> bool {
97         vis[u] = tot;
98         for (int v = 1; v <= cnt; ++v) {
99             if (adj[u][v] && (pr[v] == -1 || (vis[pr[v]] != tot && self(self, pr[v])))) {
100                 pr[v] = u;
101                 return true;
102             }
103         }
104         return false;
105     };
106     int ans = 0;
107     for (int i = 1; i <= cnt; ++i) {
108         tot++;
109         if (dfs(dfs, i)) ans++;
110     }
111     cout << cnt - ans << endl;
112 }
113
114 signed main(){
115     ios::sync_with_stdio(false);
116     cin.tie(nullptr);
117     int t = 1;
118     //cin >> t;
119     while(t--) solve();
120     return 0;
121 }

```

Segment Tree

线段树 I

```

1  #include <bits/stdc++.h>
2  #define endl '\n'
3  #define int ll
4  using namespace std;
5  typedef long double db;
6  typedef long long ll;
7
8  const ll N = 4e5 + 10;
9  const ll mod = 998244353;
10 const ll inf32 = 0x3f3f3f3f;
11 const ll inf64 = 5e18;
12
13 ll a[N], sum[N << 2], tag[N << 2];
14
15 void push_up(int x) {sum[x] = sum[x << 1] + sum[x << 1 | 1];}
16
17 void build(int u, int l, int r){
18     tag[u] = 0;
19     if (l == r) {sum[u] = a[l]; return;}
20     int mid = (l + r) >> 1;
21     build(u << 1, l, mid);
22     build(u << 1 | 1, mid + 1, r);
23     push_up(u);
24 }
25
26 inline void f(int u, int l, int r, int k){
27     tag[u] += k;
28     sum[u] += k * (r - l + 1);
29 }
30
31 inline void push_down(int u, int l, int r){
32     int mid = (l + r) >> 1;
33     f(u << 1, l, mid, tag[u]);
34     f(u << 1 | 1, mid + 1, r, tag[u]);
35     tag[u] = 0;
36 }

```

```

37
38 inline void update(int u, int nl, int nr, int l, int r, int k){
39     if (nl <= l && r <= nr) {f(u, l, r, k); return;}
40     push_down(u, l, r);
41     int mid = (l + r) >> 1;
42     if (nl <= mid) update(u << 1, nl, nr, l, mid, k);
43     if (nr > mid) update(u << 1 | 1, nl, nr, mid + 1, r, k);
44     push_up(u);
45 }
46
47 inline int query(int u, int qx, int qy, int l, int r){
48     ll res = 0;
49     if (qx <= l && r <= qy) return sum[u];
50     int mid = l + r >> 1;
51     push_down(u, l, r);
52     if (qx <= mid) res += query(u << 1, qx, qy, l, mid);
53     if (qy > mid) res += query(u << 1 | 1, qx, qy, mid + 1, r);
54     return res;
55 }
56
57 void solve(){
58     int n, m;
59     cin >> n >> m;
60     for (int i = 1; i <= n; ++i) cin >> a[i];
61     build(1, 1, n);
62     while(m--){
63         int op, x, y, k;
64         cin >> op >> x >> y;
65         if (op == 1) {
66             cin >> k;
67             update(1, x, y, 1, n, k);
68         } else {
69             cout << query(1, x, y, 1, n) << endl;
70         }
71     }
72 }
73
74 signed main(){
75     ios::sync_with_stdio(false), cin.tie(0), cout.tie(0);
76     int t = 1;
77     //cin >> t;
78     while(t--){
79         solve();
80     }
81     return 0;
82 }

```

线段树 II

```

1  #include <bits/stdc++.h>
2  #define endl '\n'
3  using namespace std;
4  typedef long double db;
5  typedef long long ll;
6
7  const ll N = 4e5 + 10;
8  const ll mod = 998244353;
9  const ll inf32 = 0x3f3f3f3f;
10 const ll inf64 = 5e18;
11
12 ll a[N], sum[N << 2], taga[N << 2], tagm[N << 2], p;
13
14 void push_up(int x) {sum[x] = (sum[x << 1] + sum[x << 1 | 1]) % p;}
15
16 void build(int u, int l, int r){
17     taga[u] = 0;
18     tagm[u] = 1;
19     if (l == r) {sum[u] = a[l]; return;}
20     int mid = (l + r) >> 1;
21     build(u << 1, l, mid);
22     build(u << 1 | 1, mid + 1, r);

```

```

23     push_up(u);
24 }
25
26 inline void push_down(int u, int l, int r){
27     int m = l + r >> 1;
28     ll &tm = tagm[u], &ta = taga[u];
29     if (tm != 1){
30         taga[u << 1] = (taga[u << 1] * tm) % p;
31         taga[u << 1 | 1] = (taga[u << 1 | 1] * tm) % p;
32         tagm[u << 1] = (tagm[u << 1] * tm) % p;
33         tagm[u << 1 | 1] = (tagm[u << 1 | 1] * tm) % p;
34         sum[u << 1] = (sum[u << 1] * tm) % p;
35         sum[u << 1 | 1] = (sum[u << 1 | 1] * tm) % p;
36         tm = 1;
37     }
38     if (ta){
39         taga[u << 1] = (taga[u << 1] + ta) % p;
40         taga[u << 1 | 1] = (taga[u << 1 | 1] + ta) % p;
41         sum[u << 1] = (sum[u << 1] + ta * (m - l + 1)) % p;
42         sum[u << 1 | 1] = (sum[u << 1 | 1] + ta * (r - m)) % p;
43         ta = 0;
44     }
45 }
46
47 inline void update1(int u, int nl, int nr, int l, int r, int k){
48     if (nl <= l && r <= nr) {
49         tagm[u] = (tagm[u] * k) % p;
50         taga[u] = (taga[u] * k) % p;
51         sum[u] = (sum[u] * k) % p;
52         return;
53     }
54     int mid = (l + r) >> 1;
55     push_down(u, l, r);
56     if (nl <= mid) update1(u << 1, nl, nr, l, mid, k);
57     if (nr > mid) update1(u << 1 | 1, nl, nr, mid + 1, r, k);
58     push_up(u);
59 }
60
61 inline void update2(int u, int nl, int nr, int l, int r, int k){
62     if (nl <= l && r <= nr) {
63         taga[u] = (taga[u] + k) % p;
64         sum[u] = (sum[u] + k * (r - l + 1)) % p;
65         return;
66     }
67     int mid = (l + r) >> 1;
68     push_down(u, l, r);
69     if (nl <= mid) update2(u << 1, nl, nr, l, mid, k);
70     if (nr > mid) update2(u << 1 | 1, nl, nr, mid + 1, r, k);
71     push_up(u);
72 }
73
74 inline int query(int u, int qx, int qy, int l, int r){
75     ll res = 0;
76     if (qx <= l && r <= qy) return sum[u];
77     int mid = l + r >> 1;
78     push_down(u, l, r);
79     if (qx <= mid) res = (res + query(u << 1, qx, qy, l, mid)) % p;
80     if (qy > mid) res = (res + query(u << 1 | 1, qx, qy, mid + 1, r)) % p;
81     return res;
82 }
83
84 void solve(){
85     int n, m;
86     cin >> n >> m >> p;
87     for (int i = 1; i <= n; ++i) cin >> a[i];
88     build(1, 1, n);
89     while(m--){
90         int op, x, y, k;
91         cin >> op >> x >> y;
92         if (op == 1) {
93             cin >> k;

```

```

94         update1(1, x, y, 1, n, k);
95     } else if (op == 2){
96         cin >> k;
97         update2(1, x, y, 1, n, k);
98     } else{
99         cout << query(1, x, y, 1, n) << endl;
100     }
101 }
102 }
103
104 signed main(){
105     ios::sync_with_stdio(false), cin.tie(0), cout.tie(0);
106     int t = 1;
107     //cin >> t;
108     while(t--){
109         solve();
110     }
111     return 0;
112 }

```

扫描线

```

1  #include <bits/stdc++.h>
2  #define endl '\n'
3  using namespace std;
4  typedef long double db;
5  typedef long long ll;
6
7  const ll N = 2e5 + 10;
8  const ll mod = 998244353;
9  const ll inf32 = 0x3f3f3f3f;
10 const ll inf64 = 5e18;
11
12 struct line{
13     int x1, x2, h, tag;
14 }li[N];
15 bool cmp (line a, line b) {return a.h < b.h;}
16
17 struct Tree{
18     int l, r, len, cnt;
19 }tr[N * 8];
20
21 int d[N], n;
22
23 void build(int p, int l, int r){
24     tr[p] = {l, r};
25     if (l == r) return;
26     int mid = l + r >> 1;
27     build(p << 1, l, mid);
28     build(p << 1 | 1, mid + 1, r);
29 }
30
31 void push_up(int p){
32     int l = tr[p].l, r = tr[p].r;
33     if (tr[p].cnt) tr[p].len = d[r + 1] - d[l];
34     else tr[p].len = tr[p << 1].len + tr[p << 1 | 1].len;
35 }
36
37 void update(int p, int l, int r, int k){
38     if (l <= tr[p].l && tr[p].r <= r) {
39         tr[p].cnt += k;
40         push_up(p);
41         return;
42     }
43     int mid = tr[p].l + tr[p].r >> 1;
44     if (l <= mid) update(p << 1, l, r, k);
45     if (r > mid) update(p << 1 | 1, l, r, k);
46     push_up(p);
47 }
48
49 void solve(){

```

```

50     cin >> n;
51     for (int i = 1; i <= n; ++i){
52         int a, b, c, e;
53         cin >> a >> b >> c >> e;
54         li[i] = {a, c, b, 1};
55         li[i + n] = {a, c, e, -1};
56         d[i] = a, d[i + n] = c;
57     }
58     n <<= 1;
59     sort(li + 1, li + n + 1, cmp);
60     sort(d + 1, d + n + 1);
61     int len = unique(d + 1, d + n + 1) - d - 1;
62     build(1, 1, len - 1); //只需要 lens - 1 个区间位置即可
63     ll ans = 0;
64     for (int i = 1; i < n; ++i){
65         int x1 = lower_bound(d + 1, d + len + 1, li[i].x1) - d;
66         int x2 = lower_bound(d + 1, d + len + 1, li[i].x2) - d;
67         update(1, x1, x2 - 1, li[i].tag);
68         ans += 1ll * (li[i + 1].h - li[i].h) * tr[1].len;
69     }
70     cout << ans << endl;
71 }
72
73 signed main(){
74     ios::sync_with_stdio(false), cin.tie(0), cout.tie(0);
75     int t = 1;
76     //cin >> t;
77     while(t--) solve();
78     return 0;
79 }

```

窗口的星星 (另一个版本的扫描线)

对于每组数据:

第一行 3 个整数 n, W, H 表示有 n 颗星星, 窗口宽为 W , 高为 H 。

接下来 n 行, 每行三个整数 x_i, y_i, l_i 表示星星的坐标在 (x_i, y_i) , 亮度为 l_i 。

输出 T 个整数, 表示每组数据中窗口星星亮度总和的最大值。

```

1  #include <bits/stdc++.h>
2  #define endl '\n'
3  using namespace std;
4  typedef long double db;
5  typedef long long ll;
6
7  const ll N = 2e5 + 10;
8  const ll mod = 998244353;
9  const ll inf32 = 0x3f3f3f3f;
10 const ll inf64 = 5e18;
11
12 ll n, w, h, d[N];
13 struct line{
14     ll x1, x2, h, val;
15     bool operator < (const line& a) const{
16         return (h != a.h) ? h < a.h : val > a.val;
17     }
18 }L[N];
19
20 struct SegmentTree{
21     ll l, r, mx, add;
22 }T[N << 2];
23
24 void push_up(int p){
25     T[p].mx = max(T[p << 1].mx, T[p << 1 | 1].mx);
26 }
27
28 void build(int p, int l, int r){
29     T[p].l = l, T[p].r = r;
30     T[p].mx = T[p].add = 0;

```

```

31     if (l == r) return;
32     int mid = l + r >> 1;
33     build(p << 1, l, mid);
34     build(p << 1 | 1, mid + 1, r);
35     push_up(p);
36 }
37
38 void push_down(int p){
39     if (T[p].add){
40         T[p << 1].add += T[p].add;
41         T[p << 1 | 1].add += T[p].add;
42         T[p << 1].mx += T[p].add;
43         T[p << 1 | 1].mx += T[p].add;
44         T[p].add = 0;
45     }
46 }
47
48 void update(int p, int ql, int qr, ll val){
49     int l = T[p].l, r = T[p].r;
50     if (ql <= l && r <= qr){
51         T[p].add += val;
52         T[p].mx += val;
53         return;
54     }
55     push_down(p);
56     int mid = l + r >> 1;
57     if (ql <= mid) update(p << 1, ql, qr, val);
58     if (qr > mid) update(p << 1 | 1, ql, qr, val);
59     push_up(p);
60 }
61
62 void solve(){
63     cin >> n >> w >> h;
64     for (int i = 1; i <= n; ++i){
65         ll x, y, l;
66         cin >> x >> y >> l;
67         d[(i << 1) - 1] = y;
68         d[(i << 1)] = y + h - 1;
69         L[(i << 1) - 1] = (line){y, y + h - 1, x, l};
70         L[(i << 1)] = (line){y, y + h - 1, x + w - 1, -l};
71     }
72     n <= 1;
73     sort(d, d + n + 1);
74     sort(L + 1, L + n + 1);
75     ll cnt = unique(d, d + n + 1) - d - 1;
76     for (int i = 1; i <= n; ++i){
77         ll x1 = lower_bound(d + 1, d + cnt + 1, L[i].x1) - d;
78         ll x2 = lower_bound(d + 1, d + cnt + 1, L[i].x2) - d;
79         L[i].x1 = x1, L[i].x2 = x2;
80     }
81     build(1, 1, cnt);
82     ll ans = 0;
83     for (int i = 1; i <= n; ++i){
84         update(1, L[i].x1, L[i].x2, L[i].val);
85         ans = max(ans, T[1].mx);
86     }
87     cout << ans << endl;
88 }
89
90 signed main(){
91     ios::sync_with_stdio(false), cin.tie(0), cout.tie(0);
92     int t = 1;
93     cin >> t;
94     while(t--) solve();
95     return 0;
96 }

```

二分+线段树

输入数据的第一行为两个整数 n 和 m , n 表示序列的长度, m 表示局部排序的次数。

第二行为 n 个整数，表示 1 到 n 的一个排列。

接下来输入 m 行，每一行有三个整数 op, l, r ， op 为 0 代表升序排序， op 为 1 代表降序排序， l, r 表示排序的区间。

最后输入一个整数 q ，表示排序完之后询问的位置

输出数据仅有一行，一个整数，表示按照顺序将全部的部分排序结束后第 q 位置上的数字。

```
1  #include <bits/stdc++.h>
2  #define endl '\n'
3  using namespace std;
4  typedef long double db;
5  typedef long long ll;
6
7  const ll N = 1e5 + 10;
8  const ll mod = 998244353;
9  const ll inf32 = 0x3f3f3f3f;
10 const ll inf64 = 5e18;
11
12 struct Sort
13 { // 记录这 m 次排序操作
14     int op, l, r;
15 } q[N];
16 struct Node
17 { // 线段树
18     int l, r; // sum 记录 01 序列中 1 的个数
19     int sum, lazy; // lazy 为懒标记: 1 代表将此段全部变为 1, -1 代表将此段全部变为 0
20 } tr[N * 4];
21 int n, m, k, a[N];
22 void pushup(int u)
23 {
24     tr[u].sum = tr[u << 1].sum + tr[u << 1 | 1].sum;
25 }
26 void pushdown(int u)
27 {
28     if (tr[u].lazy)
29     {
30         tr[u << 1].lazy = tr[u << 1 | 1].lazy = tr[u].lazy;
31         if (tr[u].lazy == 1)
32         {
33             tr[u << 1].sum = tr[u << 1].r - tr[u << 1].l + 1;
34             tr[u << 1 | 1].sum = tr[u << 1 | 1].r - tr[u << 1 | 1].l + 1;
35         }
36         else
37             tr[u << 1].sum = tr[u << 1 | 1].sum = 0;
38         tr[u].lazy = 0;
39     }
40 }
41 void build(int u, int l, int r, int x)
42 {
43     if (l == r)
44         tr[u] = {l, r, a[l] >= x, 0}; // 序列中大于等于 x 的数变为 1, 小于 x 的数变为 0
45     else
46     {
47         tr[u] = {l, r};
48         int mid = l + r >> 1;
49         build(u << 1, l, mid, x), build(u << 1 | 1, mid + 1, r, x);
50         pushup(u);
51     }
52 }
53 int query(int u, int l, int r) // 查询 [l, r] 中 1 的个数
54 {
55     if (l <= tr[u].l && tr[u].r <= r)
56         return tr[u].sum;
57     pushdown(u);
58     int mid = tr[u].l + tr[u].r >> 1;
59     int sum = 0;
60     if (mid >= l)
61         sum = query(u << 1, l, r);
62     if (mid < r)
63         sum += query(u << 1 | 1, l, r);
64     return sum;
```

```

65 }
66 void update(int u, int l, int r, int c) // 将 [l,r] 区间中的数全部变为 c
67 {
68     if (l <= tr[u].l && tr[u].r <= r)
69     {
70         tr[u].sum = c * (tr[u].r - tr[u].l + 1);
71         tr[u].lazy = c ? 1 : -1;
72     }
73     else
74     {
75         pushdown(u);
76         int mid = tr[u].l + tr[u].r >> 1;
77         if (mid >= l)
78             update(u << 1, l, r, c);
79         if (mid < r)
80             update(u << 1 | 1, l, r, c);
81         pushup(u);
82     }
83 }
84 bool queryPoint(int u, int x) // 查询 x 位置上的数是否为 1
85 {
86     if (tr[u].l == tr[u].r)
87         return tr[u].sum;
88     pushdown(u);
89     int mid = tr[u].l + tr[u].r >> 1;
90     if (x <= mid)
91         return queryPoint(u << 1, x);
92     else
93         return queryPoint(u << 1 | 1, x);
94 }
95 bool check(int mid) // 检查此答案值是否合法
96 {
97     build(1, 1, n, mid); // 用此答案值建树
98     for (int i = 1; i <= m; i++)
99     {
100         int op = q[i].op, l = q[i].l, r = q[i].r; // 对 [l,r] 区间进行排序
101         int cnt = query(1, l, r); // 查询 [l,r] 中 1 的个数
102         if (cnt == 0 || cnt == r - l + 1)
103             continue; // 如果区间中的数全部相同, 那么不需要进行排序
104         if (op)
105         {
106             update(1, l, cnt + l - 1, 1);
107             update(1, cnt + l, r, 0);
108         }
109         else
110         {
111             update(1, l, r - cnt, 0);
112             update(1, r - cnt + 1, r, 1);
113         }
114     }
115     return queryPoint(1, k); // 所有操作完成后查看 k 位置上的数是否为 1
116 }
117
118 void solve()
119 {
120     cin >> n >> m;
121     for (int i = 1; i <= n; ++i)
122         cin >> a[i];
123     for (int i = 1; i <= m; ++i)
124     {
125         cin >> q[i].op >> q[i].l >> q[i].r;
126     }
127     cin >> k;
128     int l = 1, r = n;
129     while (l < r)
130     {
131         int mid = l + r + 1 >> 1;
132         if (check(mid))
133             l = mid;
134         else
135             r = mid - 1;

```

```

136     }
137     cout << l << endl;
138 }
139
140 signed main()
141 {
142     ios::sync_with_stdio(false), cin.tie(0), cout.tie(0);
143     int t = 1;
144     // cin >> t;
145     while (t--)
146         solve();
147     return 0;
148 }

```

线段树合并 (HDU2024 暑期多校 1003)

```

1  #include <bits/stdc++.h>
2  #include <vector>
3  #define endl '\n'
4  using namespace std;
5  typedef long double db;
6  typedef long long ll;
7  typedef unsigned long long ull;
8
9  const ll N = 5e5 + 10;
10 const ll mod = 998244353;
11 const ll inf32 = 0x3f3f3f3f;
12 const ll inf64 = 5e18;
13
14 vector<int> G[N];
15 int p[N], rk[N];
16
17 int rt[N], ls[N * 60], rs[N * 60], cnt;
18 int c[N * 60];
19 ull t[N * 60];
20 ull cur = 0;
21 ull ans[N], S1[N], S2[N], a[N];
22
23
24 void push_up(int p){
25     t[p] = t[ls[p]] + t[rs[p]];
26     c[p] = c[ls[p]] + c[rs[p]];
27 }
28
29 void upd(int l, int r, int &p, int x, ull v) {
30     if (!p)
31         p = ++cnt;
32     if (l == r) {
33         c[p] = 1;
34         t[p] = v;
35         return;
36     }
37     int m = (l + r) >> 1;
38     if (x <= m)
39         upd(l, m, ls[p], x, v);
40     else
41         upd(m + 1, r, rs[p], x, v);
42     push_up(p);
43 }
44
45 int merge(int x, int y, ull prev1, ull prev2) {
46     if (!x || !y) {
47         cur += prev1 * t[y];
48         cur += prev2 * t[x];
49         return x + y;
50     }
51     int z = ++cnt;
52     ls[z] = merge(ls[x], ls[y], prev1, prev2);
53     rs[z] = merge(rs[x], rs[y], prev1 + c[ls[x]], prev2 + c[ls[y]]);
54     push_up(z);
55     return z;

```

```

56 }
57
58 void dfs(int u, int fa){
59     upd(0, N, rt[u], rk[u], a[u] * a[u]);
60     S1[u] = a[u];
61     S2[u] = a[u] * a[u];
62     for (auto v : G[u]){
63         if (v == fa) continue;
64         dfs(v, u);
65         S1[u] += S1[v];
66         S2[u] += S2[v];
67         ans[u] += ans[v];
68         cur = 0;
69         rt[u] = merge(rt[u], rt[v], 0, 0);
70         ans[u] += cur;
71     }
72 }
73
74 void solve(){
75     int n;
76     cin >> n;
77     for (int i = 1; i < n; ++i){
78         int u, v;
79         cin >> u >> v;
80         --u, --v;
81         G[u].push_back(v);
82         G[v].push_back(u);
83     }
84     for (int i = 0; i < n; ++i) cin >> a[i];
85     for (int i = 0; i < n; ++i) p[i] = i;
86     sort(p, p + n, [&](int i, int j){return a[i] < a[j];});
87     for (int i = 0; i < n; ++i) rk[p[i]] = i;
88     dfs(0, -1);
89     for (int i = 0; i < n; ++i){
90         ans[i] *= 2;
91         ans[i] += S2[i];
92         ans[i] -= S1[i] * S1[i];
93     }
94     ull res = 0;
95     for (int i = 0; i < n; ++i) res ^= ans[i];
96     cout << res << endl;
97 }
98
99 signed main(){
100     ios::sync_with_stdio(false), cin.tie(0), cout.tie(0);
101     int t = 1;
102     //cin >> t;
103     while(t--) solve();
104     return 0;
105 }

```

动态开点线段树

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  const int N = 100010;
5  #define int long long
6
7  struct node {
8      int l, r;
9      int add, sum;
10 } tr[N << 1];
11 // 正常线段树, 这里不开 4 倍大小会 RE
12
13 int n, m, idx, root;
14 int a[N];
15
16 void pushup(int p) {
17     tr[p].sum = tr[tr[p].l].sum + tr[tr[p].r].sum;
18 }

```

```

19
20 void pushdown(int p, int l, int r) {
21     if(tr[p].add) {
22         int mid = l + r >> 1;
23         tr[tr[p].l].sum += (mid - l + 1) * tr[p].add, tr[tr[p].l].add += tr[p].add;
24         tr[tr[p].r].sum += (r - mid) * tr[p].add, tr[tr[p].r].add += tr[p].add;
25         tr[p].add = 0;
26     }
27 }
28
29 void build(int &p, int l, int r) {
30     if(!p) p = ++idx;
31     if(l == r) { tr[p].sum = a[l]; return; }
32     int mid = l + r >> 1;
33     build(tr[p].l, l, mid), build(tr[p].r, mid + 1, r);
34     pushup(p);
35 }
36
37 void modify(int &p, int l, int r, int ql, int qr, int k) {
38     if(!p) p = ++idx;
39     if(l >= ql && r <= qr) {
40         tr[p].sum += (r - l + 1) * k;
41         tr[p].add += k;
42         return;
43     }
44     pushdown(p, l, r);
45     int mid = l + r >> 1;
46     if(ql <= mid) modify(tr[p].l, l, mid, ql, qr, k);
47     if(qr > mid) modify(tr[p].r, mid + 1, r, ql, qr, k);
48     pushup(p);
49 }
50
51 int query(int p, int l, int r, int ql, int qr) {
52     if(l >= ql && r <= qr) { return tr[p].sum; }
53     int mid = l + r >> 1;
54     pushdown(p, l, r);
55     int v = 0;
56     if(ql <= mid) v = query(tr[p].l, l, mid, ql, qr);
57     if(qr > mid) v += query(tr[p].r, mid + 1, r, ql, qr);
58     return v;
59 }
60
61 signed main() {
62     cin >> n >> m;
63     for (int i = 1; i <= n; i++) cin >> a[i];
64
65     build(root, 1, n);
66
67     int op, x, y, k;
68     while(m--) {
69         cin >> op >> x >> y;
70         if(op == 1) {
71             cin >> k;
72             modify(root, 1, n, x, y, k);
73         } else {
74             cout << query(root, 1, n, x, y) << endl;
75         }
76     }
77     return 0;
78 }

```

线段树 (单点修改、区间最值)

```

1  #include <bits/stdc++.h>
2  #include <vector>
3  #define endl '\n'
4  using namespace std;
5  typedef long long ll;
6
7  const int N = 2e5 + 10;
8  const int mod = 998244353;

```

```

9  const int inf32 = 0x3f3f3f3f;
10 const ll inf64 = 4e18;
11
12 struct Node
13 {
14     int l, r;
15     ll v1, v2;
16 }tr[N * 4];
17
18 ll a[N], b[N];
19
20 void pushup(int u)
21 {
22     tr[u].v1 = max(tr[u << 1].v1, tr[u << 1 | 1].v1);
23     tr[u].v2 = min(tr[u << 1].v2, tr[u << 1 | 1].v2);
24 }
25
26 void build(int u, int l, int r)
27 {
28     tr[u] = {l, r};
29     if (l == r) {
30         tr[u].v1 = tr[u].v2 = b[l];
31         return;
32     }
33     int mid = l + r >> 1;
34     build(u << 1, l, mid), build(u << 1 | 1, mid + 1, r);
35     pushup(u);
36 }
37
38 ll query1(int u, int l, int r)
39 {
40     if (tr[u].l >= l && tr[u].r <= r) return tr[u].v1;
41     int mid = tr[u].l + tr[u].r >> 1;
42     ll v = -inf64;
43     if (l <= mid) v = query1(u << 1, l, r);
44     if (r > mid) v = max(v, query1(u << 1 | 1, l, r));
45     return v;
46 }
47
48 ll query2(int u, int l, int r)
49 {
50     if (tr[u].l >= l && tr[u].r <= r) return tr[u].v2;
51     int mid = tr[u].l + tr[u].r >> 1;
52     ll v = inf64;
53     if (l <= mid) v = query2(u << 1, l, r);
54     if (r > mid) v = min(v, query2(u << 1 | 1, l, r));
55     return v;
56 }
57
58 void modify(int u, int x, ll v)
59 {
60     if (tr[u].l == x && tr[u].r == x) tr[u].v1 += v, tr[u].v2 += v;
61     else
62     {
63         int mid = tr[u].l + tr[u].r >> 1;
64         if (x <= mid) modify(u << 1, x, v);
65         else modify(u << 1 | 1, x, v);
66         pushup(u);
67     }
68 }
69
70
71 void solve(){
72     int n;
73     cin >> n;
74     for (int i = 1; i <= n; ++i)
75         cin >> a[i];
76     for (int i = 2; i <= n; ++i) {
77         b[i] = a[i] - a[i - 1];
78     }
79     build(1, 1, n);

```

```

80     int q;
81     cin >> q;
82     while (q--){
83         int op, l, r;
84         ll x;
85         cin >> op >> l >> r;
86         if (op == 1) {
87             cin >> x;
88             modify(1, l, x);
89             modify(1, r + 1, -x);
90         }else if (op == 2){
91             if (l == r){
92                 cout << 1 << endl;
93             }else{
94                 ll mx = query1(1, l + 1, r);
95                 ll mn = query2(1, l + 1, r);
96                 if (mx == 0 && mn == 0){
97                     cout << 1 << endl;
98                 }else{
99                     cout << 0 << endl;
100                 }
101             }
102         }else if (op == 3){
103             if (l == r){
104                 cout << 1 << endl;
105             }else{
106                 ll mn = query2(1, l + 1, r);
107                 cout << (mn > 0) << endl;
108             }
109         }else if (op == 4){
110             if (l == r){
111                 cout << 1 << endl;
112             }else{
113                 ll mx = query1(1, l + 1, r);
114                 cout << (mx < 0) << endl;
115             }
116         }else {
117             int ql = l + 1, qr = r + 1, ok = 0;
118             while (ql < qr){
119                 int mid = (ql + qr) >> 1;
120                 ll mn = query2(1, l + 1, mid);
121                 if (mn > 0) {
122                     ok = mid;
123                     ql = mid + 1;
124                 }else{
125                     qr = mid;
126                 }
127             }
128             if (!ok) {
129                 cout << 0 << endl;
130             }else{
131                 if (ok >= r) {
132                     cout << 0 << endl;
133                 }else{
134                     ll mx = query1(1, ok + 1, r);
135                     if (mx < 0) {
136                         cout << 1 << endl;
137                     }else{
138                         cout << 0 << endl;
139                     }
140                 }
141             }
142         }
143     }
144 }
145
146 signed main(){
147     ios::sync_with_stdio(false);
148     cin.tie(nullptr);
149     int t = 1;
150     //cin >> t;

```

```

151     while(t--) solve();
152     return 0;
153 }

```

单点修改，区间最大字段和

```

1  #include<bits/stdc++.h>
2  using namespace std;
3
4  #define int long long
5
6  const int maxn = 1001000;
7  int n, m;
8  int ans;
9
10 struct tree{
11     int lmax; // 当前区间最大前缀和
12     int rmax; // 当前区间最大后缀和
13     int maxx; // 当前区间最大子段和
14     int sum; // 当前区间的和
15 }t[4 * maxn];
16
17 void push_up(int rt){
18     t[rt].sum = t[rt << 1].sum + t[rt << 1 | 1].sum;
19     // 当前区间的和: 左子树的和 + 右子树的和
20     t[rt].rmax = max(t[rt << 1 | 1].rmax, t[rt << 1 | 1].sum + t[rt << 1].rmax);
21     // 当前区间的最大后缀和: 右子树的最大后缀和 or 右子树的和 + 左子树的最大后缀和
22     t[rt].lmax = max(t[rt << 1].lmax, t[rt << 1].sum + t[rt << 1 | 1].lmax);
23     // 当前区间的最大前缀和: 左子树的最大前缀和 or 左子树的和 + 右子树的最大前缀和
24     t[rt].maxx = max(t[rt << 1].rmax + t[rt << 1 | 1].lmax, max(t[rt << 1].maxx, t[rt << 1 | 1].maxx));
25     // 当前区间的最大子段和: 左子树的最大子段和 or 右子树的最大子段和 or 左子树的最大后缀和 + 右子树的最大前缀和
26 }
27
28 void build(int rt, int l, int r){
29     if(l == r){
30         cin >> t[rt].maxx;
31         t[rt].lmax = t[rt].rmax = t[rt].sum = t[rt].maxx;
32         return;
33     }
34     int mid = l + r >> 1;
35     build(rt << 1, l, mid);
36     build(rt << 1 | 1, mid + 1, r);
37     push_up(rt);
38 }
39
40 void update(int rt, int l, int r, int x, int y){
41     if(l == r){
42         t[rt].lmax = t[rt].rmax = t[rt].maxx = t[rt].sum = y;
43         return;
44     }
45     int mid = l + r >> 1;
46     if(mid >= x) update(rt << 1, l, mid, x, y);
47     else update(rt << 1 | 1, mid + 1, r, x, y);
48     push_up(rt);
49 }
50
51 tree query(int rt, int l, int r, int x, int y){
52     if(l >= x && r <= y) return t[rt]; // 区间完全覆盖，直接返回该节点
53     int mid = l + r >> 1;
54     if(y <= mid) return query(rt << 1, l, mid, x, y); // 只在左区间，直接查询左区间
55     else if(x > mid) return query(rt << 1 | 1, mid + 1, r, x, y); // 只在右区间，直接查询右区间
56     else{
57         tree res_l = query(rt << 1, l, mid, x, y);
58         tree res_r = query(rt << 1 | 1, mid + 1, r, x, y);
59         tree res;
60         // res_l 记录左覆盖区间，res_r 记录右覆盖区间，合并后得到 res
61         // 用 push_up 同样的方式更新 res
62         res.sum = res_l.sum + res_r.sum;
63         res.lmax = max(res_l.sum + res_r.lmax, res_l.lmax);
64         res.rmax = max(res_r.rmax, res_r.sum + res_l.rmax);
65         res.maxx = max(max(res_l.maxx, res_r.maxx), res_l.rmax + res_r.lmax);

```



```

66     return res;
67 }
68 }
69
70 signed main(){
71     cin >> n >> m;
72     build(1, 1, n);
73     int opt, x, y;
74     while(m--){
75         cin >> opt >> x >> y;
76         if(opt == 1){
77             if(x > y) swap(x, y);
78             tree ans = query(1, 1, n, x, y);
79             cout << ans.maxx << '\n';
80         }
81         else update(1, 1, n, x, y);
82     }
83     return 0;
84 }

```

半群 (一次函数嵌套)

```

1  #include <bits/stdc++.h>
2  #define endl '\n'
3  #define int ll
4  using namespace std;
5  typedef long long ll;
6
7  const int N = 5e5 + 10;
8  const int mod = 998244353;
9  const int inf32 = 0x3f3f3f3f;
10 const ll inf64 = 4e18;
11
12 int n, m, K[N], b[N];
13
14 struct node{
15     int l, r;
16     ll mul, sum;
17 }t[N << 2];
18
19 node unite(node x, node y){
20     node ans;
21     ans.l = x.l, ans.r = y.r;
22     ans.mul = x.mul * y.mul % mod;
23     ans.sum = (x.sum * y.mul % mod + y.sum) % mod;
24     return ans;
25 }
26
27 void push_up(int u){
28     t[u] = unite(t[u << 1], t[u << 1 | 1]);
29 }
30
31 void build(int u, int l, int r){
32     if (l == r){
33         t[u].l = t[u].r = l;
34         t[u].sum = b[l];
35         t[u].mul = K[l];
36         return;
37     }
38     int mid = l + r >> 1;
39     build(u << 1, l, mid);
40     build(u << 1 | 1, mid + 1, r);
41     push_up(u);
42 }
43
44 void modify(int u, int x, int K, int B){
45     if (t[u].l == t[u].r)
46     {
47         t[u].sum = B;
48         t[u].mul = K;
49         return;

```

```

50     }
51     int mid = t[u].l + t[u].r >> 1;
52     if (x <= mid) modify(u << 1, x, K, B);
53     else modify(u << 1 | 1, x, K, B);
54     push_up(u);
55 }
56
57 node query(int u, int l, int r){
58     if (t[u].l == l && t[u].r == r) return t[u];
59     int mid = t[u].l + t[u].r >> 1;
60     if (r <= mid) return query(u << 1, l, r);
61     else if (l > mid) return query(u << 1 | 1, l, r);
62     else return unite(query(u << 1, l, mid), query(u << 1 | 1, mid + 1, r));
63 }
64
65 void solve(){
66     cin >> n >> m;
67     for (int i = 1; i <= n; ++i) cin >> K[i] >> b[i];
68     build(1, 1, n);
69     for (int i = 1; i <= m; ++i){
70         int op;
71         cin >> op;
72         if (op == 0){
73             int p, c, d;
74             cin >> p >> c >> d;
75             p++;
76             modify(1, p, c, d);
77         }else{
78             int l, r, x;
79             cin >> l >> r >> x;
80             l++;
81             auto ans = query(1, l, r);
82             cout << (ans.mul * x % mod + ans.sum) % mod << endl;
83         }
84     }
85 }
86
87 signed main(){
88     ios::sync_with_stdio(false);
89     cin.tie(nullptr);
90     int t = 1;
91     //cin >> t;
92     while(t--) solve();
93     return 0;
94 }

```

树上半群修改, 路径查询

```

1  #include <bits/stdc++.h>
2  #define endl '\n'
3  using namespace std;
4  typedef long long ll;
5
6  const int N = 2e5 + 10;
7  const int mod = 998244353;
8  const int inf32 = 0x3f3f3f3f;
9  const ll inf64 = 4e18;
10
11 struct Info {
12     int a1, b1, a2, b2;
13 };
14
15 Info operator + (const Info &l, const Info &r) {
16     Info ret;
17     //信息合并
18     ret.a1 = (1LL * l.a1 * r.a1 % mod); //向下的
19     ret.b1 = (1LL * r.a1 * l.b1 % mod + r.b1) % mod;
20     ret.a2 = (1LL * l.a2 * r.a2 % mod); // 向上的
21     ret.b2 = (1LL * l.a2 * r.b2 % mod + l.b2) % mod;
22     return ret;
23 }

```

```

24
25 struct node {
26     int a, b;
27 };
28
29 node operator + (const node &l , const node &r) {
30     node ret;
31     //信息合并
32     ret.a = (1LL * l.a * r.a % mod);
33     ret.b = (1LL * r.a * l.b % mod + r.b) % mod;
34     return ret;
35 }
36
37 Info t[N << 2];
38 int a[N], b[N], sz[N], top[N], dep[N], in[N], dfn[N], f[N];
39
40 struct Segment{
41     int n;
42
43     void push_up(int u) {
44         t[u] = t[u << 1] + t[u << 1 | 1];
45     }
46
47     void build(int u, int l, int r){
48         if (l == r){
49             t[u].a1 = t[u].a2 = a[dfn[l]];
50             t[u].b1 = t[u].b2 = b[dfn[l]];
51             return;
52         }
53         int mid = l + r >> 1;
54         build(u << 1, l, mid);
55         build(u << 1 | 1, mid + 1, r);
56         push_up(u);
57     }
58
59     void modify(int u, int l, int r, int p, const Info &v){
60         if (l == r){
61             t[u] = v;
62             return;
63         }
64         int mid = l + r >> 1;
65         if (p <= mid) modify(u << 1, l, mid, p, v);
66         else modify(u << 1 | 1, mid + 1, r, p, v);
67         push_up(u);
68     }
69
70     void modify(int u, int l, int r, int ql, int qr, const Info &v){
71         if (l == ql && r == qr){
72             t[u] = v;
73             return;
74         }
75         int mid = l + r >> 1;
76         if (qr <= mid) modify(u << 1, l, mid, ql, qr, v);
77         else if (ql > mid) modify(u << 1 | 1, mid + 1, r, ql, qr, v);
78         else {
79             modify(u << 1, l, mid, ql, mid, v);
80             modify(u << 1 | 1, mid + 1, r, mid + 1, qr, v);
81         }
82         push_up(u);
83     }
84
85     Info query(int rt, int l, int r, int ql, int qr) {
86         if (l == ql && r == qr) {
87             return t[rt];
88         }
89         int mid = l + r >> 1;
90         if (qr <= mid) return query(rt << 1, l, mid, ql, qr);
91         else if (ql > mid) return query(rt << 1 | 1, mid + 1, r, ql, qr);
92         else {
93             return query(rt << 1, l, mid, ql, mid) + query(rt << 1 | 1, mid + 1, r, mid + 1, qr);
94         }

```

```

95     }
96
97     Info query(int ql, int qr) {
98         if (ql > qr){
99             return Info {1, 0, 1, 0};
100         }
101         return query(1, 1, n, ql, qr);
102     }
103 }tree;
104
105 void solve(){
106     int n, q;
107     cin >> n >> q;
108     for (int i = 1; i <= n; ++i){
109         cin >> a[i] >> b[i];
110     }
111     vector<vector<int>> G(n + 1);
112     for (int i = 1; i < n; ++i){
113         int u, v;
114         cin >> u >> v;
115         u++, v++;
116         G[u].push_back(v);
117         G[v].push_back(u);
118     }
119     auto dfs1 = [&](auto self, int u, int fa) -> void {
120         if (fa) G[u].erase(find(G[u].begin(), G[u].end(), fa));
121         sz[u]++;
122         for (auto &v : G[u]){
123             dep[v] = dep[u] + 1;
124             f[v] = u;
125             self(self, v, u);
126             sz[u] += sz[v];
127             if (sz[v] > sz[G[u][0]]){
128                 swap(G[u][0], v);
129             }
130         }
131     };
132
133     int tot = 0 ;
134     auto dfs2 = [&](auto self, int u, int fa) -> void {
135         in[u] = ++tot;
136         dfn[in[u]] = u;
137         for (auto &v : G[u]){
138             top[v] = (v == G[u][0] ? top[u] : v);
139             self(self, v, u);
140         }
141     };
142
143     dep[1] = 0;
144     dfs1(dfs1, 1, 0);
145     top[1] = 0;
146     dfs2(dfs2, 1, 0);
147     tree.n = n;
148     tree.build(1, 1, n);
149
150     // 向上的
151     auto path1 = [&](auto self, int x, int y) -> node{
152         node ans = {1, 0};
153         while(top[x] != top[y]){
154             if (dep[top[x]] < dep[top[y]]) std::swap(x, y);
155             Info res = tree.query(in[top[x]], in[x]);
156             ans = (ans + node{res.a2, res.b2});
157             x = f[top[x]];
158         }
159         if (dep[x] > dep[y]) std::swap(x, y);
160         Info res = tree.query(in[x] + 1, in[y]);
161         ans = (ans + node{res.a2, res.b2});
162         return ans;
163     };
164
165     auto path2 = [&](auto self, int x, int y) -> node{

```

```

166     node ans = {1, 0};
167     while(top[x] != top[y]){
168         if (dep[top[x]] < dep[top[y]]) std::swap(x, y);
169         Info res = tree.query(in[top[x]], in[x]);
170         ans = (node{res.a1, res.b1} + ans);
171         x = f[top[x]];
172     }
173     if (dep[x] > dep[y]) std::swap(x, y);
174     Info res = tree.query(in[x], in[y]);
175     ans = (node{res.a1, res.b1} + ans);
176     return ans;
177 };
178
179 auto lca = [&](auto self, int u, int v) -> int{
180     while (top[u] != top[v]) {
181         if (dep[top[u]] > dep[top[v]]) {
182             u = f[top[u]];
183         } else {
184             v = f[top[v]];
185         }
186     }
187     return dep[u] < dep[v] ? u : v;
188 };
189
190 while(q--){
191     int op, l, r, x;
192     cin >> op >> l >> r >> x;
193     l++;
194     if (op == 0){
195         tree.modify(1, 1, n, in[l], Info{r, x, r, x});
196     }else{
197         r++;
198         int lc = lca(lca, l, r);
199         node ansL = path1(path1, l, lc);
200         node ansR = path2(path2, lc, r);
201         int ans = (1ll * ansL.a * x + ansL.b) % mod;
202         ans = (1ll * ansR.a * ans + ansR.b) % mod;
203         cout << ans << endl;
204     }
205 }
206 }
207
208 signed main(){
209     ios::sync_with_stdio(false);
210     cin.tie(nullptr);
211     int t = 1;
212     //cin >> t;
213     while(t--) solve();
214     return 0;
215 }

```

动态开点, 权值线段树

```

1  #include <bits/stdc++.h>
2  #define endl '\n'
3
4  using ll = long long;
5
6  constexpr int N = 1e5 + 10;
7  constexpr int M = 1e7 + 10;
8  constexpr int mod = 998244353;
9
10 using namespace std;
11
12 struct SegmentTree{
13     int ls, rs;
14     int num;
15 }tr[M];
16
17 int n, q, cnt, np1[N], np2[N], f[N];
18

```

```

19 ll m, k, a[N];
20
21 void update(int u, ll l, ll r, ll p, int sum) {
22     tr[u].num = sum;
23     if (l == r) return;
24     ll mid = l + r >> 1;
25     if (p <= mid){
26         if (!tr[u].ls){
27             tr[u].ls = ++cnt;
28         }
29         update(tr[u].ls, l, mid, p, sum);
30     }else{
31         if (!tr[u].rs){
32             tr[u].rs = ++cnt;
33         }
34         update(tr[u].rs, mid + 1, r, p, sum);
35     }
36 }
37
38 int query(int u, ll l, ll r, ll ql, ll qr) {
39     if (ql <= l && r <= qr) return tr[u].num;
40     ll mid = l + r >> 1;
41     int ans = n + 2;
42     if (ql <= mid) {
43         ans = min(ans, query(tr[u].ls, l, mid, ql, qr));
44     }
45     if (qr > mid) {
46         ans = min(ans, query(tr[u].rs, mid + 1, r, ql, qr));
47     }
48     return ans;
49 }
50
51 void solve(){
52     cin >> n >> m >> k;
53     for (int i = 1; i <= n; ++i) cin >> a[i];
54     cnt = 1, tr[0].num = tr[1].num = n + 1;
55     for (int i = n; i >= 1; --i){
56         np1[i] = query(1, 1ll, m, a[i], min(m, a[i] + k));
57         np2[i] = query(1, 1ll, m, max(1ll, a[i] - k), a[i]);
58         update(1, 1ll, m, a[i], i);
59     }
60     cin >> q;
61     while (q--){
62         int l, r;
63         cin >> l >> r;
64         for (int i = l; i <= r; ++i) f[i] = 0;
65         int ans = 0;
66         for (int i = r; i >= l; --i){
67             f[i] = 1;
68             if (np1[i] <= r) f[i] = max(f[i], f[np1[i]] + 1);
69             if (np2[i] <= r) f[i] = max(f[i], f[np2[i]] + 1);
70             ans = max(ans, f[i]);
71         }
72         cout << (r - l + 1 - ans) << endl;
73     }
74     for (int i = 0; i <= cnt; ++i) tr[i].num = tr[i].ls = tr[i].rs = 0;
75 }
76
77 signed main(){
78     ios::sync_with_stdio(false);
79     cin.tie(nullptr);
80     int t = 1;
81     cin >> t;
82     while(t--) solve();
83     return 0;
84 }

```

左右端点限制的最大区间字段和

```

1 #include <bits/stdc++.h>
2 #define endl '\n'

```

```

3
4 using ll = long long;
5
6 constexpr int N = 1e5 + 10;
7 constexpr int mod = 998244353;
8
9 using namespace std;
10
11 int a[N];
12
13 struct Info{
14     int lmax, smax, rmax, sum;
15 };
16
17 Info merge(Info s1, Info s2) {
18     Info res;
19     res.lmax = max(s1.lmax, s1.sum + s2.lmax);
20     res.smax = max({s1.smax, s2.smax, s1.rmax + s2.lmax});
21     res.rmax = max(s2.rmax, s2.sum + s1.rmax);
22     res.sum = s1.sum + s2.sum;
23     return res;
24 }
25
26 struct Node {
27     int l, r;
28     Info dat;
29 }tr[N << 2];
30
31 void push_up(int u) {
32     tr[u].dat = merge(tr[u << 1].dat, tr[u << 1 | 1].dat);
33 }
34
35 void build(int u, int l, int r) {
36     tr[u].l = l, tr[u].r = r;
37     if (l == r) {
38         tr[u].dat.lmax = tr[u].dat.rmax = tr[u].dat.smax = tr[u].dat.sum = a[l];
39         return;
40     }
41     int mid = (l + r) >> 1;
42     build(u << 1, l, mid);
43     build(u << 1 | 1, mid + 1, r);
44     push_up(u);
45 }
46
47 Info query_sub(int u, int l, int r) {
48     if (l > r) {
49         return Info{0, 0, 0, 0};
50     }
51     if (tr[u].l == l && tr[u].r == r) {
52         return tr[u].dat;
53     }
54     int mid = (tr[u].l + tr[u].r) >> 1;
55     if (r <= mid) return query_sub(u << 1, l, r);
56     else if (l > mid) return query_sub(u << 1 | 1, l, r);
57     return merge(query_sub(u << 1, l, mid), query_sub(u << 1 | 1, mid + 1, r));
58 }
59
60 int query(int l1, int r1, int l2, int r2) {
61     if (r1 < l2) {
62         int tmp = query_sub(1, l1, r1).rmax;
63         tmp += query_sub(1, r1 + 1, l2 - 1).sum;
64         tmp += query_sub(1, l2, r2).lmax;
65         return tmp;
66     }
67     int ans = query_sub(1, l2, r1).smax;
68     if (l1 < l2) ans = max(ans, query_sub(1, l1, l2).rmax + query_sub(1, l2, r2).lmax - a[l2]);
69     if (r2 > r1) ans = max(ans, query_sub(1, l1, r1).rmax + query_sub(1, r1, r2).lmax - a[r1]);
70     return ans;
71 }
72
73 void solve(){

```

```

74     int n;
75     cin >> n;
76     for (int i = 1; i <= n; ++i) cin >> a[i];
77     build(1, 1, n);
78     int m;
79     cin >> m;
80     while (m--) {
81         int x1, x2, y1, y2;
82         cin >> x1 >> x2 >> y1 >> y2;
83         cout << query(x1, x2, y1, y2) << endl;
84     }
85 }
86
87 signed main(){
88     ios::sync_with_stdio(false);
89     cin.tie(nullptr);
90     int t = 1;
91     cin >> t;
92     while(t--) solve();
93     return 0;
94 }

```

染色 (树上路径颜色段数量)

11122211 -> 111 222 11

```

1  #include <bits/stdc++.h>
2  #define endl '\n'
3
4  using ll = long long;
5
6  constexpr int N = 2e5 + 10;
7  constexpr int mod = 998244353;
8
9  using namespace std;
10
11 int col[N];
12
13 vector<int> G[N];
14
15 struct Node {
16     int l, r;
17     int num, lc, rc;
18     int tag;
19 }tr[N << 2];
20
21 void build(int u, int l, int r) {
22     tr[u].l = l, tr[u].r = r, tr[u].num = 0;
23     if (l == r) return;
24     int mid = (l + r) >> 1;
25     build(u << 1, l, mid);
26     build(u << 1 | 1, mid + 1, r);
27 }
28
29 void push_up(int u) {
30     tr[u].lc = tr[u << 1].lc;
31     tr[u].rc = tr[u << 1 | 1].rc;
32     int tmp = tr[u << 1].num + tr[u << 1 | 1].num;
33     if (tr[u << 1].rc == tr[u << 1 | 1].lc) tmp--;
34     tr[u].num = tmp;
35 }
36
37 void push_down(int u) {
38     if (tr[u].tag) {
39         tr[u << 1].tag = tr[u << 1 | 1].tag = tr[u].tag;
40         tr[u << 1].num = tr[u << 1 | 1].num = 1;
41         tr[u << 1].lc = tr[u << 1].rc = tr[u << 1 | 1].lc = tr[u << 1 | 1].rc = tr[u].lc;
42         tr[u].tag = 0;
43     }
44 }
45

```



```

46 void update(int u, int l, int r, int v) {
47     if (tr[u].l == l && tr[u].r == r) {
48         tr[u].num = tr[u].tag = v;
49         tr[u].lc = tr[u].rc = v;
50         return;
51     }
52     push_down(u);
53     int mid = (tr[u].l + tr[u].r) >> 1;
54     if (r <= mid) update(u << 1, l, r, v);
55     else if (l > mid) update(u << 1 | 1, l, r, v);
56     else update(u << 1, l, mid, v), update(u << 1 | 1, mid + 1, r, v);
57     push_up(u);
58 }
59
60 int LC, RC;
61
62 int query(int u, int ql, int qr, int L, int R) {
63     if (tr[u].l == L) LC = tr[u].lc;
64     if (tr[u].r == R) RC = tr[u].rc;
65     if (tr[u].l == ql && tr[u].r == qr) {
66         return tr[u].num;
67     }
68     push_down(u);
69     int mid = (tr[u].l + tr[u].r) >> 1;
70     if (qr <= mid) return query(u << 1, ql, qr, L, R);
71     else if (ql > mid) return query(u << 1 | 1, ql, qr, L, R);
72     else {
73         int ans = query(u << 1, ql, mid, L, R) + query(u << 1 | 1, mid + 1, qr, L, R);
74         if (tr[u << 1].rc == tr[u << 1 | 1].lc) ans--;
75         return ans;
76     }
77 }
78
79
80 void solve(){
81     int n, m;
82     cin >> n >> m;
83     for (int i = 1; i <= n; ++i) cin >> col[i];
84     for (int i = 1; i < n; ++i) {
85         int u, v;
86         cin >> u >> v;
87         G[u].push_back(v);
88         G[v].push_back(u);
89     }
90
91     vector<int> sz(n + 1), dfn(n + 1), top(n + 1), dep(n + 1), son(n + 1), f(n + 1);
92     int tot = 0;
93     auto dfs1 = [&](auto self, int u, int fa) -> void {
94         sz[u] = 1; dep[u] = dep[fa] + 1;
95         f[u] = fa;
96         for (auto v : G[u]) {
97             if (v == fa) continue;
98             self(self, v, u);
99             sz[u] += sz[v];
100             if (sz[v] > sz[son[u]]) {
101                 son[u] = v;
102             }
103         }
104     };
105
106     auto dfs2 = [&](auto self, int u, int t) -> void {
107         dfn[u] = ++tot;
108         top[u] = t;
109         if (son[u]) {
110             self(self, son[u], top[u]);
111         }
112         for (auto v : G[u]) {
113             if (v == f[u] || v == son[u]) continue;
114             self(self, v, v);
115         }
116     };

```

```

117
118     dfs1(dfs1, 1, 0);
119     dfs2(dfs2, 1, 1);
120     build(1, 1, n);
121
122     for (int i = 1; i <= n; ++i) update(1, dfn[i], dfn[i], col[i]);
123
124     auto calc = [&](int op, int u, int v, int c) -> int {
125         int ans = 0;
126         if (op == 1) {
127             while (top[u] != top[v]) {
128                 if (dep[top[u]] < dep[top[v]]) swap(u, v);
129                 update(1, dfn[top[u]], dfn[u], c);
130                 u = f[top[u]];
131             }
132             if (dep[u] > dep[v]) swap(u, v);
133             update(1, dfn[u], dfn[v], c);
134         } else {
135             int ans1 = -1, ans2 = -1;
136             while (top[u] != top[v]) {
137                 if (dep[top[u]] < dep[top[v]]) swap(u, v), swap(ans1, ans2);
138                 ans += query(1, dfn[top[u]], dfn[u], dfn[top[u]], dfn[u]);
139                 if (RC == ans1) ans--;
140                 ans1 = LC;
141                 u = f[top[u]];
142             }
143             if (dep[u] < dep[v]) swap(u, v), swap(ans1, ans2);
144             ans += query(1, dfn[v], dfn[u], dfn[v], dfn[u]);
145             if (RC == ans1) ans--;
146             if (LC == ans2) ans--;
147         }
148         return ans;
149     };
150
151     while (m--) {
152         char op;
153         int x, y, v;
154         cin >> op;
155         if (op == 'C') {
156             cin >> x >> y >> v;
157             calc(1, x, y, v);
158         } else {
159             cin >> x >> y;
160             cout << calc(2, x, y, 0) << endl;
161         }
162     }
163 }
164
165 signed main(){
166     ios::sync_with_stdio(false);
167     cin.tie(nullptr);
168     int t = 1;
169     //cin >> t;
170     while(t--) solve();
171     return 0;
172 }

```

线段树区间排序，整体查询

```

1  #include <bits/stdc++.h>
2  #define endl '\n'
3
4  using ll = long long;
5
6  constexpr int N = 2e5 + 10;
7  constexpr int M = 27;
8  constexpr int mod = 998244353;
9
10 using namespace std;
11
12 string s;

```

```

13  int n, q;
14
15  struct Info {
16      int sum[M];
17  }empty_Info;
18
19  struct Node {
20      int l, r;
21      int tag[M];
22      Info res;
23  }tr[N << 2];
24
25  void push_down(int u) {
26      for (int i = 1; i <= 26; ++i) {
27          if (tr[u].tag[i]) {
28              for (int j = 1; j <= 26; ++j) {
29                  tr[u << 1].tag[j] = 0;
30                  tr[u << 1].res.sum[j] = 0;
31                  tr[u << 1 | 1].tag[j] = 0;
32                  tr[u << 1 | 1].res.sum[j] = 0;
33              }
34              tr[u << 1].tag[i] = 1;
35              tr[u << 1].res.sum[i] = tr[u << 1].r - tr[u << 1].l + 1;
36              tr[u << 1 | 1].tag[i] = 1;
37              tr[u << 1 | 1].res.sum[i] = tr[u << 1 | 1].r - tr[u << 1 | 1].l + 1;
38              tr[u].tag[i] = 0;
39              break;
40          }
41      }
42  }
43
44  Info merge(Info ls, Info rs) {
45      Info res = empty_Info;
46      for (int i = 1; i <= 26; ++i) {
47          res.sum[i] = ls.sum[i] + rs.sum[i];
48      }
49      return res;
50  }
51
52  void push_up(int u) {
53      tr[u].res = merge(tr[u << 1].res, tr[u << 1 | 1].res);
54  }
55
56  void build(int u, int l, int r) {
57      tr[u].l = l, tr[u].r = r;
58      if (l == r) {
59          tr[u].res.sum[s[l] - 'a' + 1] = 1;
60          return;
61      }
62      int mid = (l + r) >> 1;
63      build(u << 1, l, mid);
64      build(u << 1 | 1, mid + 1, r);
65      push_up(u);
66  }
67
68
69  Info query(int u, int ql, int qr) {
70      if (ql > qr) {
71          return empty_Info;
72      }
73      if (tr[u].l == ql && tr[u].r == qr) {
74          return tr[u].res;
75      }
76      push_down(u);
77      int mid = (tr[u].l + tr[u].r) >> 1;
78      if (qr <= mid) return query(u << 1, ql, qr);
79      else if (ql > mid) return query(u << 1 | 1, ql, qr);
80      return merge(query(u << 1, ql, mid), query(u << 1 | 1, mid + 1, qr));
81  }
82
83  void update(int u, int ql, int qr, int val) {

```

```

84     if (ql > qr) return;
85     if (tr[u].l == ql && tr[u].r == qr) {
86         for (int i = 1; i <= 26; ++i) {
87             tr[u].res.sum[i] = 0;
88             tr[u].tag[i] = 0;
89         }
90         tr[u].tag[val] = 1;
91         tr[u].res.sum[val] = tr[u].r - tr[u].l + 1;
92         return;
93     }
94     push_down(u);
95     int mid = (tr[u].l + tr[u].r) >> 1;
96     if (qr <= mid) update(u << 1, ql, qr, val);
97     else if (ql > mid) update(u << 1 | 1, ql, qr, val);
98     else {
99         update(u << 1, ql, mid, val);
100        update(u << 1 | 1, mid + 1, qr, val);
101    }
102    push_up(u);
103 }
104
105 void push_all(int u) {
106     if (tr[u].l == tr[u].r) return;
107     push_down(u);
108     push_all(u << 1);
109     push_all(u << 1 | 1);
110 }
111
112 void print(int u) {
113     if (tr[u].l == tr[u].r) {
114         for (int i = 1; i <= 26; ++i) {
115             if (tr[u].res.sum[i]) {
116                 cout << char('a' + i - 1);
117             }
118         }
119         return;
120     }
121     print(u << 1);
122     print(u << 1 | 1);
123 }
124
125 void solve(){
126     cin >> n >> q;
127     cin >> s; s = " " + s;
128     build(1, 1, n);
129     while (q--) {
130         int op, l, r;
131         cin >> l >> r >> op;
132         Info tmp = query(1, l, r);
133
134         // for (int i = 1; i <= 26; ++i) {
135         //     cerr << tmp.sum[i] << " \n"[i == 26];
136         // }
137         if (op == 1) {
138             for (int i = 1; i <= 26; ++i) {
139                 update(1, l, l + tmp.sum[i] - 1, i);
140                 l += tmp.sum[i];
141             }
142         } else {
143             for (int i = 26; i >= 1; --i) {
144                 update(1, l, l + tmp.sum[i] - 1, i);
145                 l += tmp.sum[i];
146             }
147         }
148     }
149     push_all(1);
150     print(1);
151     cout << endl;
152 }
153
154 signed main(){

```

```

155     ios::sync_with_stdio(false);
156     cin.tie(nullptr);
157     int t = 1;
158     //cin >> t;
159     while(t--) solve();
160     return 0;
161 }

```

线段树区间最小值数量

```

1  #include <bits/stdc++.h>
2  #define endl '\n'
3
4  using ll = long long;
5
6  constexpr int N = 2e5 + 10;
7  constexpr int mod = 998244353;
8  constexpr int inf = 1e9;
9
10 using namespace std;
11
12 struct Node{
13     ll sum; int len;
14     Node() : sum(0), len(1) {}
15     Node(ll sum, int len) : sum(sum), len(len) {a}
16 };
17
18 using Tag = ll;
19
20 Node operator + (const Node &a, const Node &b){
21
22     if(a.sum < b.sum) return a;
23     if(a.sum > b.sum) return b;
24     Node c;
25     c.sum = a.sum, c.len = a.len + b.len;
26     return c;
27 }
28
29 void apply(Node &x, Tag &a, Tag f){
30     x.sum += f;
31     a += f;
32 }
33
34 template<class Node, class Tag, class Merge = plus<Node>>
35 struct Tree{
36     int n;
37     Merge merge;
38     vector<Node> info;
39     vector<Tag> tag;
40     Tree(int n = 0) : n(n), merge(Merge()), info((n << 2) + 1), tag((n << 2) + 1){}
41     Tree(vector<Node> init) : Tree((int)init.size()){
42         function<void(int, int, int)> build = [&](int p, int l, int r){
43             if (l == r){
44                 info[p] = init[l - 1];
45                 return;
46             }
47             int m = (l + r) / 2;
48             build(2 * p, l, m);
49             build(2 * p + 1, m + 1, r);
50             push_up(p);
51         };
52         build(1, 1, n);
53     }
54     void push_up(int p){
55         info[p] = merge(info[2 * p], info[2 * p + 1]);
56     }
57     void apply(int p, const Tag &v){
58         ::apply(info[p], tag[p], v);
59     }
60     void push_down(int p){
61         apply(2 * p, tag[p]);

```

```

62     apply(2 * p + 1, tag[p]);
63     tag[p] = Tag();
64 }
65 void modify(int p, int l, int r, int x, const Node &v){
66     if (l == r){
67         info[p] = v;
68         return;
69     }
70     int m = (l + r) / 2;
71     push_down(p);
72     if (x <= m){
73         modify(2 * p, l, m, x, v);
74     }
75     else{
76         modify(2 * p + 1, m + 1, r, x, v);
77     }
78     push_up(p);
79 }
80 void modify(int p, const Node &v){
81     modify(1, 1, n, p, v);
82 }
83 Node query(int p, int l, int r, int x, int y){
84     if (l > y || r < x){
85         return Node(Inf,1);
86     }
87     if (l >= x && r <= y){
88         return info[p];
89     }
90     int m = (l + r) / 2;
91     push_down(p);
92     return merge(query(2 * p, l, m, x, y), query(2 * p + 1, m + 1, r, x, y));
93 }
94 Node query(int l, int r){
95     return query(1, 1, n, l, r);
96 }
97 void modify(int p, int l, int r, int x, int y, const Tag &v){
98     if (l > y || r < x){
99         return;
100     }
101     if (l >= x && r <= y){
102         apply(p, v);
103         return;
104     }
105     int m = (l + r) / 2;
106     push_down(p);
107     modify(2 * p, l, m, x, y, v);
108     modify(2 * p + 1, m + 1, r, x, y, v);
109     push_up(p);
110 }
111 void modify(int l, int r, const Tag &v){
112     return modify(1, 1, n, l, r, v);
113 }
114 };
115 using T = Tree<Node, Tag>;
116
117 void solve(){
118     int n, k;
119     cin >> n >> k;
120     vector<ll> a(n + 1), prek(n + 1), lst(n + 1);
121     map<ll, vector<int>> pos;
122     for (int i = 1; i <= n; ++i){
123         cin >> a[i];
124         if (pos[a[i]].size()) lst[i] = pos[a[i]].back();
125         pos[a[i]].push_back(i);
126     }
127     for (auto [x, v] : pos){
128         for (int i = k - 1; i < v.size(); ++i){
129             prek[v[i]] = v[i - k + 1];
130         }
131     }
132 }

```

```

133     ll ans = 0;
134     map<ll, int> mp;
135     vector<Node> init(n);
136     T tr(init);
137
138     for (int i = 1; i <= n; ++i){
139         if (mp[a[i]]){
140             if (prek[mp[a[i]]] < mp[a[i]]){
141                 tr.modify(prek[mp[a[i]]] + 1, mp[a[i]], -1);
142             }
143             if (lst[prek[mp[a[i]]]]){
144                 tr.modify(1, lst[prek[mp[a[i]]]], -1);
145             }
146         }
147
148         if (prek[i] < i){
149             tr.modify(prek[i] + 1, i, 1);
150         }
151         if (lst[prek[i]]){
152             tr.modify(1, lst[prek[i]], 1);
153         }
154         auto qry = tr.query(1, i);
155         if (!qry.sum) ans += qry.len;
156         mp[a[i]] = i;
157     }
158     cout << ans << endl;
159 }
160
161 signed main(){
162     ios::sync_with_stdio(false);
163     cin.tie(nullptr);
164     int t = 1;
165     cin >> t;
166     while(t--) solve();
167     return 0;
168 }

```

线段树区间最小 x 的 y 和

```

1  #include <bits/stdc++.h>
2  #define endl '\n'
3
4  using ll = long long;
5
6  constexpr int N = 2e5 + 10;
7  constexpr int mod = 998244353;
8
9  using namespace std;
10
11 struct Node {
12     ll sumtag, sumval;
13     ll tag;
14 };
15
16 struct LazySegmentTree {
17     Node merge(Node a, Node b) {
18         a.tag = b.tag = 0;
19         if (a.sumtag == b.sumtag) a.sumval = (a.sumval + b.sumval) % mod;
20         if (a.sumtag < b.sumtag) swap(a, b);
21         return a;
22     }
23
24     vector<Node> Info;
25
26     void init(int n = 2e5 + 10) {
27         Info.resize(4 * n);
28     }
29
30     void push_down(int u){
31         if (Info[u].tag) {
32             Info[u << 1].tag += Info[u].tag;

```

```

33         Info[u << 1].sumtag += Info[u].tag;
34         Info[u << 1 | 1].tag += Info[u].tag;
35         Info[u << 1 | 1].sumtag += Info[u].tag;
36         Info[u].tag = 0;
37     }
38 }
39
40 void push_up(int u) {
41     Info[u] = merge(Info[u << 1], Info[u << 1 | 1]);
42 }
43
44 void modify(int u, int l, int r, int ql, int qr, ll val){
45     if (ql <= l && r <= qr) {
46         Info[u].tag += val;
47         Info[u].sumtag += val;
48         return;
49     }
50     push_down(u);
51     int mid = (l + r) >> 1;
52     if (ql <= mid) modify(u << 1, l, mid, ql, qr, val);
53     if (qr > mid) modify(u << 1 | 1, mid + 1, r, ql, qr, val);
54     push_up(u);
55 }
56
57 void updateDp(int u, int l, int r, int pos, ll val){
58     if (l == r) {
59         Info[u].sumval = val;
60         return;
61     }
62     push_down(u);
63     int mid = (l + r) >> 1;
64     if (pos <= mid) updateDp(u << 1, l, mid, pos, val);
65     else updateDp(u << 1 | 1, mid + 1, r, pos, val);
66     push_up(u);
67 }
68 };
69
70 void solve(){
71     int n, m;
72     cin >> n >> m;
73     LazySegmentTree Tr;
74     Tr.init();
75     vector<int> a(n + 1);
76     vector<vector<int>> pos(n + 1);
77     for (int i = 1; i <= n; ++i) {
78         cin >> a[i];
79         pos[i].push_back(0);
80     }
81     vector<int> p(m + 1);
82     vector<ll> dp(n + 1);
83     for (int i = 1; i <= m; ++i) cin >> p[i];
84     Tr.updateDp(1, 0, n, 0, 1);
85     for (int i = 1; i <= n; ++i) {
86         pos[a[i]].push_back(i);
87         int u = pos[a[i]].size() - 1;
88         for (int j = 1; j <= m; ++j){
89             if (u >= p[j]) Tr.modify(1, 0, n, pos[a[i]][u - p[j]], pos[a[i]][u - p[j] + 1] - 1, -1);
90             if (u >= p[j] + 1) Tr.modify(1, 0, n, pos[a[i]][u - p[j] - 1], pos[a[i]][u - p[j]] - 1, 1);
91         }
92         dp[i] = Tr.Info[1].sumval;
93         Tr.updateDp(1, 0, n, i, dp[i]);
94     }
95     cout << dp[n] << endl;
96 }
97
98 signed main(){
99     ios::sync_with_stdio(false);
100     cin.tie(nullptr);
101     int t = 1;
102     //cin >> t;
103     while(t--) solve();

```



```

104     return 0;
105 }

```

线性 dp

最长单峰子序列

```

1  #include <bits/stdc++.h>
2  #define endl '\n'
3
4  using ll = long long;
5
6  constexpr int N = 2e5 + 10;
7  constexpr int mod = 998244353;
8
9  using namespace std;
10
11 struct SegmentTree
12 {
13     ll tr[N << 2];
14
15     void push_up(int u) {
16         tr[u] = max(tr[u << 1], tr[u << 1 | 1]);
17     }
18
19     void modify(int u, int l, int r, int pos, ll val) {
20         if (l == r) {
21             tr[u] = val;
22             return;
23         }
24         int mid = (l + r) >> 1;
25         if (pos <= mid) modify(u << 1, l, mid, pos, val);
26         else modify(u << 1 | 1, mid + 1, r, pos, val);
27         push_up(u);
28     }
29
30     ll query_max(int u, int l, int r, int ql, int qr) {
31         if (ql <= l && r <= qr) return tr[u];
32         int mid = (l + r) >> 1;
33         ll ans = 0;
34         if (ql <= mid) {
35             ll tmp = query_max(u << 1, l, mid, ql, qr);
36             ans = max(ans, tmp);
37         }
38         if (qr > mid) {
39             ll tmp = query_max(u << 1 | 1, mid + 1, r, ql, qr);
40             ans = max(ans, tmp);
41         }
42         return ans;
43     }
44 }T0, T1;
45
46 ll a[N];
47
48 void solve(){
49     int n, m;
50     cin >> n >> m;
51     for (int i = 1; i <= n; ++i) {
52         int l, r;
53         cin >> l >> r;
54         a[l]++;
55         a[r + 1]--;
56     }
57     for (int i = 1; i <= m; ++i) a[i] += a[i - 1];
58     for (int i = 1; i <= m; ++i) {
59         ll v = T0.query_max(1, 0, n, 0, a[i]);
60         T0.modify(1, 0, n, a[i], v + 1);
61         T1.modify(1, 0, n, a[i], max(T1.query_max(1, 0, n, a[i], n), v) + 1);
62     }
63     cout << (T0.query_max(1, 0, n, 0, n), T1.query_max(1, 0, n, 0, n)) << endl;
64 }

```

```
65
66 signed main(){
67     ios::sync_with_stdio(false);
68     cin.tie(nullptr);
69     int t = 1;
70     //cin >> t;
71     while(t--) solve();
72     return 0;
73 }
```